

Symbol

From NEM

Symbol from NEM

技術リファレンス

Version 0.10.0.4

May 9, 2026

Contents

序文	vi
表記規則	vii
1 はじめに	1
1.1 ネットワーク・フィンガープリント	2
2 システム	3
2.1 トランザクションプラグイン	3
2.2 Symbol エクステンション	4
2.3 サーバー	6
2.3.1 キャッシュデータベース	7
2.4 ブローカー	8
2.5 リカバリー	9
2.6 一般的なトポロジー	9
3 暗号	11
3.1 公開鍵/秘密鍵ペア	11
3.2 署名と検証	12
3.2.1 バッチ検証	12
3.3 検証可能ランダム関数 (VRF)	13
3.4 投票鍵リスト	14
3.4.1 署名	15
4 木構造	16
4.1 マークル木	16
4.2 パトリシア木	17
4.3 マークルパトリシア木	19
4.4 マークルパトリシア木証明	20
5 アカウントとアドレス	22

5.1	アドレス	22
5.1.1	アドレス導出	23
5.1.2	アドレスエイリアス	25
5.1.3	意図的なアドレス衝突	25
5.2	公開鍵	25
6	トランザクション	27
6.1	基本トランザクション	27
6.2	アグリゲートトランザクション	29
6.2.1	埋め込みトランザクション	30
6.2.2	コサインネチャ	31
6.2.3	拡張レイアウト	33
6.3	トランザクションハッシュ	33
7	ブロック	35
7.1	ブロックフィールド	35
7.1.1	インポートانسブロックフィールド	38
7.2	レシート	38
7.2.1	レシートソース	39
7.2.2	トランザクションステートメント	39
7.2.3	解決ステートメント	40
7.2.4	レシートハッシュ	41
7.3	状態ハッシュ	42
7.4	拡張レイアウト	42
7.5	ブロックハッシュ	43
8	ブロックチェーン	44
8.1	ブロック難易度	44
8.2	ブロックスコア	45
8.3	ブロック生成	45
8.4	ブロック生成ハッシュ	48
8.5	ブロックヒットとターゲット	48

8.6	自動委任ハーベスター検出	50
8.7	ブロックチェーン同期	52
8.8	ブロックチェーン処理	54
9	ディスラプター	56
9.1	コンシューマー	57
9.1.1	共通コンシューマー	58
9.1.2	追加ブロックコンシューマー	60
9.1.3	追加トランザクションコンシューマー	61
10	未確認トランザクション	62
10.1	未確認トランザクションキャッシュ	63
10.2	スパムスロットル	63
11	パーシャルトランザクション	66
11.1	パーシャルトランザクション処理	67
12	ネットワーク	70
12.1	ビーコンノード	70
12.2	接続ハンドシェイク	71
12.3	パケット	72
12.4	接続種別	72
12.5	ピアプロベナンス	73
12.6	ノード探索	75
13	レピュテーション	76
13.1	接続管理	76
13.2	重み付きノード選択	77
13.3	ノードバン	78
14	コンセンサス	81
14.1	重み付けアルゴリズム	82
14.2	シビル攻撃	84

14.3	ステークなし攻撃	86
14.4	手数料攻撃	88
15	ファイナライゼーション	92
15.1	高レベルの概要	93
15.2	ラウンド	95
15.3	投票者	96
15.4	メッセージ	96
15.5	アルゴリズム	97
15.5.1	Prevote	98
15.5.2	Precommit	99
15.5.3	Commit	99
15.6	証明	99
15.7	シビル攻撃	101
15.8	ステークなし攻撃	101
15.9	例	101
16	時刻同期	104
16.1	サンプル収集	104
16.2	不良データを除去するためのフィルタリング	105
16.3	有効オフセットの計算	106
16.4	結合と閾値	106
17	メッセージング	107
17.1	メッセージチャンネルとトピック	107
17.2	接続とサブスクリプション	108
17.3	ブロックメッセージ	108
17.4	トランザクションメッセージ	110
17.4.1	コサインチャメッセージ	113
	References	115
	Index	116

序文

“

打たないシュートは 100% 外れる。

”

- Wayne Gretzky

NEMは、ビットコインのフォーラムスレッドへの「参加者募集」という形で、2014 年 1 月にひっそりと始まった。暗号資産空間は 2013 年末にブームを迎えていた——数年後に達するレベルには遠く及ばなかったが——空間全体に大きな熱気があった。NXT が最初の PoS ブロックチェーンの一つとしてちょうど立ち上がったばかりで、初期 NEM コミュニティの多くが NXT コミュニティから刺激を受け、そちらとのつながりを持っていた。これには残る 3 名のコア開発者全員も含まれる。

何を構築するかについて最初はいくつかの議論があったが、私たちは早い段階でゼロから新しいものを作ることを決めた。これにより設計の柔軟性が高まるとともに、最初から高いコーディング水準を採用することができた。また、ブロックチェーン界に新たなものを貢献する機会も得られた。多大な努力——ほとんどは夜間と週末——の末、これは 2015 年 3 月の NIS1 メインネット公開という形で実を結んだ。自分たちが構築したものには満足していたが、いくつかの近道を取ったことも自覚しており、改善を続けた。やがて、パフォーマンスのコアとなるボトルネックを解消し、将来のより迅速なイノベーションを可能にするためには、元のソリューションを再設計する必要があるという結論に達した。

ゼロからまったく新しいチェーン——**Symbol**——を構築するための支援を提供してくれた Tech Bureau に感謝する。これが NIS1 に内在する多くの問題を解決し、将来の改善・拡張のための堅固な基盤を提供することを願っている。私たちの使命は、決定論的ファイナライゼーションのオプションサポートを持つ、高性能な * ブロックチェーン * を構築することだった。その点において、私たちは成功したと考えている。

これは私たちにとって長い旅であったが、これからまだ何が訪れるか、そしてあなたが**Symbol** を使って何を生み出すかを見るのが楽しみだ。改めて、貢献者たちと、私たちに刺激を与えてくれた多くの方々に感謝を申し上げる...

BloodyRookie gimre Jaguar0625

表記規則

説明	属性	例
プロジェクト名	太字、色付き	Symbol
クラス名	固定幅フォント	ProcessBootstrapper
フィールド名	固定幅フォント	FeeMultiplier
ファイルパス	固定幅フォント	commit_step.dat
設定項目（設定ファイル名プレフィックス付き）テキストモード	イタリック	<i>network:maxDifficultyBlocks</i>
設定項目（設定ファイル名なし）数式・数式内	通常体	maxDifficultyBlocks
数式・数式内のフィールド	通常体	$T::\text{SignerPublicKey}$
数式・数式内の関数名	通常体	$\text{VerifiableDataBuffer}()$

Table 1: この文書で使用する表記規則

1 はじめに

“

灰の中から火が目覚め、
影の中から光が湧き出る；
折れた剣は再び甦り、
冠なき者はまた王となる。

”

- J.R.R. Tolkien

TRUSTLESS（信頼不要）、高パフォーマンス、レイヤードアーキテクチャ、ブロックチェーンベースの DLT プロトコル——これらが **Symbol** の開発に影響を与えた第一原則である。いくつかの DLT プロトコルが検討されたが、トラストレス性の理想に最も忠実であるとして、ブロックチェーンプロトコルが早い段階で選択された。どのノードも完全なチェーンのコピーをダウンロードし、常に独立して検証できる。十分なハーベスティング能力を持つノードは常にブロックを作成でき、リーダーに依存する必要がない。これらの選択は他のプロトコルと比べてスループットをある程度犠牲にするが、Bitcoin[Nak09] の哲学的基盤と最も整合していると考えられる。

Symbol は確率論的ファイナライゼーションと決定論的ファイナライゼーションの両方をサポートする。確率論的ファイナライゼーションでは、特定のブロックがロールバックされる確率は、より多くのブロックがそのブロックに追加——すなわちチェーン化——されるにつれて減少する。確率は極めて小さくなることはあっても、常にゼロではない¹。これに対し、決定論的ファイナライゼーションはプロトコルにチェックポイントを設定するメカニズムを含み、チェックポイントは決してロールバックされない。これにより深いロールバックが発生する可能性があるが、より強い保証を提供する。いずれの場合も、ブロックのロールバックとトランザクション無効化のリスクはユーザーが負う。

トラストレス性への注力の一環として、NEM に対して以下の機能が追加された：

- トランザクションデータなしにブロックヘッダーを同期でき、チェーンの整合性を検証できる。
- トランザクションのマークルツリーにより、ブロック内のトランザクションの存在（または非存在）を暗号的に証明できる。
- レシートにより、間接的にトリガーされる状態変化の透明性が高まる。

¹確率論的ファイナライゼーションが有効な場合、パフォーマンス上の理由から、最大 `network: maxRollbackBlocks` ブロックまでのロールバックが許可される。それより古いブロックはファイナライズされたと見なされるが、ロールバックの確率が非常に低いため保証はされない。

-
- ・ 状態証明により、ブロックチェーン内の特定の状態をトラストレスに検証できる。

Symbol では、異なるプラグイン（トランザクションサポート用）とエクステンション（機能用）を読み込むことでカスタマイズできる単一のサーバー実行ファイルが存在する。ネットワークごとに 3 つの主要な設定があるが、特定のエクステンションを有効化または無効化することで、さらにカスタマイズされたハイブリッド設定も可能である。

3 つの主要な設定は以下のとおりである：

1. ピア: これらのノードはネットワークのバックボーンであり、新しいブロックを作成する。
2. API: これらのノードは MongoDB データベースにデータを保存してより簡単なクエリを可能にし、NodeJS REST サーバーと組み合わせて使用できる。
3. デュアル: これらのノードはピアノードと API ノードの両方の機能を果たす。

強固なネットワークは通常、多数のピアノードと、クライアントからの要求をサポートするのに十分な API ノードを持つ。実際のニーズに基づいてノードの構成を動的に変化させることで、より最適化されたネットワーク全体のリソース利用が期待できる。

コアのブロックおよびトランザクション処理パイプラインをディスラプターパターン上に構築し、可能な限り並列処理を使用することで、一般的なブロックチェーンプロトコルと比較して高い毎秒トランザクション数を実現する。

NIS1 はブロックチェーン界への優れた参入であり、**Symbol** はそのさらなる進化である。これは終わりではなく、新たな始まりである。まだやるべきことがある。

1.1 ネットワーク・フィンガープリント

各ネットワークは以下で構成される固有のフィンガープリントを持つ：

1. ネットワーク識別子—ネットワーク間で共有できる 1 バイトの識別子。ネットワークと互換性のあるすべてのアドレスは、この値を最初のバイトとして持つ必要がある。
2. 生成ハッシュシード—ネットワーク間でグローバルに一意でなければならないランダムな 32 バイト値。クロスネットワークのリプレイ攻撃を防ぐため、この値はハッシュ化または署名の前にトランザクションデータに付加される。

2 システム

((

新しい技術は社会を変革できるが、その技術がまだ萌芽期にある時、その潜在力の全容を見通せる人はごく少数だ。))

- Liu Cixin

SYMBOL はネットワークレベルと個々のノードレベルの両方で高いカスタマイズをサポートする。*network* で指定されるネットワーク全体の設定は、ネットワーク内のすべてのノードで同一でなければならない。一方、ノード固有の設定はネットワーク内のすべてのノードで異なることができ、*node* に配置される。

Symbol はチューリング完全なスマートコントラクトをサポートする代わりに、プラグイン/エクステンションアプローチを使用するように設計された。後者はユーザーの柔軟性を高めることができるが、ユーザーの観点からはエラーも多くなる。プラグインモデルはブロックチェーン上で実行できる操作を制限し、その結果として攻撃対象領域が小さくなる。さらに、無限のセットよりも有限のセットのパフォーマンスを最適化する方がはるかに簡単である。これにより **Symbol** は設計された高いスループットを達成できる。

2.1 トランザクションプラグイン

ネットワーク内のすべてのノードは同じ種類のトランザクションをサポートし、すべてのノードがグローバルなブロックチェーン状態に合意できるよう全く同じ方法で処理しなければならない。ネットワークは各ノードに一連のトランザクションプラグインを読み込むことを要求し、このセットがネットワークがサポートするトランザクションの種類を決定する。このセットは *network* 設定における *network:plugin** セクションの存在によって決定される。これらのプラグインへの変更、追加、削除はすべてのネットワークノードによって調整・受け入れられなければならない。これらの変更の一部のノードしか同意しない場合、それらのノードはフォーク上にあることになる。すべての組み込み **Symbol** トランザクションは、その拡張性を検証するためにこのプラグインモデルを使用して構築されている。

プラグインは以下の形式で単一のエントリポイントを公開する別個の動的リンクライブラリである ²：

```
extern "C" PLUGIN_API
void RegisterSubsystem(
```

²プラグインの形式はターゲットのオペレーティングシステムと使用するコンパイラによって異なるため、すべてのホストアプリケーションとプラグインは同じコンパイラバージョンとオプションでビルドする必要がある。

```
catapult::plugins::PluginManager& manager);
```

PluginManager はプラグインが初期化するために必要な設定のサブセットへのアクセスを提供する。このクラスを通じて、プラグインは以下のゼロ個以上のものを登録できる：

1. トランザクション—新しいトランザクション種別とそれらの種別から解析ルールへのマッピングを指定できる。具体的には、プラグインはトランザクションをさらなる処理に使用されるコンポーネント通知に変換するルールを定義する。処理制約を少数指定することもでき、例えばトランザクションがアグリゲートトランザクション（[6.2: アグリゲートトランザクション](#) を参照）内にのみ現れることができることを示す。
2. キャッシュ—新しいキャッシュ種別とモデル種別のバイナリへのシリアライズおよびデシリアライズのルールを指定できる。各状態関連キャッシュは、その機能が有効な場合にブロックの StateHash（[7.3: 状態ハッシュ](#) を参照）の計算にオプションで含めることができる。
3. ハンドラー—これらは常にアクセス可能な API である。
4. ダイアグノスティクス—これらはノードが診断モードで実行されている場合にのみアクセス可能な API とカウンターである。
5. バリデーター—ステートレスおよびステートフルバリデーターはブロックおよびトランザクション処理によって生成された通知を処理する。登録されたバリデーターは一般的またはプラグイン定義の通知を購読し、許可されない値や状態変化を拒否できる。
6. オブザーバー—オブザーバーはブロックおよびトランザクション処理によって生成された通知を処理する。登録されたオブザーバーは一般的またはプラグイン定義の通知を購読し、その値に基づいてブロックチェーン状態を更新できる。オブザーバーは適用可能なすべてのバリデーターが成功した後にのみ呼び出されるため、検証ロジックを必要としない。
7. リゾルバー—未解決から解決済み種別へのカスタムマッピングを指定できる。例えば、これはネームスペースプラグインがエイリアス解決のサポートを追加するために使用される。

2.2 Symbol エクステンション

ネットワーク内の個々のノードは異種混在の機能をサポートできる。例えば、一部のノードはデータをデータベースに保存したり、イベントをメッセージキューに公開し

たりする場合がある。これらの機能はいずれもコンセンサスに影響しないため、すべてオプションである。そのような機能は、`extensions- $\{process\}$ :extensions` で指定されたノードが読み込む エクステンションのセットによって決定される。ほとんどの組み込み **Symbol** 機能は、その拡張性を検証するためにこのエクステンションモデルを使用して構築されている。

エクステンションは以下の形式で単一のエントリポイントを公開する別個の動的リンクライブラリである³：

```
extern "C" PLUGIN_API
void RegisterExtension(
    catapult::extensions::ProcessBootstrapper& bootstrapper);
```

`ProcessBootstrapper` はエクステンションが初期化するために必要な完全な **Symbol** 設定とサービスへのアクセスを提供する。この追加アクセスを提供することで、エクステンションはプラグインよりも強力になれる。このクラスを通じて、エクステンションは以下のゼロ個以上のものを登録できる：

1. サービス—サービスは独立した動作を表す。サービスには実行ファイルの状態を表すオブジェクトが渡され、それを使用して多数のものを設定できる。特に、サービスはダイアグノスティクスカウンターを追加し、API（ダイアグノスティクスと非ダイアグノスティクスの両方）を定義し、タスクスケジューラーにタスクを追加できる。また、依存サービスを作成し、そのライフタイムをホストする実行ファイルのものに結びつけることもできる。サービスができることに対する制限はほとんどなく、大幅なカスタマイズの可能性を持つ。
2. サブスクリプション—エクステンションはサポートされた任意のブロックチェーンイベントを購読できる。変更が検出されるとイベントが発生する。ブロック、状態、未確認トランザクション、およびパーシャルトランザクション変更イベントがサポートされる。トランザクションステータスイベントはトランザクションの処理が完了すると発生する。ノードイベントはリモートノードが発見されると発生する。

上記に加えて、エクステンションはノードをより複雑な方法で設定できる。例えば、エクステンションはカスタムネットワーク時刻サプライヤーを登録できる。実際、[16: 時刻同期](#) で説明したアルゴリズムに基づいて時刻サプライヤーを設定する特殊なエクステンションがある。これはこのエクステンションモデルによって可能な高レベルのカスタマイズの例である。エクステンションによって可能な拡張性の全範囲を理解するには、プロジェクトコードまたは開発者ドキュメントを参照のこと⁴。

³エクステンションの形式はターゲットのオペレーティングシステムと使用するコンパイラによって異なるため、すべてのホストアプリケーションとプラグインは同じコンパイラバージョンとオプションでビルドする必要がある。

⁴詳細は <https://nemtech.github.io/> を参照のこと。

2.3 サーバー

最もシンプルな **Symbol** トポロジーは単一のサーバー実行ファイルで構成される。ネットワークが必要とするトランザクションプラグインとノードオペレーターが必要とする **Symbol** エクステンションはサーバーによって読み込まれ初期化される。

Symbol はそのすべてのデータを data ディレクトリに保存する。データディレクトリの内容は以下の通りである：

1. ブロックバージョン管理ディレクトリ—これらのディレクトリは独自フォーマットのブロック情報を含む。各確認済みブロックのバイナリデータ、トランザクション、および関連データは .dat および .stmt 拡張子を持つファイルにこれらのディレクトリに保存される。各ブロックの処理時に生成されるステートメント（[7.2: レシート](#) を参照）も迅速なアクセスのためにここに保存される。バージョン管理ディレクトリの例は最初のブロックグループを含む 00000 である。hashes.dat にはブロック高からブロックハッシュへのマッピングが含まれる。

確定的ファイナライゼーションが有効な場合、これらのディレクトリには各ファイナライゼーションエポックの証明情報を含むバージョン管理 .proof ファイルも含まれる。

2. audit —監査コンシューマー（[9.1.1: 共通コンシューマー](#) を参照）によって作成された監査ファイルはこのディレクトリに保存される。
3. importance —各インポートانس再計算ポイントの重要アカウントに関する情報を含むバージョン管理ファイル。このディレクトリのファイルを使用して、任意のインポートانس再計算ポイントで重要アカウントのセットを再構成できる。これらのファイルは複数のインポートانس計算を元に戻す必要がある深いロールバックを可能にする。このディレクトリは確定的ファイナライゼーションが有効な場合にのみ作成される。
4. logs —ファイルベースのロギングが有効な場合に作成されるバージョン管理ログファイルはこのディレクトリに保存される。実行中のプロセスに関連するアクティブなログファイルはデータディレクトリに直接保存される。各ログファイルは通常、ソースプロセスの名前で始まる。
5. spool —サブスクリプション通知はこのディレクトリに書き出される。サーバーからブローカーへメッセージを渡すためのメッセージキューとして使用される。また、突然の終了の場合にデータを回復するためにリカバリプロセスによっても使用される。

-
6. `state` —**Symbol** はその独自ストレージファイルをこのディレクトリに保存する。`supplemental.dat` と `_summary.dat` で終わるファイルは要約データを保存する。`Cache.dat` で終わるファイルは完全なキャッシュデータを保存する。
 7. `statedb` —`node:enableCacheDatabaseStorage` が設定されている場合、このディレクトリには RocksDB ファイルが含まれる。
 8. `transfer_message` —`user:enableDelegatedHarvestersAutoDetection` が設定されている場合、このディレクトリには現在のノードへの抽出されたデリゲートハーベスティングリクエストが含まれる。
 9. `commit_step.dat` —これはサーバーによって開始されたコミットプロセスの最新ステップを保存する。これは主にリカバリ目的で使用される。
 10. `index.dat` —これはディスクに保存されたブロック数を含むカウンターである。
 11. `proof.index.dat` —これはディスクに保存されたファイナライゼーション証明の数を含むカウンターである。
 12. `voting_status.dat` —これはノードが送信した最後のファイナライゼーションメッセージに関する情報を保存する。

2.3.1 キャッシュデータベース

サーバーはキャッシュデータベースを使用するかどうかを選択できる。`node:enableCacheDatabaseStorage` が設定されている場合、RocksDB がキャッシュデータの保存に使用される。検証可能な状態（[7.3: 状態ハッシュ](#) を参照）はキャッシュデータベースを必要とし、ほとんどのネットワーク設定ではそれが有効化されていることが期待される。

キャッシュデータベースは以下のすべてが当てはまる場合にのみ無効化すべきである：

1. 毎秒高いトランザクションレートが望まれる場合。
2. キャッシュ状態のトラストレスな検証が重要でない場合。
3. サーバーが大量の RAM で設定されている場合。

このモードでは、すべてのキャッシュエントリは常にメモリに常駐する。シャットダウン時、キャッシュデータは複数のフラットファイルにディスクに書き込まれる。起動時、このデータが読み取られてメモリキャッシュの設定に使用される。

キャッシュデータベースが有効な場合、要約されたキャッシュデータが複数のフラットファイルにディスクに書き込まれる。この要約データはキャッシュに保存されたデータから導出可能である。一例として、少なくとも `network:minHarvesterBalance` の残高を持つすべての高価値アカウントのリストがある。このリストはアカウント状態キャッシュに保存されたすべてのアカウントを（再）検査することで生成できるが、最適化としてディスクに保存・ロードされる。

2.4 ブローカー

ブローカープロセスにより、並列化を犠牲にすることなく、より複雑な **Symbol** の動作を追加できる。ネットワークが必要とするトランザクションプラグインとノードオペレーターが必要とする **Symbol** エクステンションはブローカーによって読み込まれ初期化される。ブローカーはトランザクションプラグインのすべての機能をサポートするが、**Symbol** エクステンション機能のサブセットのみをサポートする。例えば、ブローカーでのネットワーク時刻サプライヤーのオーバーライドはサポートされていない。ブローカーエクステンションは主にサブスクライバーを登録し、それらのサブスクライバーに転送されたイベントに反応することを目的としている。したがって、サーバーとブローカーは異なるエクステンションを読み込むことが期待される。詳細はプロジェクトコードまたは開発者ドキュメントを参照のこと。

ブローカーは `spool` ディレクトリの変更を監視し、任意のイベント通知を読み込まれたエクステンションによって登録されたサブスクライバーに転送する。エクステンションはこれらのイベントを処理するためにサブスクライバーを登録する。例えば、データベースエクステンションはこれらのイベントを読み取り、グローバルなブロックチェーン状態を正確に反映するようにデータベースを更新するために使用できる。

`spool` ディレクトリは一方方向メッセージキューとして機能する。サーバーがメッセージを書き込み、ブローカーがそれを読み取る。ブローカーからサーバーへメッセージを送る方法はない。この分離は意図的であり、パフォーマンス上の理由から行われた。

サーバーはブロックチェーンデータへの排他ロックを保持している時に、ブロックチェーン同期コンシューマー（[9.1.2: 追加ブロックコンシューマー](#) を参照）でサブスクリプションイベントを発生させる。これらの操作はサーバーが排他ロックを保持している間の低速なデータベース操作を防ぐためにブローカーにオフロードされる。ブローカーが使用するデータのほとんどは、突然のサーバー終了後のデータ回復にも必要とされるため、サーバーのオーバーヘッドは最小限である。

2.5 リカバリー

リカバリープロセスは、突然のサーバーおよび/またはブローカーの終了後にグローバルなブロックチェーン状態を修復するために使用される。ネットワークが必要とするトランザクションプラグインとノードオペレーターが必要とする **Symbol** エクステンションはリカバリープロセスによって読み込まれ初期化される。ブローカーが使用される場合、リカバリープロセスはブローカーと同じエクステンションを読み込まなければならない。

具体的なリカバリー手順はプロセスの設定と `commit_step.dat` ファイルの値に依存する。一般的に、サーバーが状態変化をディスクにフラッシュした後に終了した場合、それらの変更は再適用される。ブロックチェーン状態はサーバーがそれらの変更を適用してコミットした場合と同じになる。それ以外の場合、サーバーが状態変化をディスクにフラッシュする前に終了した場合、保留中の変更は破棄される。ブロックチェーン状態はサーバーがそれらの変更の処理を試みたことがなかった場合と同じになる。

リカバリープロセスが完了した後、ブロックチェーン状態は突然終了したことの無いノードの状態と区別できないものとなるべきである。spool ディレクトリは修復・処理される。ディスクに保存されたブロックとキャッシュデータは調整・更新される。保留中の状態変化は、適用可能な場合に適用される。突然の終了の存在を示す他のファイルは更新または削除される。

2.6 一般的なトポロジー

ネットワークは多数の異種トポロジーで構成できるが、ほとんどのノードはピア、API、またはデュアルの 3 つのカテゴリのいずれかに属する可能性が高い。これらすべてのトポロジーで同じサーバープロセスが使用される。唯一の違いは各トポロジーが読み込むエクステンションである。

ピアノードは軽量なノードである。ブロックチェーンネットワークにセキュリティを追加するのに十分な機能を持つが、それ以上はほとんどない。他のノードと同期し、新しいブロックをハーベストできる。

API ノードはより重量級のノードである。他のノードと同期できるが、新しいブロックをハーベストすることはできない。ボンデッドアグリゲートトランザクションのホスティングと、それらを完成させるためのコサインエチャの収集をサポートする。MongoDB データベースにデータを書き込み、変更をパブリックメッセージキューを通じてサブスクリバに伝播するように設定されたブローカープロセスを必要とする。REST API はこれら両方の機能に依存しており、レイテンシを最小化するためにパフォーマンス上の理由から API ノードと同じ場所に配置されるのが一般的であ

る。

デュアルノードはピアノードと API ノードのスーパーセットである。両方のノード種別のすべての機能をサポートする。これらのノードはすべての API ノード機能をサポートするため、ブローカーも必要とする。

3 暗号

((

公開鍵暗号の重要性は原則として理解していたが、すべてが私の予想をはるかに上回る速さで進んだ。高度な通信技術の中心的な要素になるとは予想していなかった。))

- Whitfield Diffie

BLOCKCHAINブロックチェーン技術はいくつかの暗号学的概念の使用を必要とする。**Symbol** は楕円曲線暗号 (ECC) に基づく暗号を使用する。セキュリティと速度を保証するために、使用する曲線の選択は重要である。

Symbol は Ed25519 デジタル署名アルゴリズムを使用する。このアルゴリズムは以下の *Twisted Edwards* 曲線を使用する：

$$-x^2 + y^2 = 1 - \frac{121665}{121666}x^2y^2$$

素数 $2^{255} - 19$ によって定義される有限体上で定義される。対応するグループ G の基点は B と呼ばれる。グループは $q = 2^{252} + 27742317777372353535851937790883648493$ 個の要素を持つ。D. J. Bernstein らによって開発され、最も安全かつ高速なデジタル署名アルゴリズムの一つである [Ber+11]。

Symbol の目的において重要な点として、このアルゴリズムは短い 64 バイトの署名を生成し、高速な署名検証をサポートする。鍵生成も署名もブロック処理中には使用されないため、これらの操作の速度は重要ではない。

3.1 公開鍵/秘密鍵ペア

秘密鍵はランダムな 256 ビット整数 k である。そこから公開鍵 A を導出するには、以下の手順を行う：

$$H(k) = (h_0, h_1, \dots, h_{511}) \quad (1)$$

$$a = 2^{254} + \sum_{3 \leq i \leq 253} 2^i h_i \quad (2)$$

$$A = aB \quad (3)$$

A はグループ要素であるため、256 ビット整数 A にエンコードでき、これが公開鍵として機能する。

3.2 署名と検証

メッセージ M 、秘密鍵 k およびその対応する公開鍵 $\underline{A}$ が与えられた場合、署名を作成するには以下の手順を行う：

$$H(k) = (h_0, h_1, \dots, h_{511}) \quad (4)$$

$$r = H(h_{256}, \dots, h_{511}, M) \text{ ここでカンマは連結を意味する} \quad (5)$$

$$R = rB \quad (6)$$

$$S = (r + H(\underline{R}, \underline{A}, M)a) \bmod q \quad (7)$$

$(\underline{R}, \underline{S})$ が秘密鍵 k の下でのメッセージ M の署名である。署名の延性の問題を**防ぐため**、 $S < q$ かつ $S > 0$ の署名のみが有効とみなされることに注意。

メッセージ M と公開鍵 $\underline{A}$ に対して署名 $(\underline{R}, \underline{S})$ を検証するには、検証者は $S < q$ かつ $S > 0$ を確認してから以下を計算する：

$$\tilde{R} = SB - H(\underline{R}, \underline{A}, M)A$$

そして以下を検証する：

$$\tilde{R} = R \quad (8)$$

S が (7) のように計算されていれば、

$$SB = rB + (H(\underline{R}, \underline{A}, M)a)B = R + H(\underline{R}, \underline{A}, M)A$$

となるため、(8) が成立する。

3.2.1 バッチ検証

多数の署名を検証する必要がある場合、バッチ署名検証によって処理を約 80% 高速化できる。**Symbol** は [Ber+11] に概説されたアルゴリズムを使用する。 (R_i, S_i) がメッセージ M_i の公開鍵 A_i による署名である (M_i, A_i, R_i, S_i) のバッチが与えられた場合、 $H_i = H(R_i, A_i, M_i)$ とする。さらに、対応する数の一様分布の 128 ビット独立乱数 z_i が生成されると仮定する。以下の方程式を考える：

$$\left(- \sum_i z_i S_i \bmod q \right) B + \sum_i z_i R_i + \sum_i (z_i H_i \bmod q) A_i = 0 \quad (9)$$

$P_i = 8R_i + 8H_i A_i - 8S_i B$ とすると、(9) が成立する場合、以下が示唆される：

$$\sum_i z_i P_i = 0 \quad (10)$$

すべての P_i は巡回群の要素である (q は素数であることを思い出してほしい)。ある P_i (例えば P_2) がゼロでない場合、与えられた整数 $z_0, z_1, z_3, z_4 \dots$ に対して、(10) を満たす z_2 の選択はちょうど一つである。その確率は 2^{-128} である。したがって、(9) が成立する場合、すべての i に対して $P_i = 0$ である確率はほぼ確実である。これは署名が有効であることを示唆する。

(9) が成立しない場合、少なくとも一つの無効な署名が存在することを意味する。その場合、**Symbol** は無効な署名を特定するために単一署名検証にフォールバックする。

3.3 検証可能ランダム関数 (VRF)

検証可能ランダム関数 (VRF) は公開鍵/秘密鍵ペアを使用して擬似乱数値を生成する。秘密鍵の所有者のみが、攻撃者には事前に予測できない値を生成できる。公開鍵を持つ人なら誰でも、その値が対応する秘密鍵によって生成されたかどうかを検証できる。**Symbol** は [Gol+20] で定義された ECVRF-EDWARDS25519-SHA512-TAI を使用する。

公開鍵 Y に対応する秘密鍵 $SK = xB$ と入力シード $alpha^5$ が与えられた場合に証明⁶を生成するには：

$$\begin{aligned} H &= \text{map_to_group_element}(alpha, Y) \\ \gamma &= xH \\ k &= \text{generate_nonce}(H) \\ c &= \text{IetfHash}(3, 2, H, \gamma, kB, kH)[0..15] \\ s &= (k + cx) \mod q \\ proof &= (\gamma, c, s) \end{aligned}$$

上記の関数が生成した証明は以下の手順で検証できる：

$$\begin{aligned} H &= \text{map_to_group_element}(alpha, Y) \\ U &= sB - cY \\ V &= sH - c\gamma \\ verification_hash &= \text{IetfHash}(3, 2, H, \gamma, U, V)[0..15] \end{aligned}$$

⁵このセクションで提供されるリストは補助関数を定義していない。これらの関数の完全な説明は [Gol+20] に記載されている。

⁶これは通常、プルービング (proving) と呼ばれる。検証 (verifying) と混同しないように。秘密鍵の所有者が秘密鍵を使用してランダム値を生成したことを証明する必要があるためである。

計算した $verification_hash$ が証明の c の部分と一致する場合、ランダム値の検証は成功する。

検証済み証明の γ から、VRF ハッシュ出力とも呼ばれる証明ハッシュを導出できる：

$$proof_hash = \text{IetfHash}(3, 3, 8\gamma) \quad (11)$$

3.4 投票鍵リスト

ファイナライゼーション投票者は、ルート投票鍵が投票に参加できるファイナライゼーションエポックの範囲を指定する必要がある。ルート投票鍵をアナウンスする前に、投票者はエポックに固定された投票鍵を含む投票鍵リストを構築する必要がある。リストの構築は、簡略化された Bellare-Miner [BM99] 構造に大まかに基づいている。リスト内の各鍵は単一のエポックにバインドされ、ファイナライゼーションプロセスが次のエポックに進むと消去される。これにより前方秘匿性が提供される。攻撃者が投票鍵リストを入手しても、完了したエポックを変更または否認することはできない。

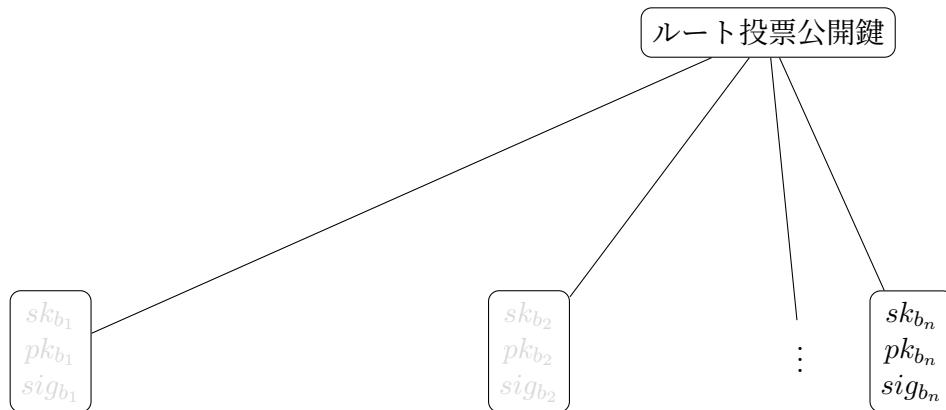


Figure 1: 投票鍵リスト

リストは投票鍵リンクトランザクションをアナウンスする前に完全に構築される。まず、ルート投票鍵ペアが生成される。ルート投票公開鍵は、投票鍵リンクトランザクションの一部としてアカウントの署名公開鍵で署名される。

次に、エポックバインドの鍵が生成される。各鍵ペアについて、ルート鍵ペアがエポックバインドの公開鍵とそれぞれのエポックを連結したものに署名する。すべてのエポックバインドの鍵が生成された後、ルート投票秘密鍵は破棄される。式において、 i はそれぞれのエポックを指す。

$$sig_{b_i} = \text{Sign}_{\text{ルート秘密鍵}}(pk_{b_i} || \text{IntToBin}(i))$$

特定のエポックの投票メッセージに署名する場合、メッセージ署名が作成される：

$$sig_{message-i} = \text{Sign}_{sk_{b_i}}(message)$$

3.4.1 署名

特定のエポックにおける投票の署名は 2 つのペアで構成される：

- (ルート投票公開鍵, sig_{b_i})
- (pk_{b_i} , $sig_{message-i}$)

投票鍵リスト署名は以下の場合に検証済みとみなされる：

- ルート投票公開鍵が指定されたエポックへの参加に登録されている。
- メッセージの署名者鍵がエポックバインドの鍵と一致する。
- すべてのコンポーネント署名が暗号的に検証される。

4 木構造

“

私たちの知恵はすべて木々の中に蓄えられている。

”

- Santosh Kalwar

SYMBOL は信頼性の低いライトクライアントをサポートするために木構造を使用する。マークル木はクライアントがその中に格納されたデータの存在を暗号的に確認することを可能にする。パトリシア木はクライアントがその中に格納されたデータの存在または非存在を暗号的に確認することを可能にする。

4.1 マークル木

マークル木 [Mer88] はハッシュの木であり、効率的な存在証明を可能にする。Symbol内では、すべての基本マークル木はバランス型かつ二分木に制約されている。各リーフノードはいくつかのデータのハッシュを含む。各非リーフノードは子ノードに格納されたハッシュをハッシュ化することで構築される。Symbolの実装では、任意の（ルート以外の）層が奇数のハッシュを含む場合、親ハッシュを計算する際に最後のハッシュが二重化される。

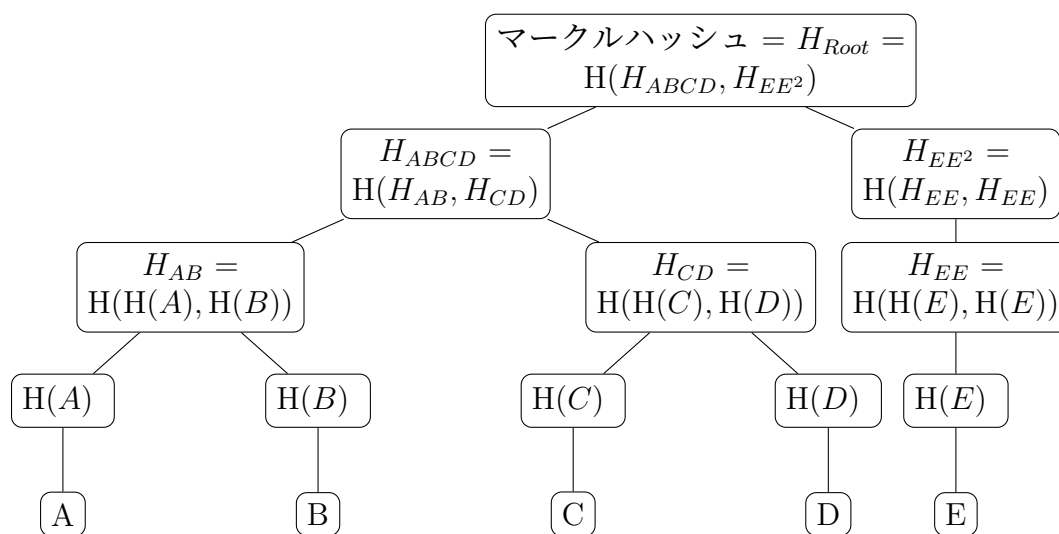


Figure 2: 5 つのデータアイテムで構成される 4 層マークル木

マークル木を使用する利点は、木内のハッシュの存在を $\log(N)$ 個のハッシュのみで証明できることである。これにより比較的低い帯域幅要件での存在証明が可能になる。

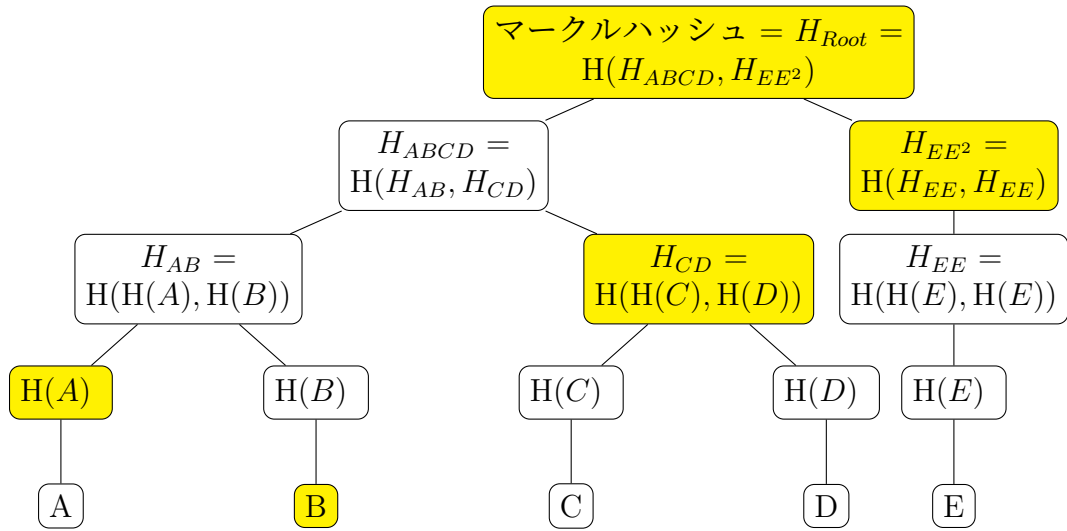


Figure 3: 木内の B の存在を証明するために必要なマークル証明

存在のマークル証明には木の各層から 1 つのハッシュが必要である。B の存在を証明するために、クライアントは以下を行う必要がある：

1. $H(B)$ を計算する。
2. H_{Root} を取得する。Symbol ではこれはブロックヘッダーに格納されている。
3. $H(A), H_{CD}, H_{EE^2}$ をリクエストする。
4. $H_{Root'} = H(H(H(H(A), H(B)), H_{CD}), H_{EE^2})$ を計算する。
5. H_{Root} と $H_{Root'}$ を比較する。一致すれば $H(B)$ は木に格納されているはずである。

4.2 パトリシア木

パトリシア木 [Mor68] は決定論的に順序付けられた木である。キーと値のペアから構築され、 $\log(N)$ 個のハッシュのみを必要とする存在証明と非存在証明の両方をサポートする。この木は決定論的にキーでソートされているため、非存在証明が可能である。同じデータをどの順序で適用しても、常に同じ木になる。

木に新しいキーと値のペアを挿入する際、キーはニブルに分解され、各ニブルは論理的に木内の独自ノードとなる。単一の木内のすべてのキーは同じ長さを持つことが要求されており、これによりわずかに最適化されたアルゴリズムが可能になる。

例として、Table 2 の以下のキーと値のペアを考える。いくつかの例では ASCII キーを使用して概念をより明確に説明し、他の例では 16 進数キーを使用して Symbol の実装をより正確に示す。

Figure 4 は各文字が別個のノードで表現される完全なパトリシア木を示す。この木は論理的に正しいが、かなり広がっており多くのメモリを使用する。典型的なキーは 32 バイトのハッシュ値であり、単一の値を格納するために最大 64 個のノードが必要になる可能性がある。この制限を回避するため、連続する空のブランチノードを少なくとも 2 つの接続を持つブランチノードまたはリーフノードに折りたたむことができる。これにより Figure 5 に示す異なるがよりコンパクトな木になる。

キー	16 進数キー	値
do**	646F0000	verb
dog*	646F6700	puppy
doge	646F6765	mascot
hors	686F7273	stallion

Table 2: パトリシア木サンプルデータ

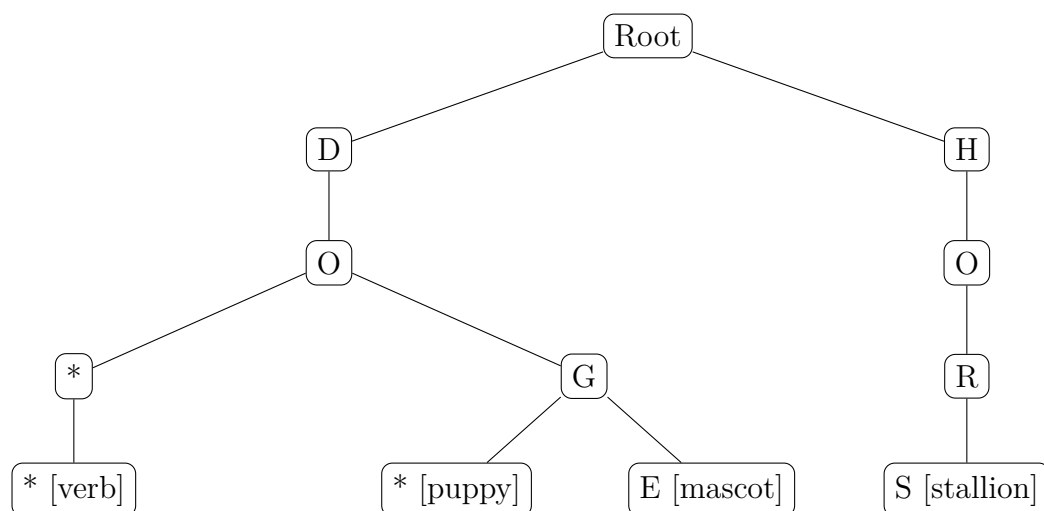


Figure 4: 4 つのデータアイテムで構成される概念的（展開）パトリシア木

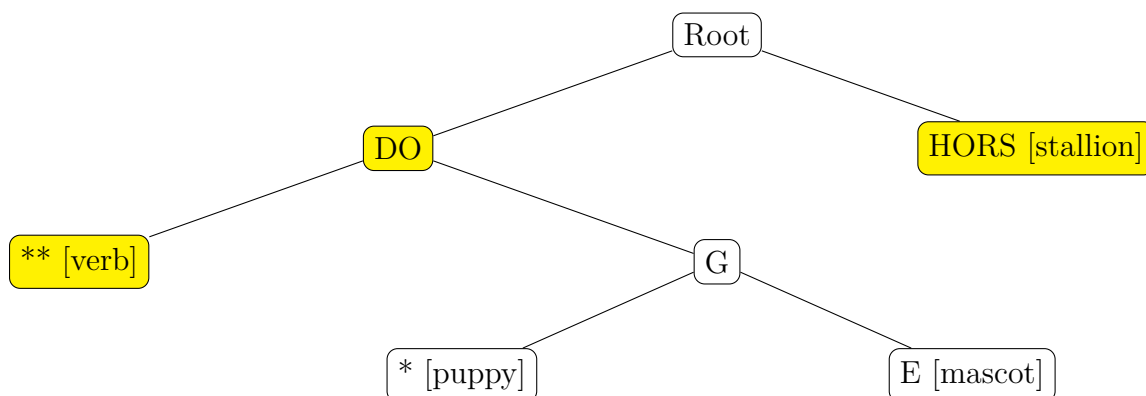


Figure 5: 4 つのデータアイテムで構成される概念的（コンパクト）パトリシア木

4.3 マークルパトリシア木

マークルパトリシア木はマークル木とパトリシア木を組み合わせたものである。**Symbol** の実装はリーフノードとブランチノードの 2 種類のノードを中心に構成されている。各リーフノードはいくつかのデータのハッシュを含む。各ブランチノードは子ノードへの最大 16 個のポインターを含む。

基本マークル木と同様に、各マークルパトリシア木にはルートハッシュがあるが、マークルパトリシア木に格納されたハッシュはわずかに複雑である。

木内のすべてのノードには木ノードパスがある。このパスはセンチネルニブルの後に 0 個以上のパスニブルが続くことで構成される。パスがリーフノードを表す場合、センチネルニブルに 0x2 がセットされる。パスが奇数のニブルで構成される場合、センチネルニブルに 0x1 がセットされ、2 番目のニブルには最初のパスニブルが含まれる。パスが偶数のニブルで構成される場合、2 番目のニブルは 0x0 に設定され、2 番目のバイトには最初のパスニブルが含まれる。

リーフノードは以下の 2 つの項目で構成される：

1. `TreeNodePath`: エンコードされた木ノードパス（リーフビットがセット）。
2. `ValueHash`: リーフで終わるキーに関連する値のハッシュ。

リーフノードのハッシュはその構成要素をハッシュ化することで計算できる：

$$H(Leaf) = H(TreeNodePath, ValueHash)$$

ブランチノードは以下の項目で構成される：

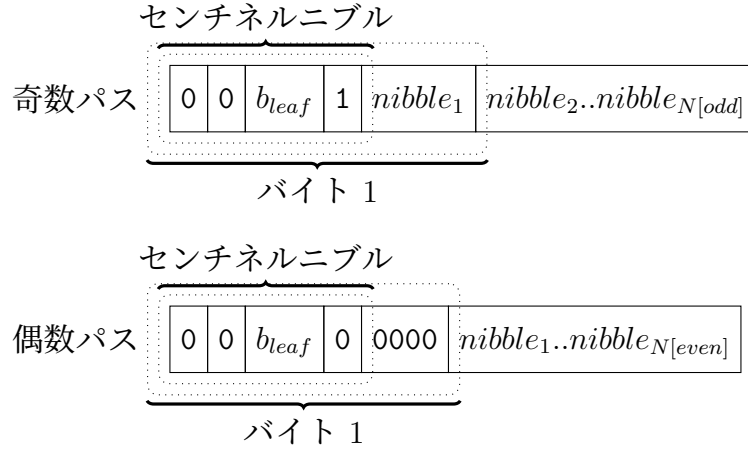


Figure 6: 木ノードパスのエンコーディング

1. *TreeNodePath*: エンコードされた木ノードパス（リーフビットがアンセット）。
2. $LinkHash_0, \dots, LinkHash_{15}$: インデックスがパスの次のニブル部分である子ノードのハッシュ。インデックスに子ノードが存在しない場合は、代わりにゼロハッシュを使用する。

ブランチノードのハッシュはその構成要素をハッシュ化することで計算できる：

$$H(\text{Branch}) = H(\text{TreeNodePath}, LinkHash_0, \dots, LinkHash_{15})$$

16 進数キーを用いた先ほどの例を再構築すると、**Symbol** の木がどのように構築されるかをより正確に示す木が得られる。各ブランチノードインデックスがパスの単一ニブルを構成することに注意されたい。これは Figure 7 に示されている。

4.4 マークルパトリシア木証明

存在のマークル証明には木の各層から 1 つのノードが必要である。 $\{key = 646F6765, value = H(mascot)\}$ の存在を証明するために、クライアントは以下を行う必要がある：

1. $H(mascot)$ を計算する（すべてのリーフ値はハッシュであることを覚えておくこと）。
2. パス 646F6765 上のすべてのノードをリクエストする： $Node_6, Node_{646F}, Node_{646F67}$ 。
3. $Node_{646F67} :: Link[6]$ が $H(\text{Leaf}(mascot))$ と等しいことを検証する。

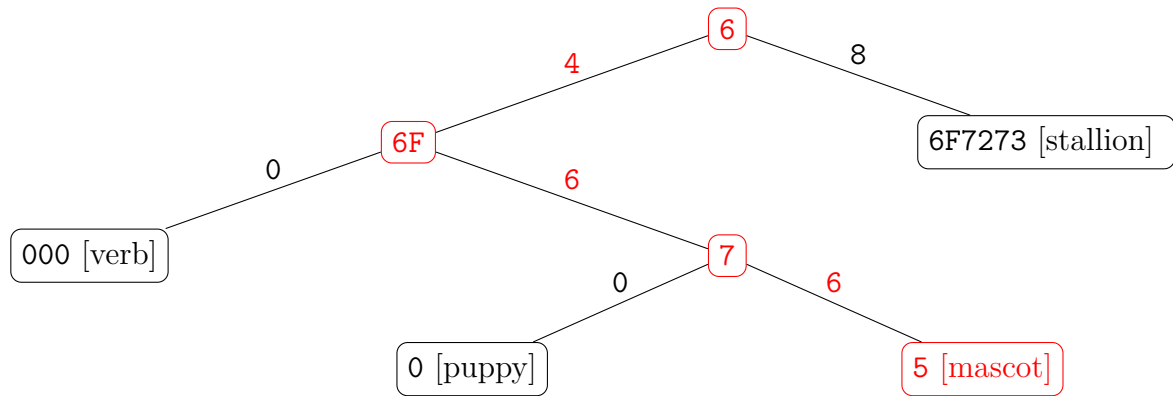


Figure 7: ブランチノードとリーフノードおよびすべての最適化を含む実際のパトリシア木。 *mascot* [646F6765] へのパスが強調表示されている。

4. $H(Node_{646F67})$ を計算し、 $Node_{6467} :: Link[6]$ が $H(Node_{646F67})$ と等しいことを検証する。
5. $H(Node_{6467})$ を計算し、 $Node_6 :: Link[4]$ が $H(Node_{6467})$ と等しいことを検証する。
6. すべての計算ハッシュと実際のハッシュが一致すれば存在が証明される。

非存在のマークル証明には木の各層から 1 つのノードが必要である。 $\{key = 646F6764, value = H(mascot)\}$ の非存在を証明するために、クライアントは以下を行う必要がある：

1. $H(mascot)$ を計算する（すべてのリーフ値はハッシュであることを覚えておくこと）。
2. パス 646F6764 上のすべてのノードをリクエストする： $Node_6, Node_{646F}, Node_{646F67}$ 。
3. $Node_{646F67} :: Link[5]$ が $H(Leaf(mascot))$ と等しいことを検証する。 $Link[5]$ がアンセットであるため、このチェックは失敗する。探しているデータが木に存在する場合、木の決定論性によりこのノードにリンクされているはずである。

5 アカウントとアドレス

((

始まりとは、バランスが正確であることに最も細心の注意を払うべき時である。))

- Frank Herbert

SYMBOL は楕円曲線暗号を使用し、すべてのトランザクションの機密性、真正性、および否認不能性を保証する。アカウントはアドレスによって一意に識別され、アドレスは署名公開鍵の方向変換から部分的に導出される。各アカウントはネットワークによってトランザクションが承認されると更新される可変状態にリンクされている。この状態はグローバルに一貫しており、ゼロ個以上の公開鍵を含む場合がある。

5.1 アドレス

デコード済みアドレスは以下の 3 つの部分から構成される 24 バイトの値である：

- ネットワークバイト
- アカウントの署名公開鍵の 160 ビットハッシュ
- 3 バイトのチェックサム

チェックサムにより、誤入力されたアドレスを素早く認識できる。そのアドレスが以前にトランザクションに参加していなくても、有効なアドレスにモザイク⁷を送ることができる。モザイクが送られたアカウントの秘密鍵を誰も持っていない場合、そのモザイクは永遠に失われる可能性が高い。

エンコード済みアドレスはデコード済みアドレスの Base32⁸エンコードである。人間が読みやすく、クライアントで使用される。

Base32 によるバイナリデータのエンコードは $\frac{8}{5}$ の拡張係数を持ち、デコード済みアドレスのサイズは 5 で割り切れない。そのため、デコード済みアドレスの Base32 表現には常に余分な情報が含まれる。これは本質的に問題ではないが、デコード済みアドレスをエンコードする際、クライアント間の一貫性のために余分な入力バイトは慣例としてゼロに設定される。また、結果として得られる 40 文字の文字列の最後のバイトは削除される。

⁷モザイクとはSymbolで定義されたデジタル資産である。他の技術ではモザイクをトークンと呼ぶ。

⁸<http://en.wikipedia.org/wiki/Base32>

5.1.1 アドレス導出

公開鍵をアドレスに変換するには、以下の手順を実行する：

1. 公開鍵に対して 256 ビット SHA3 を実行する。
2. 手順 1 の結果のハッシュに対して 160 ビット RIPEMD_160 を実行する。
3. RIPEMD_160 ハッシュの先頭にネットワークバージョンバイトを付加する。
4. その結果に対して 256 ビット SHA3 を実行し、最初の 3 バイトをチェックサムとして取得する。
5. 手順 3 の出力と手順 4 のチェックサムを連結する。
6. 結果を Base32 でエンコードする。

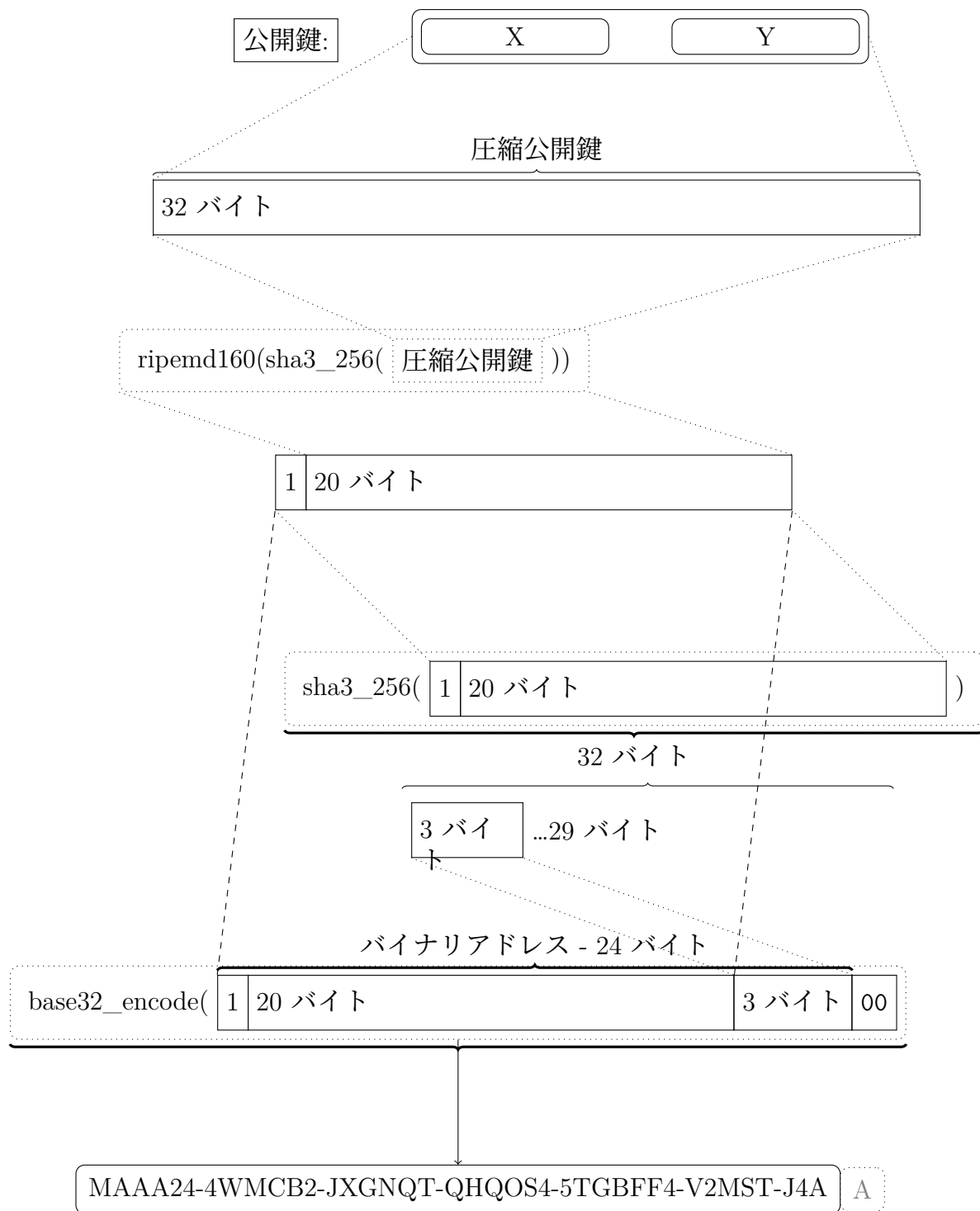


Figure 8: アドレス生成

5.1.2 アドレスエイリアス

アドレスには、アドレスエイリアストランザクション⁹を使用して1つ以上のエイリアスを割り当てることができる。アドレスを受け入れるすべてのトランザクションは、公開鍵由来のアドレスまたはアドレスエイリアスのいずれかの使用をサポートする。このようなトランザクションでは、アドレスエイリアスフィールドのフォーマットは以下のとおりである：

- ネットワークバイトと値1のOR
- エイリアスである8バイトのネームスペースID
- 15のゼロバイト

5.1.3 意図的なアドレス衝突

2つの異なる署名公開鍵が同じアドレスを生成することは理論的に可能である。そのようなアドレスが貴重な資産を含み、かつ以前に公開鍵に関連付けられていない場合（例えば、そのアカウントからトランザクションを送信することで関連付けられる）、攻撃者はそのアカウントから資金を引き出すことができる可能性がある。

攻撃を成功させるには、攻撃者は公開鍵のSHA3_256が**同時に**上記の160ビットハッシュのRIPEMD_160原像と等しくなるような秘密鍵・公開鍵ペアを見つける必要がある。SHA3_256は128ビットのセキュリティを提供するため、単一のSHA3_256衝突を発見することは数学的に非常に困難である。**Symbol**アドレスとBitcoinアドレスの類似性から、**Symbol**アドレスの衝突を引き起こす確率はBitcoinアドレスの衝突を引き起こす確率とほぼ同じである。

5.2 公開鍵

アカウントはゼロ個以上の公開鍵に関連付けられる。サポートされる鍵の種類は以下のとおりである：

1. 署名鍵:
データの署名と検証に使用されるED25519公開鍵。どのアカウントもデータを受信できるが、この公開鍵を持つアカウントのみがデータを送信できる。この公開鍵はアカウントのアドレス計算への入力として使用される唯一の鍵である。

⁹詳細は <https://nemtech.github.io/concepts/namespace.html#addressaliastransaction> を参照のこと。

2. リンク鍵:

この公開鍵はメインアカウントとリモートハーベスターをリンクする。Main アカウントに対しては、この公開鍵がその代わりにブロックに署名できるリモートハーベスターアカウントを指定する。Remote アカウントに対しては、この公開鍵がブロックに署名できるメインアカウントを指定する。これらのリンクは双方向であり、常にペアで設定される。

3. ノード鍵:

この公開鍵は Main アカウントに設定される。デリゲートハーベスティングを行えるノードの公開鍵を指定する。重要なことに、これはリモートがそのノードでアクティブにハーベスティングしていることを示すのではなく、そうする権限があることのみを示す。ハーベスティングアカウントまたはノードオーナーのどちらかが誠実である限り、アカウントは一度に単一のノードでのデリゲートハーベスティングに制限される。

誠実なハーベスターは、リモートハーベスターの秘密鍵を一度に単一のノードにのみ送信すべきである。リモートを変更すると、他のすべてのノードに付与された以前のリモートハーベスティング権限が無効化される（これはデリゲート鍵の前方秘匿性を意味する）。古いリモートハーベスターの秘密鍵はもはや有効ではなく、ブロックのハーベスティングに使用できない。誠実なノードオーナーは、現在ノードの公開鍵にリンクされているリモートハーベスターの秘密鍵でのみリモートハーベスティングを行うべきである。

4. VRF 鍵:

ランダム値の生成と検証に使用される ED25519 公開鍵。アカウントがハーベスティングの対象となるためには、この公開鍵を Main アカウントに設定する必要がある。

5. 投票鍵:

ファイナライゼーションメッセージの署名と検証に使用される BLS 公開鍵。すべての投票鍵は一時的なものであり、開始エポックと終了エポックの両方を指定して登録する必要がある。アカウントが投票に参加できるようにするためには、この公開鍵を Main アカウントに設定する必要がある。現在のエポックが登録された範囲内にある場合にのみ有効である。鍵のシームレスな切り替えを可能にするため、アカウントは最大 *network:maxVotingKeysPerAccount* 個の投票鍵を同時に登録できる。

6 トランザクション

((

実際、私たちが経済活動と考えるものの大部分は、高いトランザクションコストが妨げるものを達成するため、あるいはトランザクションコストを下げて個人が自由に交渉できるようにし、Friedrich Hayek が語ってくれた分散した知識を活用できるようにするために設計されている。))

- Ronald Coase

TRANSACTIONS トランザクションはグローバルなチェーン状態を変更する命令である。それらはアトミックに処理され、ブロックにグループ化される。トランザクションの処理のいずれかの部分が失敗した場合、グローバルなチェーン状態はトランザクション適用の試み前の状態にリセットされる。

トランザクションには 2 つの基本的な種類がある：基本トランザクションとアグリゲートトランザクション。基本トランザクションは単一の操作を表し、単一の署名を必要とする。アグリゲートトランザクションは、複数の署名を必要とする場合がある 1 つ以上のトランザクションのコンテナである。

アグリゲートトランザクションにより、基本トランザクションを組み合わせて潜在的に複雑な操作にし、アトミックに実行できる。これにより、個々の操作のアトミック性のみを保証しながらも、有限の許可された操作セットにグローバルな操作を制限するシステムと比べて、開発者の柔軟性が向上する。チューリング完全な言語とそれが持つ固有の欠点を導入する必要がない。開発者は新しい言語を学ぶ必要も、ゼロからカスタムのコントラクト実装を開発する必要もない。トランザクションの構成は、計算完全なプログラムを実装するよりもエラーが少なく、バグも少なくなるはずである。

6.1 基本トランザクション

基本トランザクションは暗号的に検証可能なデータと検証不可能なデータの両方で構成される。すべての検証可能なデータは連続しており、トランザクション署名者によって署名される。すべての検証不可能なデータは無視されるか（例：パディングバイト）、検証可能なデータから決定論的に計算可能である。各基本トランザクションはちょうど 1 つの署名の検証を必要とする。

検証不可能なヘッダーフィールドはいずれも検証する必要がない。Size はトランザクションのシリアライズされたサイズであり、検証可能なトランザクションデータから常に導出できる。Signature は署名の出力であり、検証への入力である。SignerPublicKey は署名と検証の両方への入力である。トランザクション T が署名検証をパスするためには、Signature と SignerPublicKey の両方が、Size に相対し

た長さを持つ検証可能なデータと一致する必要がある。

`verify(T::Signature, T::SignerPublicKey, VerifiableDataBuffer(T))`

Reserved バイトはトランザクションのパディングに使用され、すべての整数フィールドと暗号プリミティブが自然なアライメントを持つようにする。これらのバイトは意味を持たないため、暗号学的な保証を無効にすることなく取り除くことができる。

すべてのトランザクション種別のバイナリレイアウトは **Symbol** のオープンソーススキーマ言語で指定されている¹⁰。最新の仕様については公開されたスキーマを参照のこと。

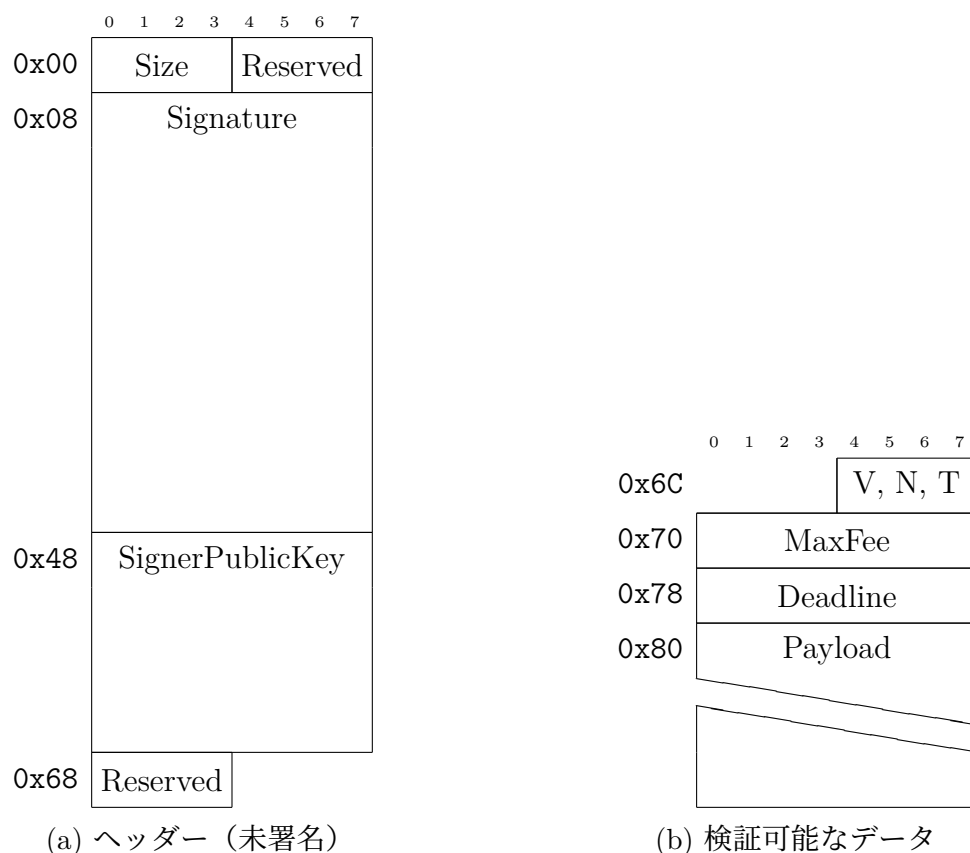


Figure 9: 基本トランザクションのバイナリレイアウト

図では、(V)ersion、(N)etwork、(T)ype はスペースの都合で省略されている。

¹⁰スキーマは <https://github.com/nemtech/catbuffer> で確認できる。

6.2 アグリゲートトランザクション

アグリゲートトランザクションのレイアウトは基本トランザクションより複雑だが、いくつかの共通点がある。アグリゲートトランザクションは基本トランザクションと同じ検証不可能なヘッダーを持ち、このデータは同じ方法で処理される。さらに、アグリゲートトランザクションは検証不可能なデータのフッターと、その後に埋め込みトランザクションおよびコサインネチャを持つ。

アグリゲートトランザクションは常に必要なすべてのコサインネチャを添付してネットワークに送信できる。この場合、完成しているとされ、特別な処理なしに通常のトランザクションと同様に扱われる。

API ノードは不完全なコサインネチャを持つボンデッドアグリゲートトランザクションも受け入れることができる。送信者は、トランザクションがタイムアウトする前にすべての必要なコサインネチャが収集された場合にのみ返還される保証金を支払わなければならない。この保証金が事前に支払われると仮定して、API ノードは十分な署名が揃うかタイムアウトするまで、このトランザクションに関連するコサインネチャを収集する。

TransactionsHash はアグリゲートトランザクションの最も重要なフィールドである。アグリゲート内に格納された埋め込みトランザクションのハッシュの Merkle ルートハッシュである。このフィールドを計算するために、各埋め込みトランザクションハッシュを自然順序で追加して Merkle ツリーが構築される。結果のルートハッシュがこのフィールドに割り当てられる。

検証不可能なフッターフィールドはいずれも検証する必要がある。PayloadSize は、TransactionsHash の計算に使用されたのと同まったく同じ埋め込みトランザクションを抽出するために正確でなければならない計算済みサイズフィールドである。Reserved バイトはパディングに使用され、固有の意味を持たない。

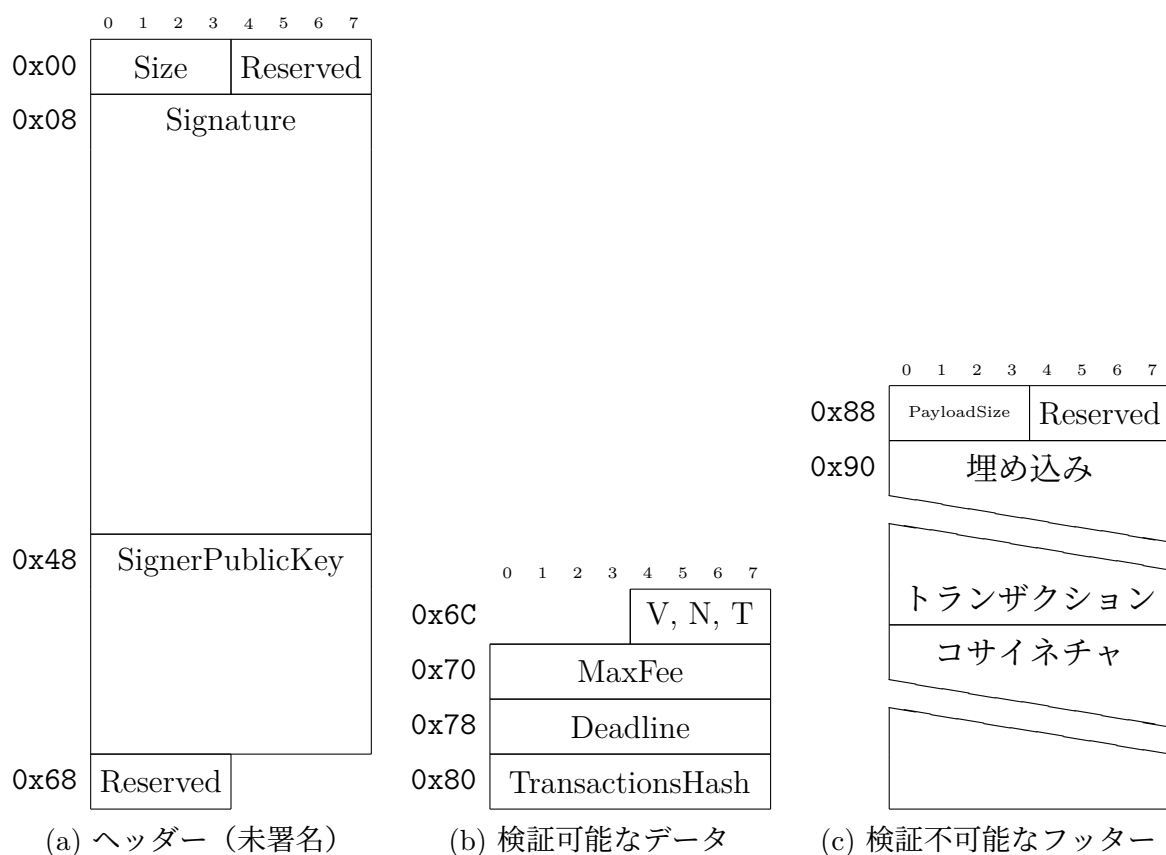


Figure 10: アグリゲートトランザクションヘッダーのバイナリレイアウト

6.2.1 埋め込みトランザクション

埋め込みトランザクションはアグリゲートトランザクション内に含まれるトランザクションである。基本トランザクションと比較してヘッダーは小さいが、トランザクション固有のデータは同じである。Signature はすべての署名情報がコサインネチャに含まれているため削除される。MaxFee と Deadline は親のアグリゲートトランザクションで指定されるため削除される。

クライアント実装は基本トランザクションまたは埋め込みトランザクションのカスタム部分を構築するために同じコードを使用できる。違いは異なるヘッダーの作成と適用のみである。

すべてのトランザクションが埋め込みトランザクションとしてサポートされているわけではない。例えば、アグリゲートトランザクションを別のアグリゲートトランザクション内に埋め込むことはできない。

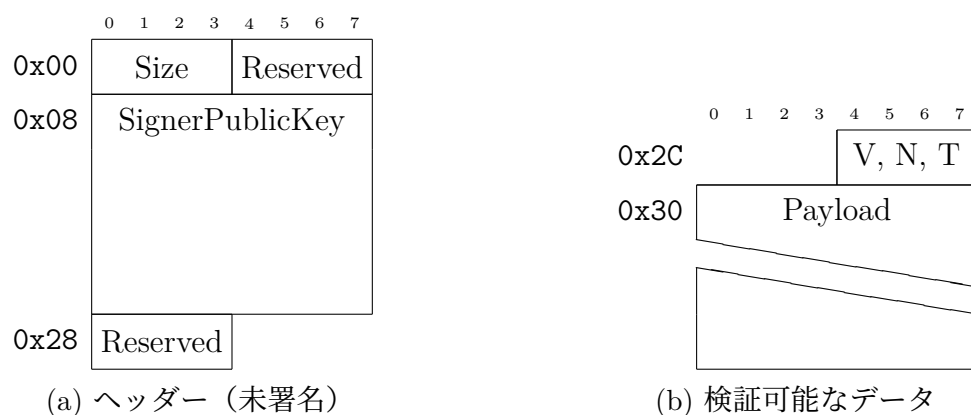


Figure 11: 埋め込みトランザクションのバイナリレイアウト

6.2.2 コサイネチャ

コサイネチャはバージョン¹¹、公開鍵、およびその対応する署名で構成される。ゼロ個以上のコサイネチャがアグリゲートトランザクションの末尾に追加される。コサイネチャは複数の当事者が関わるアグリゲートトランザクションを暗号的に検証するために使用される。

¹¹Version は将来の拡張性のために予約されている。現在は常にゼロであることが期待される。

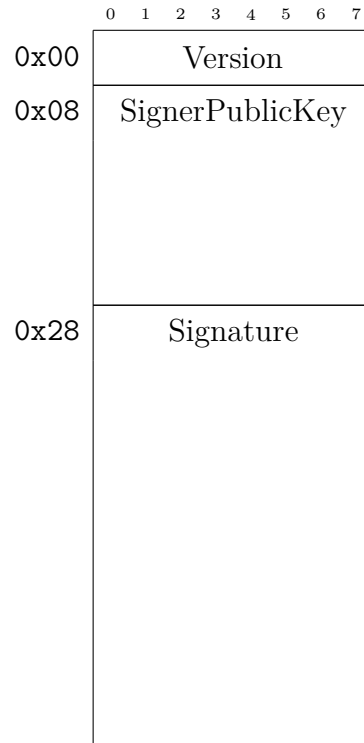


Figure 12: コサイネチャのバイナリレイアウト

アグリゲートトランザクション A が検証をパスするためには、基本トランザクションの署名検証をパスし、すべての埋め込みトランザクション署名者のコサイネチャを持っている必要がある¹²。

他のトランザクションと同様に、アグリゲートトランザクションは基本トランザクションの署名検証をパスする必要がある。

$$\text{verify}(A::\text{Signature}, A::\text{SignerPublicKey}, \text{VerifiableDataBuffer}(A))$$

さらに、すべてのコサイネチャが署名検証をパスする必要がある。コサイナーはデータを直接ではなく、アグリゲートトランザクションデータのハッシュに署名することに注意。

$$\sum_{0 \leq i \leq N_C} \text{verify}(C::\text{Signature}, C::\text{SignerPublicKey}, H(\text{VerifiableDataBuffer}(A)))$$

最後に、各埋め込みトランザクション署名者に対応し、それを満たすコサイネチャが存在しなければならない。

¹²マルチシグアカウントの場合、マルチシグアカウントの制約を満たすのに十分なコサイネチャが必要である。

6.2.3 拡張レイアウト

前述のアグリゲートトランザクションレイアウトは 1 つの簡略化をして説明したものである。すべての埋め込みトランザクションは 8 バイト境界で終わるようにパディングされる。これにより、すべての埋め込みトランザクションとコサイネチャも 8 バイト境界で始まることが保証される。パディングバイトは署名されることも、いかなるハッシュにも含まれることもない。

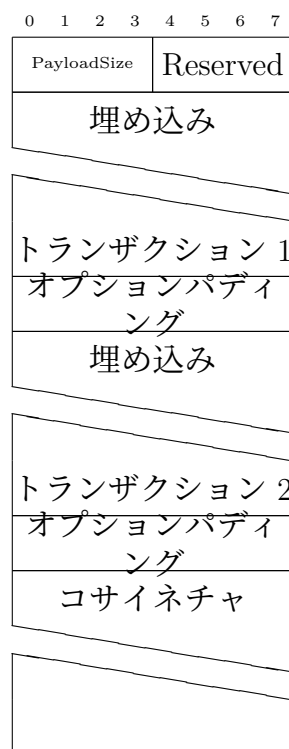


Figure 13: パディング付きアグリゲートトランザクションフッター

6.3 トランザクションハッシュ

各トランザクションには 2 つの関連するハッシュがある——エンティティハッシュと *Merkle* コンポーネントハッシュである。前者はトランザクションを一意に識別し、同じトランザクションが複数回確認されることを防ぐために使用される。後者はより具体的で、TransactionsHash（[6.2: アグリゲートトランザクション](#) を参照）の計算に使用される。

トランザクションのエンティティハッシュは以下のハッシュとして計算される：

1. トランザクション Signature ——このフィールドが含まれていない場合、攻撃

者は不正な形式の署名を含むほぼ同一のトランザクションを先行させることで、トランザクションがネットワークに含まれるのを防ぐことができる。

2. トランザクション `SignerPublicKey` — このフィールドが含まれていない場合、攻撃者は不正な形式の署名者公開鍵を含むほぼ同一のトランザクションを先行させることで、トランザクションがネットワークに含まれるのを防ぐことができる。
3. ネットワーク `GenerationHashSeed` — このフィールドはクロスネットワークのトランザクションリプレイ攻撃を防ぐ¹³。
4. トランザクションの検証可能なデータ

すべての確認済みトランザクションは一意のエンティティハッシュを持たなければならない。アグリゲートトランザクションのエンティティハッシュはそのコサインチャから独立している。これにより、同じアグリゲートトランザクションが異なる有効なコサインチャのセットで複数回確認されることを防ぐ。

通常のトランザクションの Merkle コンポーネントハッシュはそのエンティティハッシュと同一である。アグリゲートトランザクションの Merkle コンポーネントハッシュは、そのエンティティハッシュとすべてのコサインチャの公開鍵を連結したものをハッシュして計算される¹⁴。これにより、`TransactionsHash` がアグリゲートトランザクションの確認を可能にしたすべてのコサインチャを反映することが保証される。

¹³さらに、トランザクションデータの署名と検証の際、ネットワークの `GenerationHashSeed` がデータの先頭に付加されるため、トランザクション署名は一致する `GenerationHashSeed` を持つネットワークでのみ検証される。

¹⁴コサインチャの署名は含まれない。特定の公開鍵とペイロードが与えられると、単一の値しか取れないからである。

7 ブロック

((

悲観主義者にとっての躓きの石は、楽観主義者にとっての足がかりとなる。))

- Eleanor Roosevelt

SYMBOL は、その核心において、ブロックチェーンである。ブロックチェーンはブロックの順序付けられたコレクションである。**Symbol** ブロックの各部分を理解することは、このプラットフォームの機能を理解する上で基本となる。

ブロックのレイアウトはアグリゲート・トランザクションのレイアウトに似ている(6.2: [アグリゲートトランザクション](#)を参照)。ブロックはアグリゲート・トランザクションと同じ検証不可能なヘッダー¹⁵を共有しており、このデータは同じ方法で処理される。同様に、ブロックは検証不可能なデータのフッターの後にトランザクションが続く。アグリゲート・トランザクションとは異なり、ブロックの後にはエンベッドされた基本トランザクションではなく通常のトランザクションが続き、ブロック内の各トランザクションはブロック署名者とは独立して署名される¹⁶。これにより、すべての条件を満たす任意のトランザクションを任意のブロックに含めることができる。

検証不可能なフッターフィールドはいずれも検証する必要がない。Reserved バイトはパディングに使用され、固有の意味を持たない。

7.1 ブロックフィールド

Height はブロックのシーケンス番号である。ネメシスブロックと呼ばれる最初のブロックの高さは 1 である。後続の各ブロックは前のブロックの高さを 1 ずつ増加させる。

Timestamp はネメシスブロック以降に経過したミリ秒数である。ブロック時刻は厳密に増加するため、後続の各ブロックは前のブロックよりも大きいタイムスタンプを持つ必要がある。各ネットワークはブロック間の平均時間を目標ブロック時間に近づけようとする。

Difficulty は以前のブロックに基づいて、新しいブロックのハーベストがどれほど困難かを決定する。難易度は [8.1: ブロック難易度](#) で詳しく説明されている。

¹⁵強調するが、これは検証可能なブロックヘッダーを指しているのではない。検証可能なブロックヘッダーに先行する Signature などのフィールドを指している。

¹⁶アグリゲート・トランザクションでは、アグリゲート・トランザクションを作成するアカウントはそれが有効であるためにトランザクションのデータに署名する必要がある。ブロックでは、ブロック署名者はその中に含まれるトランザクションのデータに署名する必要はない。

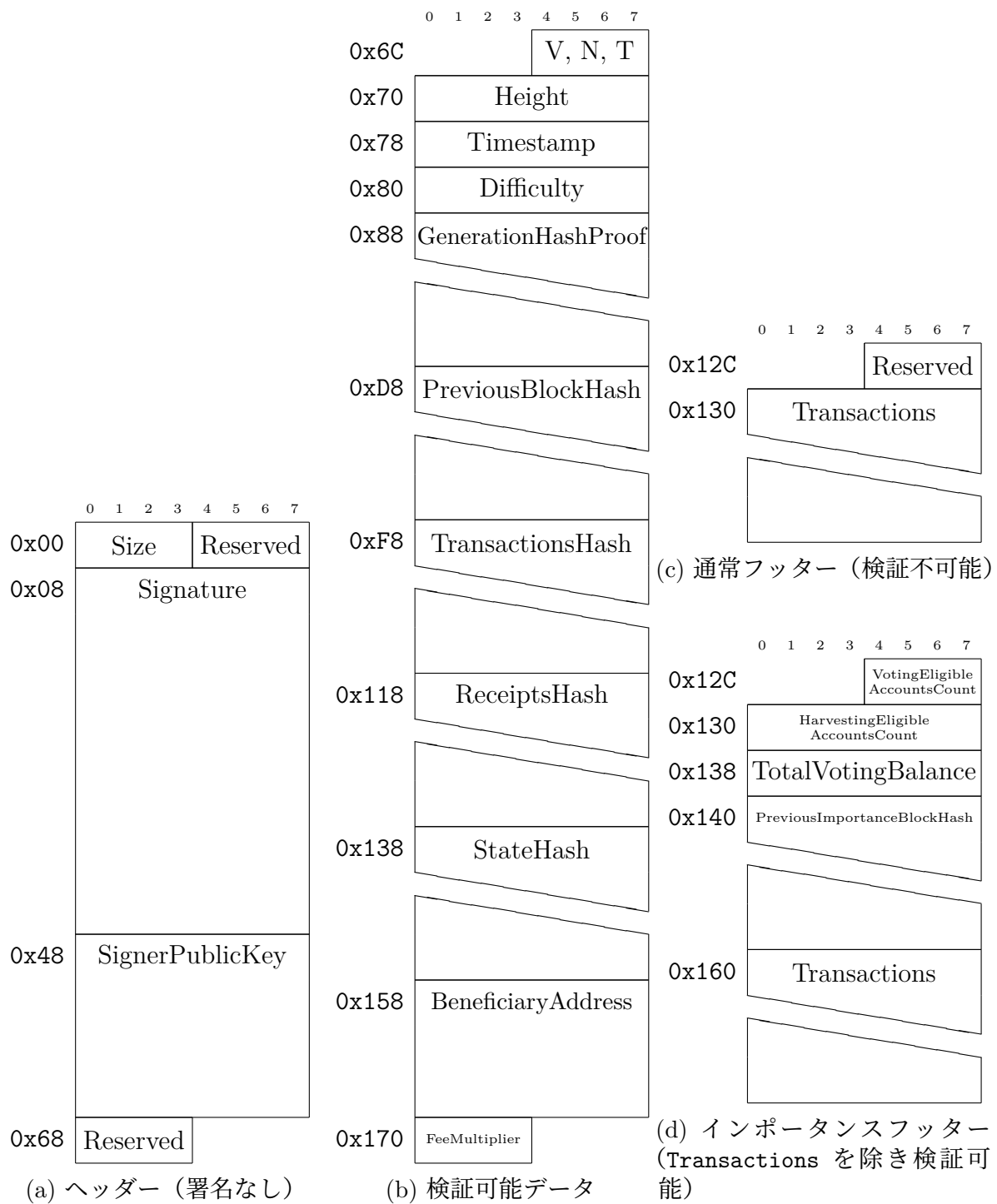


Figure 14: ブロックヘッダーのバイナリレイアウト

GenerationHashProof はブロックハーベスターの VRF 秘密鍵で生成された VRF 証明である。32 バイトの γ 、16 バイトの検証ハッシュ (c)、32 バイトのスカラー (s) で構成される (3.3: 検証可能ランダム関数 (VRF) を参照)。これは将来のブロックハーベスターが予測不可能であることを保証するために使用される (8.3: ブロック生成 を参照)。

PreviousBlockHash は前のブロックのハッシュである。これはブロックチェーン内のすべてのブロックがリンクされ決定論的に順序付けられていることを保証するために使用される。

TransactionsHash はブロック内に格納されたトランザクションのハッシュのマークルルートハッシュである¹⁷。このフィールドを計算するために、各トランザクションハッシュを自然な順序で追加することでマークル木が構築される。結果のルートハッシュがこのフィールドに割り当てられる。

ReceiptsHash はブロック処理中に生成されたレシートのハッシュのマークルルートハッシュである。ネットワークが *network:enableVerifiableReceipts* なしで設定されている場合、このフィールドはすべてのブロックでゼロにする必要がある (7.2: レシート を参照)。

StateHash はブロック処理後のグローバルなブロックチェーン状態のハッシュである。ネットワークが *network:enableVerifiableState* なしで設定されている場合、このフィールドはすべてのブロックでゼロにする必要がある。それ以外の場合、7.3: 状態ハッシュ で説明されているように計算される。

BeneficiaryAddress は、ネットワークがゼロ以外の *network:harvestBeneficiaryPercentage* を持つ場合に、ブロックの受益者シェアが割り当てられるアカウントである。このフィールドはネットワークにまだ知られていないアカウントを含む任意のアカウントに設定できる。このフィールドは新しいブロックがハーベストされる際にノード所有者によって設定される。このアカウントがノード所有者が所有するものに設定されている場合、ノード所有者はそのノードでハーベストされたすべてのブロックで支払われた手数料を共有する。これにより、ノード所有者は多くの委任ハーベスターを持つ強力なノードを運営するインセンティブが生まれる。

FeeMultiplier はブロック内に含まれる各トランザクションの実効手数料を計算するために使用される乗数である。*node:minFeeMultiplier* はノード所有者によって設定され、利益の最大化や確認済みトランザクションの最大化など、様々な目標の達成に使用できる。トランザクション T を含むブロック B を仮定すると、実効トランザクション手数料は次のように計算できる：

$$\text{effectiveFee}(T) = B::\text{FeeMultiplier} \cdot \text{sizeof}(T)$$

実効手数料がトランザクションの MaxFee よりも大きい場合、トランザクション署名

¹⁷ このフィールドはアグリゲート・トランザクション内の同名フィールドと同じ目的を持つ。

者は差額を保持する。実効手数料のみがトランザクション署名者から差し引かれ、ハーベスターにクレジットされる。手数料乗数に関する詳細情報は [8.3: ブロック生成](#) にある。

7.1.1 インポートانسブロックフィールド

インポートانس計算が行われる各ブロックには、追加の検証可能な情報を含む拡張フィールドが含まれる¹⁸。この情報はブロック処理後のグローバルなブロックチェーン状態を反映する。

`VotingEligibleAccountsCount` は次のインポートانسグループをカバーするファイナライゼーションエポックで投票する資格があるアカウントの数である。これは正確な数であり、すべての投票要件を満たすアカウントのみが含まれる。例えば、投票キーが登録されていないアカウントは除外される。

`HarvestingEligibleAccountsCount` は少なくとも `network:minHarvesterBalance` の残高を持つアカウントの数である。これは推定値であり最大値である。ハーベスティング不適格なアカウントが除外されていないため、実際のハーベスティング適格アカウント数はより少ない可能性がある。例えば、VRF キーが登録されていないアカウントは除外されない。

`TotalVotingBalance` は次のインポートانسグループをカバーするファイナライゼーションエポックで投票する資格があるすべてのアカウントの残高の合計である。この値はファイナライゼーション証明の信頼なし検証を可能にする。ファイナライゼーション証明の投票者が蓄積した投票ステークのパーセンテージを計算する際に、信頼できる分母として使用できる。

`PreviousImportanceBlockHash` は前のインポートانسブロックのハッシュである。これはブロックチェーン内のすべてのブロックがリンクされ決定論的に順序付けられていることの、`PreviousBlockHash` に比べてより長期的な保証である。すべてのブロックヘッダーではなくインポートانسブロックヘッダーのみをダウンロードする必要がある、信頼なし高速同期プロトコルの実現に使用できる。

7.2 レシート

ブロックの実行中に、0 個以上のレシートが生成される。レシートは主に副作用によってトリガーされた状態変化をクライアントに通知するために使用される。これにより、よりシンプルなクライアントでも複雑な状態変化を把握できるようになる。

例えば、ネームスペースの有効期限は、ネームスペースが作成されてから確認され

¹⁸`network:votingSetGrouping` は `network:importanceGrouping` の倍数であることが期待される。

たブロック数によってトリガーされる。トリガーとなるイベントはブロックチェーン内にあるが、有効期限が発生するブロックにはこの状態変化の指示が含まれていない。レシートがなければ、クライアントはすべてのネームスペースと有効期限の高さを追跡し続ける必要がある。レシートがあれば、クライアントは有効期限レシートを監視するだけでよい。

別の例はハーベスト報酬に関するものである。クレジットされる主要な（委任でない）アカウントと受益者の分割を示すレシートが生成される。また、インフレによって生成された通貨量も通知される。

レシートはレシートソースによって 3 種類のステートメントにグループ化されて照合される。3 種類のステートメントはトランザクション、アドレス解決、モザイク解決である。

7.2.1 レシートソース

処理されるブロックの各部分には 2 部構成のブロックスコープ識別子が割り当てられる。ソース (0, 0) は常にブロック内のトランザクション数に関係なくブロックトリガーイベントを識別するために使用される。

ソース	プライマリ ID	セカンダリ ID
ブロック	0	0
トランザクション	ブロック内の 1 始まりのインデックス	0
エンベッドトランザクション	ブロック内の含まれるアグリゲートの 1 始まりのインデックス	アグリゲート内の 1 始まりのインデックス

Table 3: レシートソース値

7.2.2 トランザクションステートメント

トランザクションステートメントは共通のレシートソースを持つレシートをグループ化するために使用される。各ステートメントはレシートソースと 1 つ以上のレシートで構成される。したがって、レシートを生成する各レシートソースには正確に 1 つの対応するトランザクションステートメントがある。

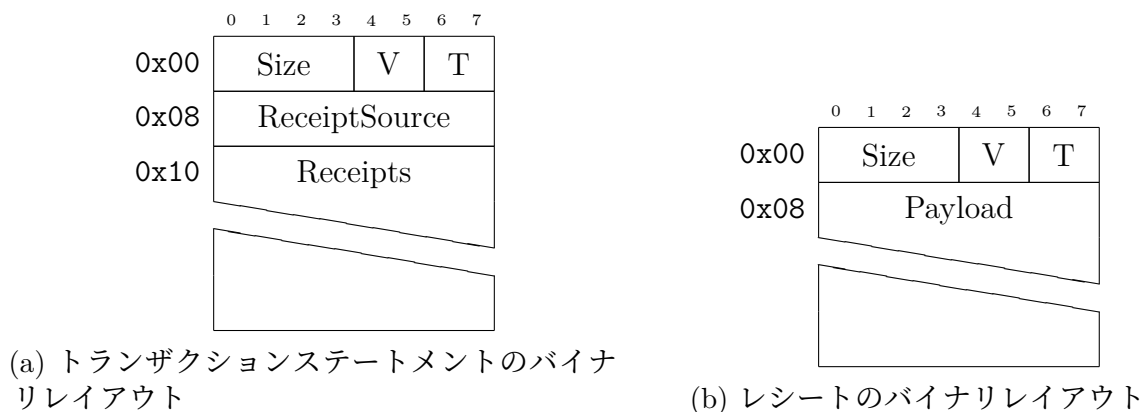


Figure 15: トランザクションステートメントのレイアウト

トランザクションステートメントデータはパディングされない。これは処理中にのみ書き込まれ、読み取ることがないため、パディングはサーバーパフォーマンスに利点をもたらさないからである。トランザクションステートメントハッシュは、他のデータから導出可能な Size フィールドを除くすべてのステートメントデータを連結してハッシュ化することで構築される。

7.2.3 解決ステートメント

解決ステートメントはエイリアス解決の指示のみに使用される。クライアントがブロック内で変更されても未解決の値を常に解決できるようにする。理論的には、同じブロック内の 2 つの未解決エイリアスは、使用の間にエイリアスの変更があれば異なる値に解決される可能性がある。これをサポートするために、各ステートメントは未解決の値と 1 つ以上の解決で構成される。

解決ステートメントには 2 種類あり、アドレスとモザイクの 2 種類のエイリアスに対応している。Figure 16 はモザイク解決ステートメントのレイアウトを示しているが、アドレス解決ステートメントのレイアウトはほぼ同一である。唯一の違いは、解決済みおよび未解決の値が 8 バイトのモザイク ID ではなく 25 バイトのアドレスであることである。

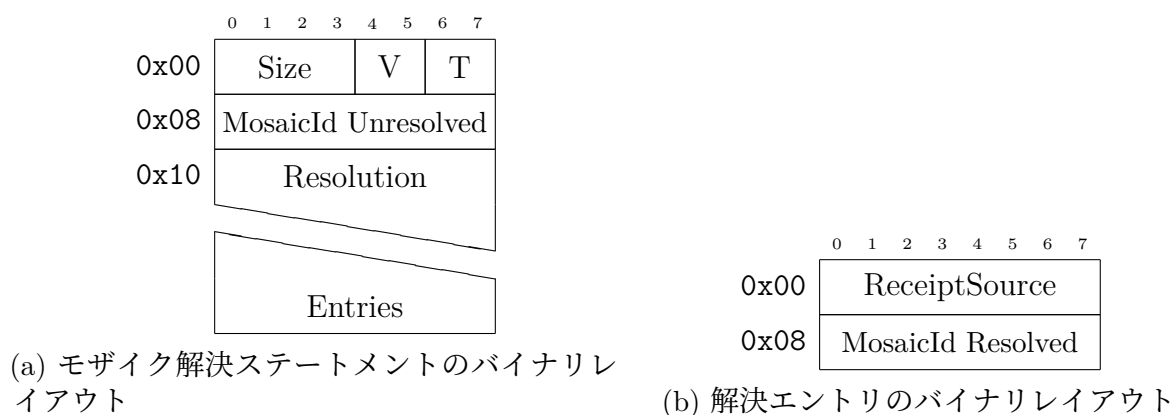


Figure 16: モザイク解決ステートメントのレイアウト

解決ステートメントデータはパディングされない。これは処理中にのみ書き込まれ、読み取られることがないため、パディングはサーバーパフォーマンスに利点をもたらさないからである。解決ステートメントハッシュは、他のデータから導出可能な Size フィールドを除くすべてのステートメントデータを連結してハッシュ化することで構築される。

解決ステートメントは解決が発生した場合にのみ生成されることに注意することが重要である。エイリアスがブロックで登録または変更されても、そのブロックで使用されなければ、解決ステートメントは生成されない。ただし、そのエイリアスを含み解決が必要な各ブロックによって解決ステートメントが生成される。

7.2.4 レシートハッシュ

ブロックのレシートハッシュを計算するために、まずブロック処理中に生成されたすべてのステートメントが収集される。次に、以下の順序ですべてのステートメントハッシュを追加することでマークル木が作成される：

1. レシートソースで順序付けられたトランザクションステートメントのハッシュ。
2. 未解決アドレスで順序付けられたアドレス解決ステートメントのハッシュ。
3. 未解決モザイク ID で順序付けられたモザイク解決ステートメントのハッシュ。

ネットワークが `network:enableVerifiableReceipts` で設定されている場合、このマークル木のルートハッシュがブロックの `ReceiptHash` として設定される。クライアントはマークル証明を実行して、特定のブロックの処理中に特定のステートメントが生成されたことを証明できる。

7.3 状態ハッシュ

Symbol は複数の型付き状態リポジトリにわたってグローバルなブロックチェーン状態を格納する。例えば、アカウント状態は 1 つのリポジトリに格納され、マルチシグネチャ状態は別のリポジトリに格納される。各リポジトリは単純なキーと値のストアである。ネットワークに存在する特定のリポジトリは、そのネットワークで有効化されたトランザクションプラグインによって決定される。

ネットワークが `network:enableVerifiableState` で設定されている場合、各リポジトリのためにパトリシア木が作成される。これにより各リポジトリを決定論的にフィンガープリントする単一のハッシュが生成される。したがって、 N 個のリポジトリがあると仮定すると、 N 個のハッシュがグローバルなブロックチェーン状態を決定論的にフィンガープリントする。

N 個のハッシュをすべて各ブロックヘッダーに直接格納することは可能だが、望ましくない。すべてのクライアントは最低限、チェーンがネメシスブロックに根付いていることを検証するためにすべてのヘッダーを同期する必要があるため、各ブロックヘッダーはできるだけ小さくすべきである。さらに、機能の追加や削除によりリポジトリ数 (N) やブロックヘッダーのフォーマットが変わる可能性がある。

代わりに、すべてのルートハッシュが連結¹⁹されてハッシュ化され、グローバルなブロックチェーン状態を決定論的にフィンガープリントする単一のハッシュである `StateHash` が計算される。

$$\begin{aligned} \text{RepositoryHashes} &= 0 \\ \text{RepositoryHashes} &= \sum_{0 \leq i \leq N} \text{concat}(\text{RepositoryHashes}, \text{RepositoryHash}_i) \\ \text{StateHash} &= H(\text{RepositoryHashes}) \end{aligned}$$

7.4 拡張レイアウト

前述のブロックレイアウトは 1 つの簡略化をして説明された²⁰。すべてのトランザクションは 8 バイト境界で終わるようにパディングされる。これにより、すべてのトランザクションが 8 バイト境界で始まることも保証される。パディングバイトは署名されず、いかなるハッシュにも含まれない。

¹⁹連結順序は固定されており、リポジトリ ID によって決定される。

²⁰これはアグリゲート・トランザクションの拡張レイアウトとも一致している。

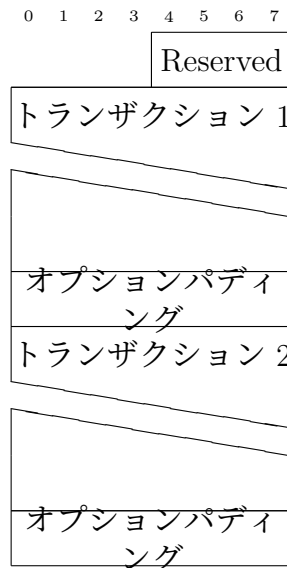


Figure 17: パディング付きブロックトランザクションフッター

7.5 ブロックハッシュ

各ブロックには 1 つの関連するハッシュがある、それがエンティティハッシュである。このハッシュはブロックを一意に識別し、同じブロックが複数回処理されることを防ぐために使用される。

ブロックのエンティティハッシュは以下のハッシュとして計算される：

1. ブロック Signature
2. ブロック SignerPublicKey
3. ブロック（ヘッダー）検証可能データ

ブロックとトランザクションのエンティティハッシュへの入力は似ている。唯一の違いは、ネットワーク GenerationHashSeed がブロックのエンティティハッシュ計算への入力ではないことである。このフィールドをブロックに含めることは目的を果たさない。少なくとも 1 つのトランザクションを含むブロックは、トランザクションエンティティハッシュがネットワーク間で異なるため、異なるネットワークでリプレイされることができない。その結果、ブロックの TransactionsHash も異なるものになる。

8 ブロックチェーン

((

先を見すぎることは誤りだ。運命の鎖の一つのリンクだけが一度に扱えるものだ。))

- Winston Churchill

Symbol はブロックを繋ぎ合わせるブロックチェーンと呼ばれる公開台帳を中心としている。完全なトランザクション履歴がブロックチェーンに保持される。すべてのブロック、およびブロック内のトランザクションは、決定論的かつ暗号的に順序付けられている。ブロックあたりの最大トランザクション数はネットワークごとに設定できる。

8.1 ブロック難易度

ネメシスブロックは 10^{14} の事前定義された初期難易度を持つ。すべての難易度は最小 10^{13} から最大 10^{15} の間にクランプされる。

新しいブロックの難易度は、最近確認されたブロックの難易度とタイムスタンプから導出される。考慮されるブロック数はネットワークごとに設定できる。

`network:maxDifficultyBlocks` 未満しか利用できない場合は、利用可能なもののみが考慮される。それ以外の場合、難易度は以下の方法で最後の `network:maxDifficultyBlocks` ブロックから計算される：

$$\begin{aligned}d &= \frac{1}{n} \sum_{i=1}^n (\text{block}_i \text{ の難易度}) && \text{(平均難易度)} \\t &= \frac{1}{n} \sum_{i=1}^n (\text{block}_i \text{ の作成時間}) && \text{(平均作成時間)} \\difficulty &= d \frac{\text{blockGenerationTargetTime}}{t} && \text{(新難易度)}\end{aligned}$$

このアルゴリズムは、望ましい `network:blockGenerationTargetTime` ネットワーク設定に近い平均時間でブロックを生成する。

新しい難易度が最後のブロックの難易度より 5% 以上大きいまたは小さい場合、変化は 5% に制限される。ブロックあたりの最大変化率 5% により、50% 未満のインポートランスを持つ攻撃者が秘密裏により良いチェーンを作成することが困難になる。難易度は現在ハーベスティングしているインポートランスの総量とほぼ相関しているため、攻撃者の秘密チェーンは必然的にはるかに低い難易度を持つことになる。ブロッ

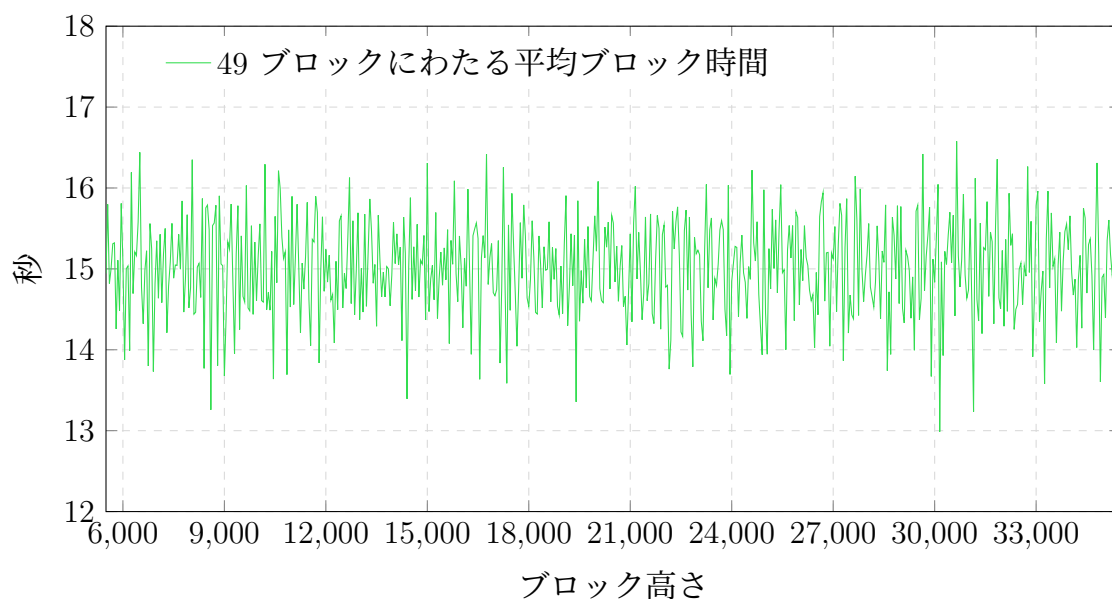


Figure 18: 開発ネットワークの平均ブロック時間（目標ブロック時間 = 15 秒）

クあたりの最大難易度低下を制限することで、攻撃者のチェーンはすぐにメインチェーンより遅れる。難易度が攻撃者のインポートランスに比例したものに調整される頃には、メインチェーンははるか先に進んでいる。攻撃者の秘密チェーンの最初はブロック時間が `network:blockGenerationTargetTime` よりも大幅に高くなる。

8.2 ブロックスコア

ブロックのスコアは難易度と最後のブロックからの経過時間（秒）から導出される：

$$score = difficulty - \text{最後のブロックからの経過時間} \quad (\text{ブロックスコア})$$

8.3 ブロック生成

新しいブロックを作成するプロセスをハーベスティングと呼ぶ。ハーベスティングアカウントはブロックに含めるトランザクションから手数料のほとんどを受け取る。これにより、ハーベスターは有効なブロックを作成し、できるだけ多くのトランザクションを追加するインセンティブが生まれる。

各ブロックで支払われた手数料は3つのバケットのいずれかに分割される。`network:harvestBeneficiaryPercentage` がゼロ以外の場合、その割合の手数料がハーベスターの指定受益者にクレジットされる。`network:harvestNetworkPercentage` がゼロ以外の場合

合、その割合の手数料がネットワークで指定された `network:harvestNetworkFeeSinkAddress` にクレジットされる。このアカウントはネットワークを自立可能にし、投票ノードへの報酬などのインセンティブプログラムに使用できることを意図している。残りの手数料はハーベスターアカウントにクレジットされる。

アカウントが以下のすべてに該当する場合、ハーベスティングの資格がある：

1. 最後のインポートانس再計算高さでのインポートانسスコアがゼロ以外である。
2. ネットワーク定義の `network:minHarvesterBalance` 以上の残高がある。
3. ネットワーク定義の `network:maxHarvesterBalance` 以下の残高がある ²¹。
4. アカウントに VRF 公開鍵が登録されている。

アカウント所有者は資金のある秘密鍵を公開するリスクを避けるために、そのインポートانسを別のアカウント²²に委任できる。

ハーベスターが受け取る実際の報酬はネットワーク設定に基づいてカスタマイズできる。`inflation` 設定がゼロ以外の値を持つ場合、各ハーベスト済みブロックに追加のインフレーションブロック報酬が含まれることがある。これによりハーベスティングがより利益を生む。ハーベスティング手数料共有が有効な場合 (`network:harvestBeneficiaryPercentage` を通じて)、ハーベスターはそのハーベスティングキーをホストするノードへの手数料の一部を放棄する。これにより、ネットワークノードの運営はより利益を生むが、ハーベスティングは利益が減る。

各ブロックはそのブロックに含まれるすべてのトランザクションが支払わなければならない実効手数料を決定する手数料乗数を指定する必要がある。通常、ノード所有者はノードによってハーベストされるすべてのブロックに適用される `node:minFeeMultiplier` を設定する。以下の条件を満たすトランザクションのみがそのノードの未確認トランザクションキャッシュに入り、そのノードによってハーベストされるブロックへの含有の資格を得る：

$$\text{トランザクション最大手数料} \geq \text{minFeeMultiplier} \cdot \text{トランザクションサイズ (バイト)} \quad (12)$$

拒否されたトランザクションは依然としてより低い要件を持つ他のノードによってハーベストされたブロックに含まれる可能性がある。ハーベストされるブロックへの含有のためにトランザクションを選択するための特定のアルゴリズムは `node:`

²¹ この機能は主にコアファンドと取引所アカウントのハーベスティングを防ぐことを目的としている。

²² 詳細は <https://nemtech.github.io/concepts/harvesting.html#accountlinktransaction> を参照のこと。

`transactionSelectionStrategy` 設定によって構成される。**Symbol** は 3 つの組み込み選択戦略²³を提供する：

1. `oldest` (最古)：

これは最もリソース消費の少ない戦略であり、高 TPS ネットワークに推奨される。トランザクションは受信した順序で新しいブロックに追加される。これにより最も古いトランザクションが最初に選択され、タイムアウトするトランザクション数を最小限にしようとする。その結果、この戦略はハーベスターにとって利益最大化になることはほとんどない。

2. `maximize-fee` (手数料最大化)：

ブロック内のすべてのトランザクションの累積手数料が最大化されるようにトランザクションが選択される。利益最適化ノードはこの戦略を選択する。総ブロック手数料を最大化することは、含まれるトランザクション数の最大化を意味するわけではない。実際、多くの場合、ハーベスターは利用可能なトランザクションのサブセットのみを含める。

3. `minimize-fee` (手数料最小化)：

この戦略は最も低い最大手数料乗数を持つトランザクションを最初に選択する。利他的なノードはこの戦略を非常に低い `node:minFeeMultiplier` と共に選択する。この設定がゼロの場合、ハーベスターは最初にゼロ手数料のトランザクションを含める。これによりユーザーは無料でブロックチェーンに含まれるトランザクションを送信できる！実際には、これをサポートするノードはほんのわずかしかないう可能性が高い。そのようなノードのサブセットが動作していても、ゼロ手数料のトランザクションはネットワーク内で最も少ないノード数しかサポートしないため、ブロックに含まれる確率は常に最も低くなる。

トランザクションは静的および動的な両方の量の転送を開始できる。静的な量は固定であり、外部要因に依存しない。例えば、転送トランザクション²⁴で指定された量は静的である。指定された正確な量が常に送信者から受信者に転送される。対照的に、動的な量は平均トランザクションコストに対して可変である。これらは通常、ネームスペースやモザイクなどのユニークなネットワーク成果物を取得するために支払われる手数料のために予約されている。そのような成果物には、一律手数料は望ましくない。これは市場の動向に反応できないからである。同様に、単一ブロックの `FeeMultiplier` だけを使用することも問題がある。これはハーベスターがゼロ手数料で自己ハーベストしたブロックに登録を含めることで不正行為をして成果物を無料で受け取れるからである。代わりに、動的手数料乗数が使用される。この乗数は最後の `network:maxDifficultyBlocks` ブロックの `FeeMultiplier` 値の中央値として計算され

²³すべての場合において、利用可能なすべてのトランザクションはすでに `node:minFeeMultiplier` が示す要件を満たしている必要がある。

²⁴詳細は <https://nemtech.github.io/concepts/transfer-transaction.html> を参照のこと。

る。値が十分にない場合やゼロ値の代替として `network:defaultDynamicFeeMultiplier` が使用される。後者の調整により実効量が常にゼロ以外になることが保証される。実効量を求めるために、基本量に動的手数料乗数を掛ける。

$$\text{実効量} = \text{基本量} \cdot \text{動的手数料乗数}$$

8.4 ブロック生成ハッシュ

ブロックの生成ハッシュは前のブロック生成ハッシュとブロックに含まれる VRF 証明から導出される：

$gh(1) = \text{generationHash}$ (生成ハッシュ)

$gh(N) = \text{verify_vrf_proof}(\text{proof}(\text{block}(N)), gh(N-1), \text{アカウントの VRF 公開鍵})$

生成ハッシュを VRF に依存させることで、実質的にランダムで予測不可能になる。これは完全な情報を持つ敵対者でさえ次のハーベスターの身元を知ることができないため重要である。ブロックがネットワークにプッシュされるまで、その生成ハッシュはハーベスター以外の誰にも知られていない。したがって、最良のブロックがネットワークにプッシュされるまで、各生成ハッシュは前の生成ハッシュに依存しているため、次のブロックの生成ハッシュ計算への入力はその時点まで不明のままである。

VRF 公開鍵はブロックのハーベストに使用される前にブロックチェーンに登録する必要がある。この事前登録要件は、敵対者が次のブロックへのヒットを最大化できる任意の VRF 公開鍵を選択することを防ぐ。その結果、敵対者はヒットを最大化しようとして多くのブロックを生成できない。

8.5 ブロックヒットとターゲット

特定のネットワーク時刻にアカウントが新しいブロックを作成できるかどうかを確認するために、以下の値が比較される：

- *hit*: ヒットが必要なブロックごとの値を定義する。
- *target*: 最後にハーベストしたブロックからの経過時間とともに増加するハーベスターごとのパワーを定義する。

$hit < target$ のときにアカウントは新しいブロックを作成できる。*target* は経過時間に比例するため、すべてのアカウントが運が悪く非常に高いヒットを生成したとしても、一定の時間後に新しいブロックが作成される。

委任ハーベスティングの場合、委任アカウントのインポータンスではなく元のアカウントのインポータンスが使用される。

ターゲットは以下のように計算される²⁵：

$$\begin{aligned} multiplier &= 2^{64} \\ t &= \text{最後のブロックからの経過時間 (秒)} \\ b &= 8999999998 \cdot (\text{アカウントインポータンス}) \\ i &= \text{チェーンの総インポータンス} \\ d &= \text{新ブロック難易度} \\ target &= \frac{multiplier \cdot t \cdot b}{i \cdot d} \end{aligned}$$

ブロック時間平滑化を有効化でき、これによりより安定したブロック時間が実現される。有効化された場合、上記の *multiplier* は以下のように計算される²⁶：

$$\begin{aligned} factor &= \text{blockTimeSmoothingFactor}/1000.0 \\ tt &= \text{blockGenerationTargetTime} \\ power &= factor \cdot \frac{\text{最後のブロックからの経過時間 (秒)} - tt}{tt} \\ smoothing &= \min(e^{power}, 100.0) \\ multiplier &= \text{integer}(2^{54} \cdot smoothing) \cdot 2^{10} \end{aligned}$$

ヒットは $2^{54} \left| \ln \left(\frac{gh}{2^{256}} \right) \right|$ の 64 ビット近似値であり、*gh* は新しい生成ハッシュである²⁷。

まず、log を底 2 で書き換える：

$$hit = \frac{2^{54}}{\log_2(e)} \cdot \left| \log_2 \left(\frac{gh}{2^{256}} \right) \right|$$

$\frac{gh}{2^{256}}$ は常に < 1 であるため、log は常に負の値を返すことに注意されたい。
 $\log_2 \left(\frac{gh}{2^{256}} \right)$ は $\log_2(gh) - \log_2(2^{256})$ と書き換えられる。

²⁵実装では丸めによる問題を避けるために浮動小数点演算ではなく 256 ビット整数を使用する。

²⁶実装では丸めによる問題を避けるために浮動小数点演算ではなく固定小数点演算を使用する。具体的には、高 112 ビットが整数部を表し、低 16 ビットが小数部を表す 128 ビット固定小数点数が使用される。 $\log_2(e)$ は $14426950408 / 10000000000$ として近似される。計算された *power* が負すぎる場合、*smoothing* はゼロに設定される。

²⁷実装では丸めによる問題を避けるために浮動小数点演算ではなく 128 ビット整数を使用する。 $\log_2(e)$ は $14426950408889634 / 10000000000000000$ として近似される。

絶対値を取り除いて書き換えると：

$$\begin{aligned} scale &= \frac{1}{\log_2(e)} \\ hit &= scale \cdot 2^{54}(\log_2(2^{256}) - \log_2(gh)) \end{aligned}$$

これをさらに簡略化すると：

$$hit = scale \cdot (2^{54} \cdot 256 - 2^{54} \cdot \log_2(gh))$$

実装では、新しい生成ハッシュの最初の 32 個の非ゼロビットのみを使用して対数を近似する。エッジケースの追加処理もある。

また、*hit* は指数分布を持つことに注意されたい。したがって、インポートランスが多くのアカウントに分割されても新しいブロックを作成する確率は変わらない。

8.6 自動委任ハーベスター検出

`user:enableDelegatedHarvestersAutoDetection` が設定されている場合、サーバーは特別な転送メッセージを通じて他のアカウントが委任ハーベスターとして登録することを許可する。サーバーはノード証明書公開鍵に一致するアカウントに送信されたすべての転送メッセージを検査し、一致するものをファイルキューにプッシュする。定期的に、スケジュールされたタスクがキューに入れられたすべてのメッセージを検査する。予期しないまたは不正な内容を含むメッセージは無視され廃棄される。有効なメッセージは復号化して処理される。

メインアカウントは以下のすべてのリンクが設定されている場合、ノードで委任ハーベスティングを行う資格がある²⁸：

- メインアカウントが委任ハーベストリクエストに署名していること。
- リンクされたリモート公開鍵が暗号化されたリンクされたリモートハーベスター秘密鍵と一致すること。
- VRF 公開鍵が暗号化された VRF 秘密鍵と一致すること。
- ノード公開鍵がノードの証明書公開鍵と一致すること（[12.2: 接続ハンドシェイク](#)を参照）。

²⁸これらの条件がすべて満たされていても、ノード所有者は引き続き資格のあるアカウントによる委任ハーベストを許可しないことを決定できる。

メッセージは敵対者がプレーンテキストのリモートハーベスターおよび VRF 秘密鍵を取得することを防ぐために部分的に暗号化されている。AES256 GCM は使用される（対称）暗号化スキームである。暗号化キーはサーバーのノード証明書キーペアとクライアントによって生成されたランダムなエフェメラルキーペアから導出される。

各メッセージには以下の内容が期待される：

名前	サイズ	暗号化？	説明
0xE201735761802AFE	8 バイト	いいえ	メッセージがハーベストリクエストであることを示すマジックバイト
エフェメラル公開鍵	32 バイト	いいえ	対称暗号化キーを導出するために使用される公開鍵
AES GCM タグ	16 バイト	いいえ	暗号化データに対する MAC タグ
AES GCM IV	12 バイト	いいえ	AES GCM アルゴリズムの初期化ベクター
署名秘密鍵	32 バイト	はい	メインアカウントにリンクされたリモートハーベスター署名秘密鍵
VRF 秘密鍵	32 バイト	はい	メインアカウントにリンクされた VRF 秘密鍵

Table 4: ハーベストリクエストメッセージフォーマット

可能であれば、アナウンスされた委任ハーベスター秘密鍵はサーバーによってブロックをハーベストするために使用される。サーバーは最大 *harvesting:maxUnlockedAccounts* 個のハーベスターを持てる。その制限に達した場合、新しい委任ハーベスターの評価は *harvesting:delegatePrioritizationPolicy* 設定に基づいて行われる。ポリシーが *Age* に設定されている場合、より早くアナウンスされたアカウントが優先される。その結果、新しい委任ハーベスターは既存の委任ハーベスターを置き換えられない。ポリシーが *Importance* に設定されている場合、より高いインポートランスを持つアカウントが優先される。その結果、新しい委任ハーベスターはより少ないインポートランスを持つ既存の委任ハーベスターを置き換えられる。

成功したアナウンスは *harvesters.dat* ファイルに格納される。受け入れられた委任ハーベスターはサーバーの再起動をまたいで永続化される。サーバーは特定の委任ハーベスターで現在ハーベスティングしているかどうかの明示的な確認を提供しない。ブロックチェーンは委任ハーベスティングに関連するすべてのリンクの記録のみを格納するが、現在のアクティビティは格納しない。

8.7 ブロックチェーン同期

ブロックのチェーンにスコアを割り当てるために、構成ブロックのスコアを合計する：

$$score = \sum_{block \in blocks} \text{ブロックスコア} \quad (\text{ブロックチェーンスコア})$$

ブロックチェーン同期は分散コンセンサスの維持に不可欠である。定期的に、ローカルノードはリモートノードにそのチェーンについて問い合わせる。リモートノードはレピュテーション（[13: レピュテーション](#)を参照）を含む様々な要因に基づいてパートナーのセットから選択される。

リモートノードがより高いスコアのチェーンを約束する場合、ローカルノードはリモートノードから提供されたハッシュを検査することで最後の共通ブロックを見つけようとする。確定的ファイナライゼーションが有効な場合、最後の共通ブロックを見つけるためにバイナリ検索が実行される。検索空間は最後のファイナライズ済みブロックから現在のローカル高さまでのすべてのハッシュである。成功した場合、リモートノードは設定が許す限り多くのブロックを提供する。

提供されたチェーンが有効な場合、ローカルノードは自身のチェーンをリモートチェーンに置き換える。提供されたチェーンが無効な場合、ローカルノードはチェーンを拒否し、リモートノードとの同期試行が失敗したとみなす。

[Figure 19](#) はプロセスをより詳細に示す。

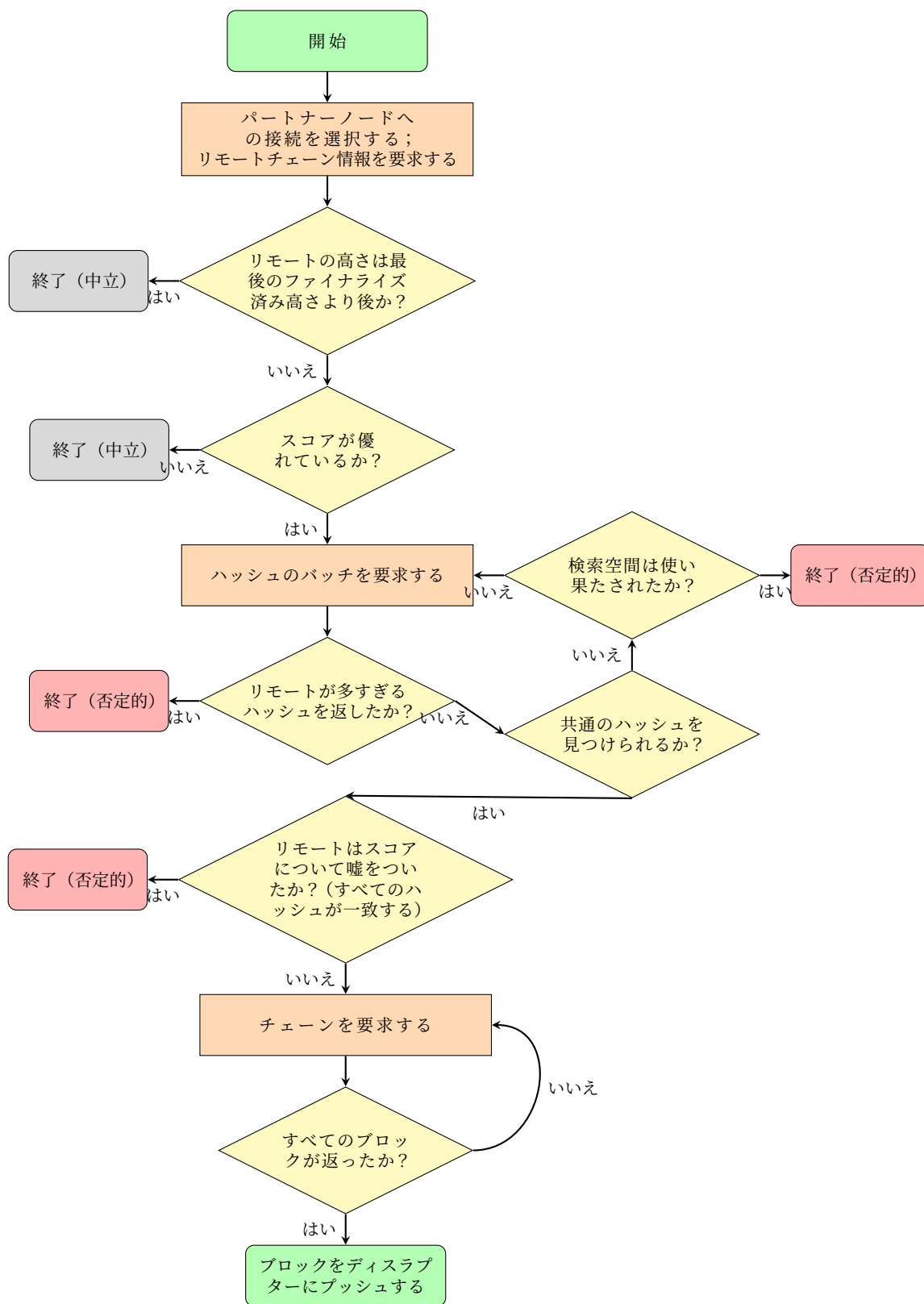


Figure 19: ブロックチェーン同期フローチャート

8.8 ブロックチェーン処理

実行

概念的に、新しいブロックを受信すると、一連のステージで処理される²⁹。処理の前に、ブロックとそのトランザクションは通知の順序付きストリームに分解される。通知は **Symbol** で使用される基本的な処理単位である。

ブロックから通知の順序付きストリームを抽出するために、そのトランザクションが順番に分解され、続いてブロックレベルのデータが分解される。各分解によって生成された通知がストリームに追加される。このプロセスの最後に、通知ストリームはブロックとそのトランザクションで指定されたすべての状態変化を完全に記述する。

通知のストリームが準備されると、各通知が個別に処理される。まず、ブロックチェーン状態から独立して検証される。次に、現在のブロックチェーン状態に対して検証される。いずれかの検証が失敗した場合、含まれるブロックは拒否される。そうでない場合、通知によって指定された変更がメモリ内ブロックチェーン状態に適用され、次の通知が処理される。このシーケンスにより、ブロック内のトランザクションは同じブロック内の前のトランザクションによって行われた変更依存できる。

ブロックによって生成されたすべての通知が処理された後、`ReceiptsHash` (7.2.4: [レシートハッシュ](#) を参照) と `StateHash` (7.3: [状態ハッシュ](#) を参照) フィールドが計算され、正確性がチェックされる。重要なことに、`network:enableVerifiableState` が有効な場合、これはすべての状態パトリシア木が更新されるポイントである。

ロールバック

以前に確認されたブロックを取り消す必要がある場合がある。これはフォーク解決を可能にするために必要である。例えば、より悪いブロックをより良いブロックに置き換えるためである。**Symbol** では、最大 `network:maxRollbackBlocks` が一度にロールバックできる。この設定より大きいフォークは調整不可能である。

ブロックがロールバックされる際、通知の順序付きストリームに分解される。このストリームは実行中に使用されたストリームに対して逆順になっている。ブロック内のトランザクションは同じブロック内の前のトランザクションによる変更依存している可能性があるため、依存関係が取り消される前に取り消す必要がある。

通知のストリームが準備されると、各通知が個別に処理される。ロールバック操作は有効であることがわかっている以前の状態にブロックチェーンを戻すため、検証は不要である。代わりに、通知によって指定された変更がメモリ内ブロックチェーン状態から単純に元に戻され、次の通知が処理される。

²⁹ これらのステージのより詳細な説明は [9.1: コンシューマー](#) で見つかる。

ブロックチェーン部分によって生成されたすべての通知が処理された後、以前のブロックチェーン状態が復元される。`network:enableVerifiableState` が有効な場合、メモリ内状態ハッシュも更新する必要がある。すべての木の変更を個別に適用する代わりに、メモリ内状態ハッシュは最後にロールバックされたブロック前の共通ブロックの状態ハッシュに強制的にリセットされる。

9 ディスラプター

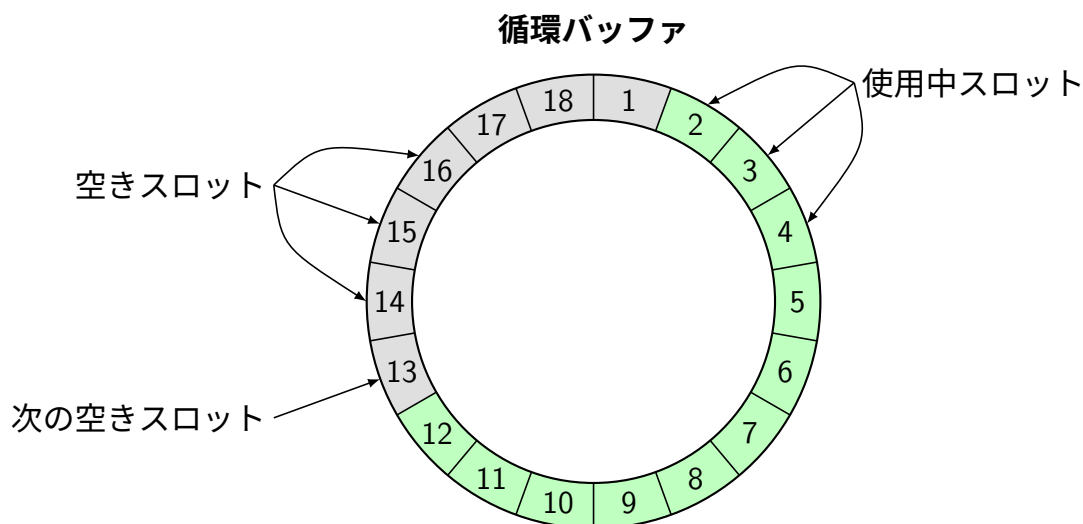
〃

死は偉大なる崩壊者だ。それは私たちを人生の鏡の前に立たせ、真実の探求へと誘い、再生が可能となり自分自身を新たに作り直せる裸の真実を明らかにする。

〃

- B.G. Bowers

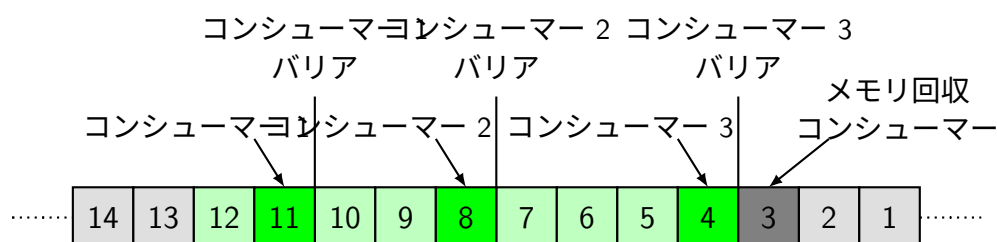
ONE**Symbol** の主な目標の一つは高いスループットを達成することである。この目標の達成を助けるために、ディスラプター³⁰パターンがほとんどのデータ処理に使用される。ディスラプターはリングバッファデータ構造を使用して処理が必要なすべての要素を保持する。新しい要素はリングバッファの次の空きスロットに挿入される。完全に処理された要素は新しい要素のためのスペースを作るために削除される。リングバッファは有限数のスロットを持つため、処理が新しい挿入に追いつけない場合にスペースが不足することがある。そのような場合の**Symbol**の動作は、サーバーを終了するか、スロットが利用可能になるまで新しいデータを破棄するように設定できる。



リングバッファ内の各要素は1つ以上のコンシューマーによって処理される。各コンシューマーは単一の要素を入力として取る。一部のコンシューマーは入力からデータを計算して要素に添付し、他のコンシューマーは要素を検証したり要素のデータを使用してグローバルなチェーン状態を変更したりする。一部のコンシューマーは以前のコンシューマーが行った処理に依存する。したがって、コンシューマーは常に事前に定義された順序で入力要素に作用する必要がある。これを保証するために、各

³⁰[https://en.wikipedia.org/wiki/Disruptor_\(software\)](https://en.wikipedia.org/wiki/Disruptor_(software))

コンシューマーには関連するバリアがある。バリアはコンシューマーが直前のコンシューマーによってまだ処理されていない要素を処理することを防ぐ。最後のコンシューマーは処理中に使用されたすべてのメモリを回収する。コンシューマーは要素がすでに既知であるか、何らかの理由で無効である場合に要素の完了ステータスを `CompletionStatus::Aborted` に設定できる。後続のコンシューマーはアボートされた要素を無視する。



9.1 コンシューマー

Symbolでは、ブロックディスラプターは着信ブロックとブロックチェーン部分を処理するために使用される。ブロックチェーン部分とは複数のブロックで構成される入力要素である。このディスラプターは主にブロックチェーンの検証、調整、および成長を担当する。

トランザクションディスラプターは着信の未確認トランザクションを処理するために使用される。完全に処理されたトランザクションは未確認トランザクションキャッシュに追加される。

すべてのディスラプターは入力要素のすべての処理を実行するコンシューマーのチェーンに関連付けられている。異なるディスラプターは異なるコンシューマーチェーンを使用してカスタマイズされる。すべてのコンシューマーは処理されているデータを検査でき、一部はそれを変更できる。

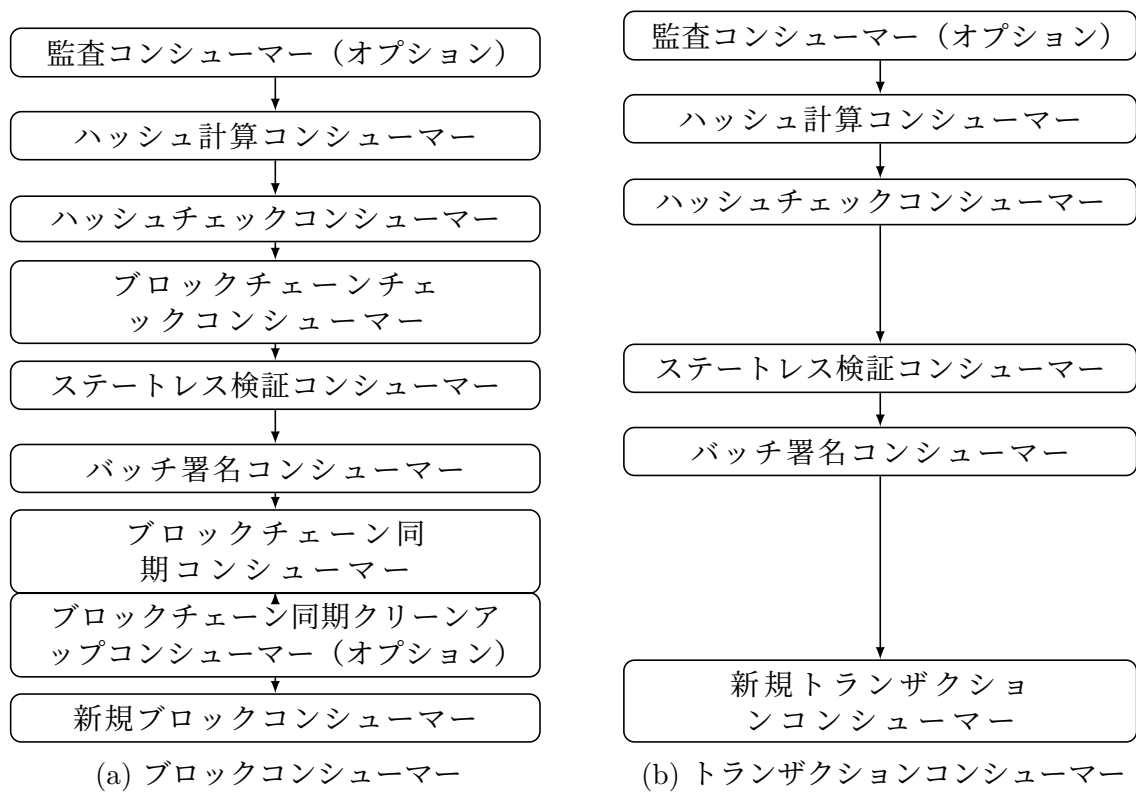


Figure 20: **Symbol** コンシューマーチェーン

9.1.1 共通コンシューマー

ブロックディスラプターとトランザクションディスラプターは多くのコンシューマーを共有する。

監査コンシューマー

このコンシューマーはオプションであり、ノード設定で有効化できる。有効化された場合、すべての新しい要素はディスクに書き込まれる。これにより着信ネットワークアクションを再生でき、デバッグに役立つ。

ハッシュ計算コンシューマーとハッシュチェックコンシューマー

ネットワークは複数のサーバーで構成されており、それらは要素を他のいくつかのサーバーにブロードキャストするため、サーバーが同じ要素を何度も受信することは非常に一般的である。パフォーマンス上の理由から、要素が既に処理されているかどうかを早い段階で検出し、再処理を避けることが望ましい。

ハッシュ計算コンシューマーは要素に関連するすべてのハッシュを計算する。ハッシュチェックコンシューマーはハッシュを使用して、最近確認されたすべての要素のハッシュを含む直近キャッシュを検索する。トランザクションディスラプターで使用されるコンシューマーは、ハッシュキャッシュ（確認済みトランザクションのハッシュを含む）と未確認トランザクションキャッシュも検索する。ハッシュがいずれかのキャッシュで見つかった場合、要素は `CompletionStatus::Aborted` としてマークされ、以降の処理はバイパスされる。

ステートレス検証コンシューマー

このコンシューマーは要素内の各エンティティを検証することで、状態非依存の検証を処理する。これは多くのスレッドを使用して並列に実行できる。各プラグインはステートレスバリデーターを追加できる。

ステートレス検証の例として、ブロックがネットワークで許可されているよりも多くのトランザクションを含まないことの検証がある。このチェックはネットワーク設定に依存するが、グローバルなブロックチェーン状態には依存しない。

バッチ署名コンシューマー

このコンシューマーは要素内のすべてのエンティティのすべての署名を検証する。これはバッチ検証を使用するため、ステートレス検証コンシューマーとは別になっている。パフォーマンス向上のため、このコンシューマーは多くの署名を個別にではなく一度にバッチで処理する。これは多くのスレッドを使用して並列に実行できる。

メモリ回収コンシューマー

このコンシューマーは要素の処理を完了し、それに関連するすべてのメモリを解放する。処理中に更新されたすべてのトランザクションのステータスのダウンストリーム伝播をトリガーする。同期操作の全体的な結果は、同期パートナーのレピュテーションを更新し、場合によってはバンするために使用される（[13: レピュテーション](#)を参照）。

9.1.2 追加ブロックコンシューマー

ブロックディスラプターはいくつかのブロック固有のコンシューマーも使用する。

ブロックチェーンチェックコンシューマー

このコンシューマーは要素内に含まれるチェーン部分の状態非依存の整合性チェックを実行する。以下を確認する：

- チェーン部分が多すぎるブロックで構成されていないこと。
- チェーン部分の最後のブロックのタイムスタンプが未来に遠すぎないこと。
- チェーン部分内のすべてのブロックがリンクされていること。
- チェーン部分内に重複したトランザクションがないこと。

ブロックチェーン同期コンシューマー

このコンシューマーは最も複雑なものである。ローカルサーバーのチェーン状態を必要とするか変更するすべてのタスクがこのコンシューマーで実行される。

まず、新しいチェーン部分が既存のチェーンに接続できるかどうかを確認する。チェーン部分が末尾ブロックの前のブロックに接続する場合、共通ブロックに到達するまで末尾ブロックから始まるすべてのブロックが逆順に取り消される。

次に、ステートフル検証を実行してから変更を観察することで各ブロックを実行する。ステートレス検証は前のコンシューマーによって実行されているためスキップされる。検証失敗がある場合、チェーン部分全体が拒否される。それ以外の場合、すべての変更はチェーン状態（ブロックとキャッシュストレージの両方）にコミットされ、未確認トランザクションキャッシュが更新される。

最後に、このコンシューマーはロールバックできない最新のブロックである最後のファイナライズ済みブロックを決定する。確率的ファイナライゼーションが有効な場合、最後のファイナライズ済みブロックは最後のブロックの `network: maxRollbackBlocks` ブロック前である。確定的ファイナライゼーションが有効な場合、最後のファイナライズ済みブロックは最後のファイナライゼーション証明で参照されるブロックである。コンシューマーは最後のファイナライズ済みブロック前のブロックへのロールバックを可能にするためにのみ必要なデータをすべてグローバルなブロックチェーン状態からプルーニングする。

このコンシューマーはチェーン状態を変更し、それへの書き込みアクセスを必要とする **Symbol** システムの唯一の部分である。

ブロックチェーン同期クリーンアップコンシューマー

このコンシューマーはオプションであり、ノード設定で有効化できる。有効化された場合、ブロックチェーン同期コンシューマーによって作成されたすべてのファイルを削除する。このコンシューマーはブローカーなしでサーバーが実行されている場合にのみ有効化すべきである。

新規ブロックコンシューマー

このコンシューマーはサーバーによってハーベストされたか、リモートサーバーからプッシュされた単一のブロックをネットワーク内の他のサーバーに転送する。

9.1.3 追加トランザクションコンシューマー

トランザクションディスラプターは単一のトランザクション固有のコンシューマーを使用する。

新規トランザクションコンシューマー

このコンシューマーは有効な署名を持ちステートレス検証をパスしたすべてのトランザクションをネットワークに転送する。ステートフル検証はトランザクションが未確認トランザクションキャッシュに追加されるまで実行されない。転送はステートフル検証の前に意図的に行われる。これは一部のサーバーが他のサーバーによって受け入れられる可能性のあるトランザクションを拒否することがあるためである（例：トランザクションのローカルサーバーに対する手数料が低すぎる場合）。その後、転送されたトランザクションにステートフル検証が実行され、有効なものが未確認トランザクションキャッシュに保存される。

10 未確認トランザクション

((

人は現実から逃げるために本を読むとは思わない。人は、そこにあると知っている——しかしまだ経験していない——現実を確認するために本を読む。

)

- Lawrence Durrell

ANY（まだ）ブロックに含まれていないトランザクションは未確認トランザクションと呼ばれる。これらのトランザクションは有効な場合も無効な場合もある。有効な未確認トランザクションはハーベストされたブロックへの収録の対象となる。トランザクションがブロックチェーンで承認されたブロックに追加されると、確認済みとなる。

未確認トランザクションがノードに届くタイミングは以下のとおりである：

1. クライアントが新しいトランザクションをノードに直接送信する場合。
2. ボンデッドアグリゲートトランザクションが必要なコサインチャをすべて収集して完成し、パーシャルトランザクションキャッシュから昇格する場合。
3. ピアノードがトランザクションをノードにブロードキャストする場合。
4. ピアノードが未確認トランザクションのリクエストに応答する場合。最適化として、リクエスト側ノードはすでに知っているトランザクションを示すことで、重複したトランザクションの受信を避ける。また、ブロック作成時に使用する最小手数料乗数（[8.3: ブロック生成](#)を参照）も提供する。これにより、リモートノードはリクエスト側ノードにすぐに拒否されるような未確認トランザクションを返さなくなる。

未確認トランザクションがノードに届くと、トランザクションディスラプター（[9: ディスラプター](#)を参照）に追加される。以前に受信されておらず、ステートレス検証をパスしたトランザクションはすべてピアノードにブロードキャストされる。この時点でも、ノードはブロードキャストされたトランザクションを拒否する可能性がある（ステートフル検証はブロードキャスト後に実行されるため）。ノードの設定が異なるため、あるノードが特定の未確認トランザクションを受け入れ、他のノードが拒否することがある。例えば、ノードによって `node:minFeeMultiplier` の設定が異なる場合がある。

10.1 未確認トランザクションキャッシュ

トランザクションがすべての検証をパスすると、ハーベストされたブロックへの収録対象となる。この時点で、ノードはそれを未確認トランザクションキャッシュに追加しようとする。これは 2 つの理由で失敗する場合がある：

1. `node:unconfirmedTransactionsCacheMaxSize` で設定された最大キャッシュサイズに達している場合。
2. キャッシュに含まれる未確認トランザクション数が 1 ブロックに収録できる数以上あり、新しいトランザクションがスパムスロットルによって拒否された場合。

新しいブロックがブロックチェーンに追加されるたびに、ブロックチェーンの状態が変化し、未確認トランザクションキャッシュに影響を与える。キャッシュ内のすべてのトランザクションは追加された時点では有効であっても、永久に有効であるとは限らない。例えば、トランザクションがすでに別のノードによってハーベストされたブロックに含まれている場合や、競合するトランザクションがブロックチェーンに追加された場合がある。つまり、以前は完全に有効だったキャッシュ内のトランザクションが、ブロックチェーンの状態変化後に無効化される可能性がある。また、トランザクションを含むブロックが取り消された場合、それらの以前に確認されたトランザクションの一部が新しいチェーンのいかなるブロックにも含まれなくなることがある。そのような取り消されたトランザクションはキャッシュに追加されるべきである。

これらの考慮事項の結果、ブロックチェーンが変化するたびに未確認トランザクションキャッシュ全体が完全に再構築される。各トランザクションはステートフルバリデーターによって再チェックされ、無効になったか、すでにブロックに含まれている場合はパージされる。そうでなければ、キャッシュに戻される。

10.2 スпамスロットル

未確認トランザクションの開始者は、未確認トランザクションキャッシュにトランザクションを保持するノードに手数料を支払う必要がない。キャッシュは貴重なリソースを使用するため、ノードは大量の未確認トランザクションによるスパムから保護する手段を持つ必要がある。これは、ノードがゼロ手数料トランザクションを寛大に受け入れる場合に特に重要である。

単純に受け入れる未確認トランザクション数を制限することは最適ではない。なぜなら、悪意のある攻撃者がネットワークをスパムしているときでも、通常のユーザーはトランザクションを送信できるべきだからである。アカウントごとの未確認トラン

ザクション数を制限することも良い方法ではない。なぜなら、アカウントは自由に作成できるからである。

Symbol は、攻撃者がキャッシュを完全に自分のトランザクションで埋めることを防ぎながら、誠実なユーザーが新しい未確認トランザクションを成功裏に送信できるスマートなスロットルを実装している。 `node:enableTransactionSpamThrottling` でスロットルを有効化できる。キャッシュがいっぱいでない場合、以下のように動作する：

1. キャッシュに含まれる未確認トランザクション数が 1 ブロックに収録できる数未満の場合、スロットリングはバイパスされる。
2. 新しいトランザクションがボンデッドアグリゲートトランザクションの場合、スロットリングはバイパスされる。
3. それ以外の場合、スパムスロットルが適用される。

$curSize$ をキャッシュ内の現在のトランザクション数、 $maxSize$ をキャッシュの設定済み最大サイズとする。また A の相対的インポートانسを A の相対的インポートانس、つまり 0 から 1 の間の数とする。署名者 A による新しい未確認トランザクション T が届いた場合、アカウント A のフェアシェアは以下のように計算される：

$$maxBoostFee = transactionSpamThrottlingMaxBoostFee$$

$$maxFee = \min(maxBoostFee, T::MaxFee)$$

$$\text{有効インポートانس} = (A \text{ の相対的インポートانس}) + 0.01 \cdot \frac{maxFee}{maxBoostFee}$$

$$\text{フェアシェア} = 100 \cdot (\text{有効インポートانس}) \cdot (maxSize - curSize) \cdot \exp\left(-3 \frac{curSize}{maxSize}\right)$$

アカウント A がすでにキャッシュ内にフェアシェアと同数のトランザクションを持っている場合、新しいトランザクションは拒否される。それ以外の場合は受け入れられる。この数式は、トランザクションの最大手数料を増やすとキャッシュ内の利用可能なスロット数が増加することを示している。ただし、実効インポートانسをブーストするこのメカニズムは `node:transactionSpamThrottlingMaxBoostFee` によって制限される。

Figure 21 は、各種実効インポートانسに対してキャッシュの充填率に応じたスロットのフェアシェアを示している。多数のスロットを占拠しようとする攻撃者は、多くのアカウントを使っても大した利益を得られない。なぜなら、各アカウントのインポートانسが非常に低くなるからである。攻撃者はトランザクションの最大手数料を増やすこともできるが、それはコストが高く、より速く資金を消費することになる。

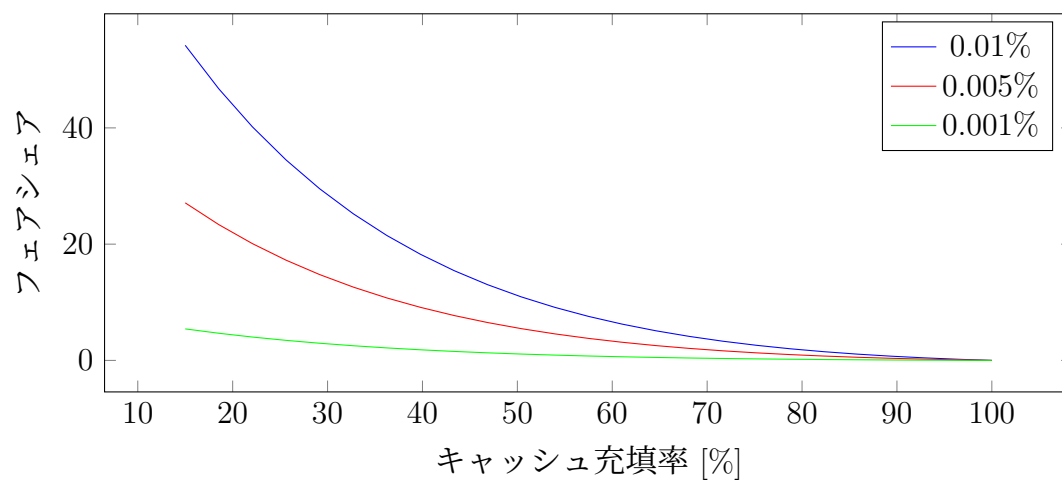


Figure 21: 最大キャッシュサイズ = 10000 のとき、各種実効インポートانسに対するフェアシェア

11 パーシャルトランザクション

((

全体は部分の総和より大きい。

)

- Aristotle

BONDEDボンデッドアグリゲートトランザクション（[6.2: アグリゲートトランザクション](#)を参照）はパーシャルトランザクションとも呼ばれる。パーシャルという名称は、コサイネチャが不足しており、さらにコサイネチャが収集されるまで検証をパスできないことに由来する。

パーシャルトランザクションの処理サポートはパーシャルトランザクションエクステンションによって提供される。ネットワークがボンデッドアグリゲートトランザクションをサポートする場合、このエクステンションはすべての API ノードおよびデュアルノードで有効にすべきである。

パーシャルトランザクションは、このエクステンションが有効になっているネットワーク内のすべてのノード間で同期される。ノードはリモートノードからプッシュされたパーシャルトランザクションとコサイネチャを受動的に受信する。また、パーシャルトランザクションプルタスクを通じてリモートノードにトランザクションとコサイネチャを定期的にリクエストする。最適化として、リクエスト側ノードはすでに知っているトランザクションとコサイネチャを示すことで、重複した情報の受信を避ける。

ハッシュロックプラグインが有効な場合、パーシャルトランザクションがネットワークに受け入れられるためには、ハッシュロックが作成されてトランザクションに関連付けられている必要がある。ハッシュロックは本質的に、組み込みのコサイネチャ収集サービスを使用するために支払われる保証金である。関連するパーシャルトランザクションがハッシュロックの有効期限前にブロックチェーンで完成・確認された場合、保証金は支払者に返還される。それ以外の場合、保証金はロックが期限切れになったブロックのハーベスターに没収される。この機能により、パーシャルトランザクションキャッシュへのスパムが高コストになる。パーシャルトランザクションがキャッシュに入るためには少なくとも一時的に `node:lockedFundsPerAggregate` の支払いが必要だからである。

ノードが新しいパーシャルトランザクションを受信すると、パーシャルトランザクションディスパッチャーにプッシュされる。受信したコサイネチャはキャッシュ内のパーシャルトランザクションと照合され、一致するトランザクションがない場合は即座に拒否される³¹。トランザクションとコサイネチャが一緒に受信された場合、それらは分割され、上記のように個別に処理される。

³¹これは、コサイネチャが受け入れられる前にパーシャルトランザクションがキャッシュ内に存在している必要があることを意味する。

通常のトランザクションディスパッチャー（[9.1: コンシューマー](#) を参照）と比較して、パーシャルトランザクションディスパッチャーは最小限の構成である。

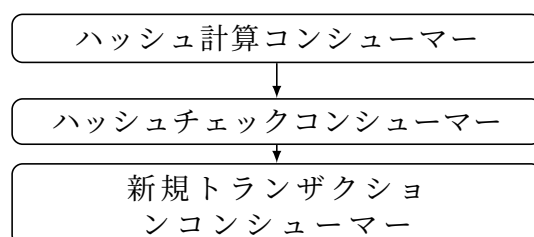


Figure 22: パーシャルトランザクションコンシューマー

ハッシュ計算コンシューマーとハッシュチェックコンシューマーは、[9.1.1: 共通コンシューマー](#) のトランザクションディスパッチャーで説明した通りに動作する。唯一の違いは、ハッシュチェックコンシューマーが以前に確認されたトランザクションを求めてパーシャルトランザクションキャッシュも検索する点である。新規トランザクションコンシューマーは [9.1.3: 追加トランザクションコンシューマー](#) で説明したものと同様の目的を持つ。違いは、未確認トランザクションの代わりにパーシャルトランザクションをブロードキャストして処理する点である。具体的には、パーシャルトランザクションをネットワークにブロードキャストし、有効なものをパーシャルトランザクションキャッシュに追加する。

11.1 パーシャルトランザクション処理

パーシャルトランザクションキャッシュには、追加のコサイネチャを待っているすべてのパーシャルトランザクションが格納される。すべての検証をパスした新しいパーシャルトランザクションが受信されると、キャッシュに追加される。十分なコサイネチャが収集されるか、または無効になるまでキャッシュに留まる。例えば、関連するハッシュロックが期限切れになった場合、トランザクションはパージされる。パーシャルトランザクションが十分なコサイネチャを収集して完成すると、直ちにトランザクションディスパッチャーに転送され、未確認トランザクションとして処理される。

パーシャルトランザクションキャッシュは、すでに含まれているトランザクションとすべての新しいコサイネチャを照合する。キャッシュに追加されるためには、コサイネチャは新規であり、検証可能であり、既存のパーシャルトランザクションに関連付けられている必要がある。以前に受け入れられたコサイネチャが無効になる可能性があり、その場合はパージされるべきである。例えば、コサイネチャの署名者がパーシャルトランザクションに参加しているマルチシグアカウントから削除された場合、コサイネチャは無効になる可能性がある。コサイネチャの照合プロセスはそのようなエッジケースを処理できるほど精巧である。

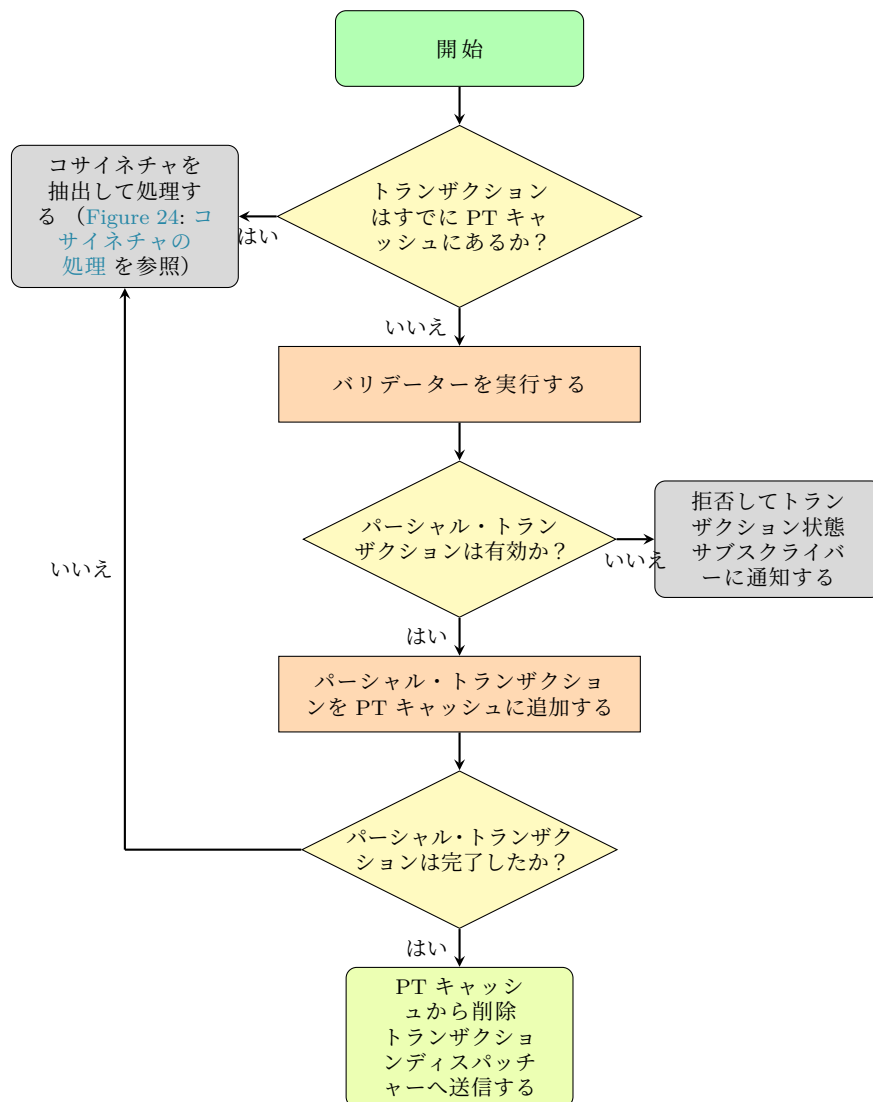


Figure 23: パーシャルトランザクションの処理

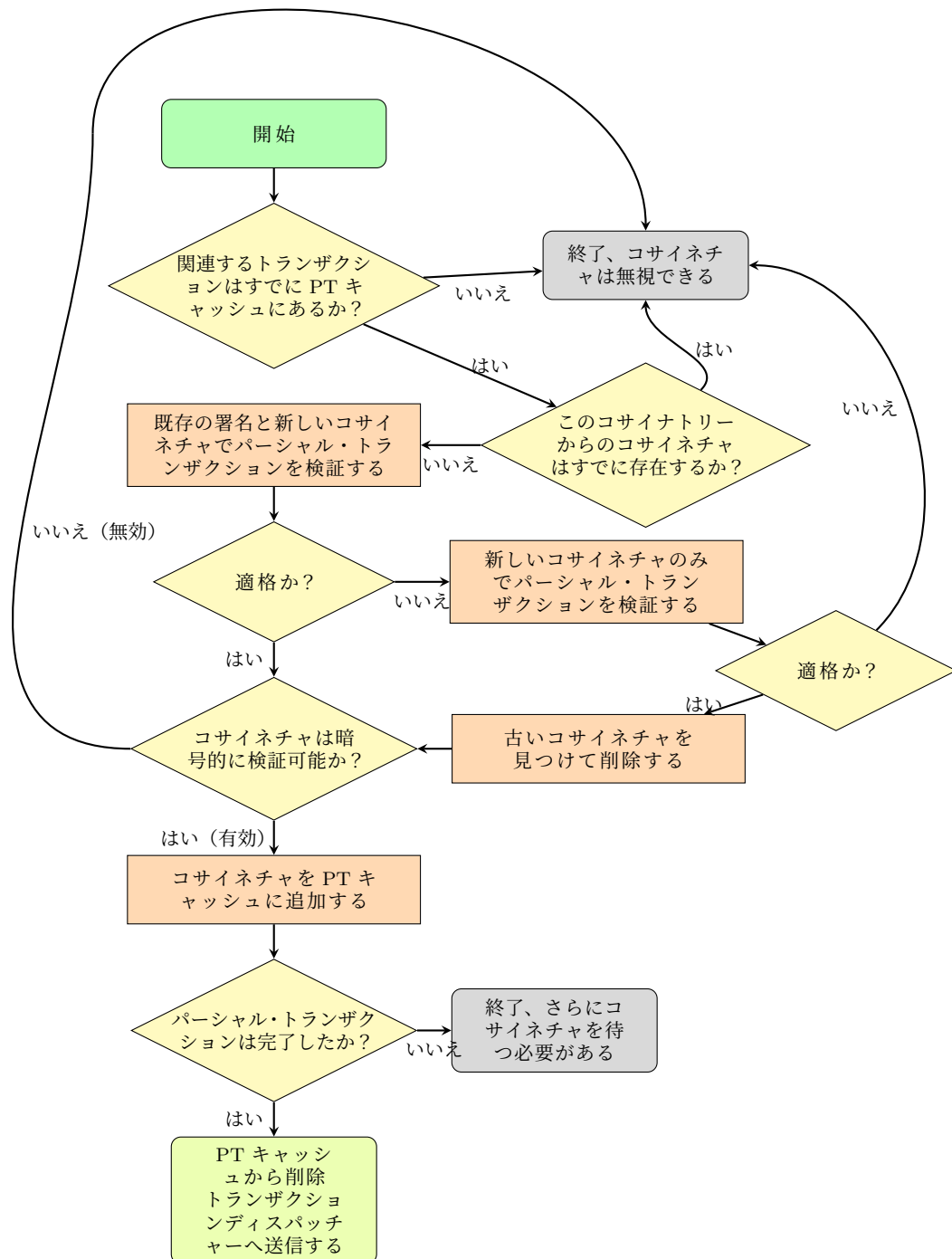


Figure 24: コサインネチャの処理

12 ネットワーク

((

良いネットワークをまとめるには、努力、誠実さ、そして時間が必要だ。

)

- Alan Collins

DYNAMIC動的なノード探索により、ピアツーピアネットワークは成長できる。**Symbol**はこの動的探索をノード探索エクステンションで実装している。公開ネットワークは通常オープンで、任意のノードの参加を許可する。プライベートネットワークは参加できるノードを制限し、フェデレーションシステムとして動作できる³²。**Symbol**は、プライベートネットワークが設定ファイルを通じてすべてのノード関係を指定できるほど柔軟である。

Symbolは `network:nodeEqualityStrategy` で設定可能なノード識別ポリシーをサポートする。有効なポリシーとして、ノードを解決済み IP (host)³³または公開ブートキー (public-key) で識別できる。公開ネットワークでは前者が推奨される。

12.1 ビーコンノード

新たに起動したノードは最初は孤立しており、どのピアにも接続されていない。ブロックの検証やハーベスティングなど意味のある貢献をするには、まずネットワークに参加する必要がある。**Symbol**では、静的ビーコンノードのリストがピア設定ファイルに保存される。ネットワークに参加するために、新しいノードはまずこれらのノードに接続する。これらのファイルはネットワーク内のすべてのノード間で同一である必要はない。

公開ネットワークでは、高可用性のビーコンノード候補のセットを指定することを推奨する。各ノードのピア設定ファイルには、これらのノードのランダムなサブセットが含まれるべきである。ランダムなサブセットは、最初にネットワークに接続する前に一度選択するか、または起動のたびにより頻繁に選択することができる。重要なのは、ビーコンノードが十分に分散されていることである。これにより個々のビーコンノードへの負荷が軽減され、ビーコンノードへの DoS 攻撃が困難になる。これらのノードは高可用性が想定されるため、非ビーコンノードに比べてノード選択 (13.2: 重み付きノード選択 を参照) でわずかに優先される。その他の特別な特権や責任は与えられない。ネットワークへの入口として考えることができる。

³²ハーベスティングモザイクと通貨モザイクを慎重に配布することで、プライベートネットワークは異なるアカウントに権限を委任できる。例えば、十分なハーベスティングモザイクを所有するアカウントのみがブロックを作成でき、非ゼロの通貨を持つアカウントのみがゼロでない手数料のトランザクションを開始できる。

³³ノードの解決済み IP は、ホスト名を指定していない場合にのみ他のノードにブロードキャストされる。動的 IP のノードをサポートするために、ホスト名が優先的に伝播される。

特定のエクステンションは独自のビーコンノードセットを必要とする場合がある。例えば、パーシャルトランザクションエクステンションは独自のビーコンノードセットを別のピア設定ファイルに保存する。このエクステンションが有効なノードは、同じエクステンションが有効な他のノード間でパーシャルトランザクションを追加で同期する必要がある（[11: パーシャルトランザクション](#)を参照）。

ノードのロールは、サポートする機能を指定する。通常、これらは接続するノードが適切なパートナーを選択するために使用される。Peer ロールを持つノードは基本的な同期をサポートする。API ロールを持つノードはパーシャルトランザクション同期をサポートする。Voting ロールを持つノードは、確定的ファイナライゼーションが有効な場合にファイナライゼーション投票手続きに参加する。IPv4 ロールを持つノードは IPv4 通信をサポートする。IPv6 ロールを持つノードは IPv6 通信をサポートする³⁴。ロールは排他的ではない。ノードは複数のロールをサポートできる。

12.2 接続ハンドシェイク

Symbol ノード間のすべての接続は、カスタム検証手順を持つ TLS v1.3 で行われる。各ノードはルート証明書とノード証明書で構成される 2 段階の X509 証明書チェーンを持つことが期待される。すべての証明書は X25519 証明書でなければならない。**Symbol** は他の証明書タイプをサポートしない。

ルート証明書は、アカウントの署名秘密鍵で自己署名されることが期待される。このアカウントはノードの唯一のオーナーと見なされる。ノードとアカウントを暗号学的にリンクすることで、選択アルゴリズムがノードオーナーのインポートانسに基づいてノードの重み付けを行えるようになる。さらに、パートナーノードはこの検証済み ID を使用してレピュテーション（[13: レピュテーション](#)を参照）情報を照合する³⁵。重要な点として、この証明書はノード証明書への署名にのみ使用される。セキュリティのため、その秘密鍵は稼働中のサーバーに保管すべきでない。

ノード証明書はルート証明書によって署名される。ランダムな公開鍵/秘密鍵ペアを含むことができる。この証明書は TLS セッションの認証と、オプションのデータを暗号化するための共有暗号化鍵の導出に使用される³⁶。必要に応じて頻繁に更新できる。

この認証手順は各パートナーノードによって独立して実行される。いずれかのノードがハンドシェイクに失敗すると、接続は即座に終了する。

³⁴ ノードが IPv4 および/または IPv6 ロールを明示的に指定していない場合、IPv4 通信のみをサポートすると見なされる。

³⁵ 公開ネットワークでは、ノードは主にその解決済み IP で識別される。

³⁶ 現在、導出される唯一の暗号化鍵は、自動デリゲートハーベスター検知に関連するメッセージの暗号化および復号化に使用されるものである（[8.6: 自動委任ハーベスター検出](#)を参照）。

12.3 パケット

Symbol は `node:port` で指定されたポートでのネットワーク通信に TCP を使用する。通信は、TCP パケットの上に構築された、それとは異なる高レベルのパケットモデルを中心とする。すべてのパケットは各パケットのサイズと種別を指定する 8 バイトのヘッダーで始まる。完全なパケットが受信されると、さらなる処理の準備が整う。

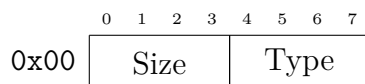


Figure 25: パケットヘッダーのバイナリレイアウト

長期接続と短期接続が混在して使用される。長期接続はブロックやトランザクションの同期など繰り返しの活動に使用される。プッシュとリクエスト/レスポンスのセマンティクスの両方をサポートする。接続はリサイクル対象となる前に `node: maxConnectionAge` 選択ラウンド（[13.1: 接続管理](#) を参照）継続できる。この設定より古い接続は主に他のパートナーノードとの直接インタラクションを可能にするため、次いでゾンビ接続への予防策としてリサイクルされる。

短期接続はノード間のより複雑な多段階インタラクションに使用される。例えば、ノード探索（[12.6: ノード探索](#) を参照）や時刻同期（[16: 時刻同期](#) を参照）に使用される。短期接続は、すべての長期接続が使用中で同期パートナーが利用できない場合に発生する同期枯渇を防ぐのに役立つ。

ハンドラーはパケットの処理に使用される。各ハンドラーは指定されたパケット種別のすべてのパケットを受け入れるために登録される。完全なパケットが処理準備できると、その種別に登録されたハンドラーにディスパッチされる。すべてのハンドラーは一致するパケットを処理のために受け入れなければならない。一部のハンドラーはリクエスト/レスポンスプロトコルを可能にするためにレスポンスパケットを書き込むこともできる。

12.4 接続種別

Symbol では、長期接続は主にリーダーまたはライターとして識別される。これは着信か発信かに直交する。次に目的、またはサービス識別子によって識別される³⁷。これにより、接続を機能によって選択し、より詳細なロギングができる。

リーダー接続は主にパッシブで、他のノードからデータを受信するために使用される。各サーバーは各リーダー接続を非同期的に読み取る。新しいパケットが完全に受

³⁷用語は似ているが、[2.2: Symbol エクステンション](#) で説明するサービスとは無関係である。

信されるたびに、適切なハンドラーにディスパッチされる。一致するハンドラーがない場合、接続は即座に閉じられる。

`node:maxIncomingConnectionsPerIdentity` の制限はすべてのサービスと長期・短期接続に対して適用される。この制限を超える着信接続はすべて即座に閉じられる。この制限は、異なる操作のために同じリモートノードと複数の短期接続が開始された場合に達することがある。これはノード起動直後に接続タスクがより積極的にスケジュールされる場合に起こりやすい。これらのエラーは通常一時的なものであり、持続しない場合は無視しても問題ない。

ライター接続はよりアクティブで、他のノードにデータを送信するために使用される。ブロードキャスト操作はすべてのアクティブかつ利用可能なライターにデータをプッシュする。さらに、ライターは個別に選択してリクエスト/レスポンスプロトコルに使用できる。受信者の処理を簡略化するために、進行中のリクエスト/レスポンスプロトコルに参加しているライターにはブロードキャストパケットは送信されない。

サービス識別子は長期接続にのみ割り当てられる。Sync サービスは Peer ロールを持つノードへの発信接続を管理するために使用される。API パーシャルサービスは API ロールを持つノードへの発信接続を管理するために使用される。リーダーサービスは着信接続を管理するために使用される。API ライターサービスは実験的であり、ポート `node:apiPort` への着信接続がライターとして登録できるようにする。

識別子	名前	方向
0x50415254	pt.writers	発信
0x52454144	readers	着信
0x53594E43	sync	発信

Figure 26: サービス識別子

13.2: 重み付きノード選択 で説明したノード選択において、ノードのエージングと選択はどちらもサービスごとにスコープされる。レピュテーション情報はすべてのサービスにわたって集約される。具体的には、あるノードが別のノードへの Sync と API パーシャルの両方の接続を行ったと仮定する。エージはサービスごとにスコープされるため、各接続は異なるエージを持つことができる。インタラクション結果はいかなる接続からのものも、常にサービスではなくノードに帰属される。

12.5 ピアプロベナンス

ノードはそのネットワーク内のすべてのノードに関するデータを収集する。データの信頼性はその出典に依存する。出典は良い順に以下の通りである：

-
1. ローカルノードは `node:localnode` で指定されている。
 2. 静的ノードはピア設定ファイルの 1 つに含まれている。
 3. 動的ノードは探索され、接続をサポートしている。
 4. 動的着信ノードは接続を行ったが、接続をサポートしていない。

静的ノードの識別特性は、少なくとも 1 つのローカルピア設定ファイルにリストされている点であることに注意が重要である。強調するために、あるパートナーがノードを静的とみなす一方で、別のパートナーが動的とみなすことは可能である。ローカルノードを除き、他のすべてのノードは動的である。動的ノードのサブセットは着信である。これらのノードは着信接続でのみ確認されており、発信接続ではない。そのため、優先ポートは不明であり、接続できない。

既存のノードデータは、新しいデータの出典が既存のデータより悪くない場合にのみ更新できる。例えば、動的な出典を持つ静的ノードに関する更新情報は破棄されるが、動的または静的な出典を持つ動的ノードに関する更新情報は許可される。

上記は接続が実際に管理される方法に起因するわずかな簡略化である。ノードがリモートノードから完全に切断して再接続する場合、更新は 2 段階のプロセスになることがある。異なる ID の公開鍵を持つリモートへの再接続を試みる動的ノードを考える。ノードが接続を開始すると、リモートはその接続を動的着信として分類し、これは動的よりも悪い出典となる。結果として、リモートはノードの情報を更新しない。代わりに、進行中の可能性がある ID 更新を示すフラグを設定する。後で、リモートがノードに直接接続すると、以前と同じ更新された情報を取得する。この時点で、進行中の ID 更新が以前に検出されていたため、アクティブな（動的着信）接続が存在していてもリモートは情報を更新する。このフラグがないと、より悪い出典を持つアクティブな接続が更新をブロックし、望ましくない結果となる。

`network:nodeEqualityStrategy` が `public-key` の場合、二次的な ID コンポーネントは解決済み IP である。アクティブな接続がない場合、これは変更可能である。このストラテジーはレピュテーションの移行をサポートしない。

`network:nodeEqualityStrategy` が `host` の場合、二次的な ID コンポーネントはノードの ID 公開鍵である。アクティブな接続がない場合、これは変更可能である。二次的な ID コンポーネントが変更されないと仮定した場合、アクティブな接続がない場合は解決済み IP の主要な ID コンポーネントも変更できる。この場合、元のホストに関連付けられたすべてのレピュテーションデータは新しいホストに移行される。あいまいな一致がある場合、主要な ID コンポーネントが一致するデータが移行され、二次的な ID コンポーネントが一致するデータは破棄される。

12.6 ノード探索

起動後、ノードはピア設定ファイルから読み込んだすべての静的ノードへの短期接続を試みる。これらの接続は主に、すべての静的ノードの解決済み IP アドレスを取得することを目的とする。これによりピア設定ファイルでホスト名を使用でき、ノード管理が簡略化される。ノードが実行されている限り、この手順は線形バックオフで定期的に繰り返される。

定期的に、ノードはリモートパートナーノードに自身の識別情報をブロードキャストする。リモートは受信したペイロードを処理し、有効性と互換性を確認する。有効であるためには、ノードが指定した ID 公開鍵がそのルート X509 証明書の公開鍵と一致しなければならない。互換性があるためには、ブロードキャストするノードと受信するノードの両方が同じネットワークを対象としている必要がある。ホスト名が提供されない場合、ノードの解決済み IP が代わりに使用される。すべての確認が成功すると、ノードは新しい潜在的なパートナーとして追加され、次の同期ラウンドで選択の対象となる。

定期的に、ノードはリモートパートナーノードから既知のすべてのピアをリクエストする。リモートノードはアクティブな静的および動的ピアすべてで応答する。リクエストするノードにとって、これらはすべて動的ノードとして扱われる。元のノードはこれらの各ノードから識別情報を直接リクエストする。この直接通信は、悪意のある行為者が他のノードに関する偽の情報を中継することを防ぎ、各新しいノードとの接続を確立できることを確認するために必要である。元のノードは受信したペイロードを処理し、上記のように有効性と互換性を確認する。すべての確認が成功すると、新しいノードは新しい潜在的なパートナーとして追加され、次の同期ラウンドで選択の対象となる。

13 レピュテーション

((

良い評判を築くには多くの善行が必要だが、失うのは一つの悪行で十分だ。))

- Benjamin Franklin

SYMBOL はピアツーピア (P2P) ネットワークを使用する。P2P ネットワークは単一のノードを排除しても停止できないため、堅牢であるという大きな利点がある。しかし、公開ネットワークには固有の課題もある。ネットワーク参加者は匿名であり、誰でも参加できる。これにより、無効な情報を拡散したり何らかの方法でネットワークを妨害しようとする敵対的なノードをネットワークに簡単に注入できる。

敵対的なノードを特定し、それらとの通信を減らす必要がある。これを達成するためにさまざまなアプローチが試みられてきた。最も成功したアプローチの一つは、ノードのレピュテーションシステムを構築することである。**Symbol** はシンプルなレピュテーションシステムを実装することでこのアプローチに従っている。このシステムは、不正なノードよりも正常に動作するノードへの接続を優先しようとする。重要な点として、レピュテーションはブロックチェーンのコンセンサスには一切影響しない。ネットワークグラフにのみ影響する。この章では使用されるヒューリスティックの概要を説明する。

13.1 接続管理

各ノードは一度に最大 `node:maxConnections` 個の永続接続を確立できる。この制限はネットワーク全体を構成する数十万のノード数よりもはるかに少ないことが想定される。孤立したノードグループの形成を避けるために、ノードは定期的に既存の接続を切断し、異なるノードへの新しい接続のための空きを作る。

切断するノードを決定する際、ノードはすべての接続の経過時間を検査する。接続のオーバーヘッドを最小化するために、少なくとも `node:maxConnectionAge` ラウンドの間確立されてきた接続のみが削除対象となる。次のノード選択ラウンドが行われる際、これらの接続は切断され、他のノードへの新しい接続に置き換えられる。これにより、時間をかけて各ノードはネットワーク内のさまざまなノードと接続するようになる。

13.2 重み付きノード選択

ノードは主に確立した現在の永続接続を通じてお互いに通信する。ノードは別のノードに新しいトランザクションやブロックを照会したり、パートナーノードが通信したことのある他のノードのリストを要求したりできる。ノードはまた、自発的に他のノードにデータを送信することもできる。ノード間の各通信はインタラクションとみなされ、各インタラクションは成功、中立、失敗のいずれかとしてスコアリングされる。例えば、リモートノードが新しい有効なデータを送信した場合、二つのノードの同期に貢献したため、インタラクションは成功とみなされる。リモートノードに新しいデータがない場合は中立である。それ以外の場合はインタラクションは失敗とみなされる。

各ノードは他のノードとのインタラクション結果を記録している。これらの結果はローカルでのみ使用され、他のノードとは共有されない。他のノードとのインタラクション結果は、パートナーノードの選択に影響する。インタラクション結果は最大 1 週間保存されるが、ノードの再起動でリセットされる。これらの結果は時間制限があり、一時的な障害を経験しているノードが良いパートナーとして再確立できるようにする。

パートナーノードを選択する際、ノードはまず候補ノードのセットを決定する。各候補ノードには以下の基準に従って 500 から 10000 の間の生の重みが割り当てられる：

- ・ リモートノードとの非中立のインタラクションが 3 回以下の場合、中程度の生の重み 5000 が与えられる。これにより新しいノードが選択される良い機会が与えられる。
- ・ それ以外の場合、 s を成功したインタラクション数、 f を失敗したインタラクション数とする。生の重みは以下の数式で計算される：

$$rawWeight = \max \left(500, \frac{s \cdot 10000}{s + 9 \cdot f} \right)$$

この数式は、失敗したインタラクションがリモートノードの重みとその選択される確率を急速に低下させることを保証する。最小スコアの存在により、多くの失敗を持つノードも選ばれ、より多くのインタラクションでスコアを改善するわずかなチャンスを持てる。

生の重みに重み乗数をかけると、ノードの最終的な重みを得られる。静的ノードの場合、乗数は 2 である。動的ノードの場合は 1 である。連続したインタラクション失敗によりノードがバンされている場合（[13.3: ノードバン](#)を参照）、乗数は 1 減少す

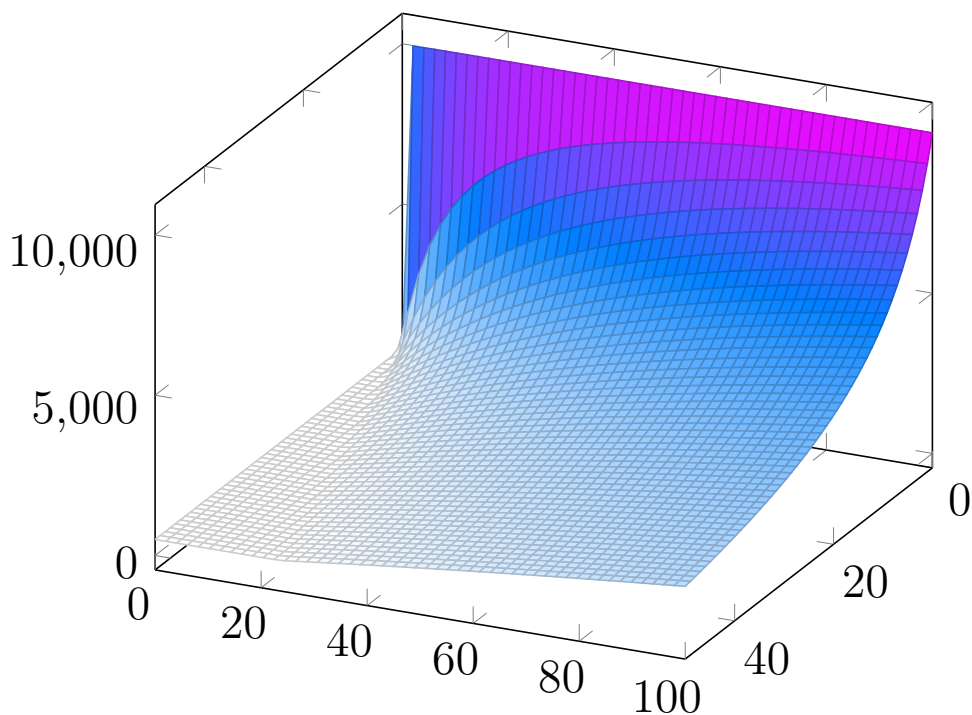


Figure 27: ノードの生の重み

る。これにより、ノードはバンされた動的ノードに接続しないことが保証される。静的なバン済みノードへの接続確率は半分に減少する。

削除候補は接続の経過時間に基づいて決定される。閉じられる各削除候補は、ノードが目標の接続数を維持できるように新しいノードへの接続に置き換えられる。最後に、各空きスロットに対して、候補ノードが選択される確率は以下の通りである：

$$P(\text{ノードが選択される}) = \frac{\text{ノード重み}}{\sum \text{候補ノード重み}}$$

13.3 ノードバン

公開ネットワークには、ネットワークの通常処理を妨げようとする潜在的な悪意のあるノードが存在する可能性がある。そのため、ノードがリモートノードを悪意があると判断した場合、そのノードへの接続を防ぎ、そこからの着信接続も受け入れない。

バンはノードレベルで適用され、ノードのネットワークスコープの識別子（[12.6: ノ](#)

ード探索を参照)に関連付けられる。不正なノードは即座に `node:defaultBanDuration` の期間バンされる。ノードがアクティブなバン状態でなくなっても、ローカルノードはしばらくの間 (`node:keepAliveDuration`) そのノードが不正な振る舞いをしていたことを記憶し、`node:maxBanDuration` を上限として長期間バンすることで繰り返しの違反をより厳しく扱う。バン中は、バンされたノードとの接続は確立されない。バンの期限が切れると、ノードは通常のインタラクションパートナーとして扱われる。リモートノードがバンされるシナリオはさまざまある。ペナルティは原因によって異なる。

	接続 切断	リモートの 再接続 可否	リモートの 選択 可否	リモートの データ送信 可否
連続した インタラクション 失敗	なし	-	静的: 可 動的: 不可	可
無効な データ	あり - すべて ³⁸	不可	不可	不可
読み込み レート超過	あり - すべて	不可	不可	不可
予期しない データ受信	あり	可	可 (再接続後)	可 (再接続後)

Figure 28: バンルール

連続したインタラクション失敗

ネットワーク障害またはステートフル障害により同じノードとのインタラクションが連続して何度も失敗する場合、そのノードとのすべてのインタラクションをしばらく停止し、将来的にノードがより良い振る舞いをするよう期待する方が良い。ノードがインタラクションパートナーとしてバンされるまでの連続インタラクション失敗回数は設定可能である。ノードがバンされる時間はセレクションラウンド数で測定され、これも設定可能である。バン中、ノードはインタラクションパートナーとして積極的には選択されないが、新しいデータを送信することはできる。したがって、この違反は部分的なバンのみをもたらす。

³⁸不正なノードに関連するすべてのアクティブ接続は即座に切断される。違反を引き起こした接続だけでなく、すべてである。

無効なデータ

データはさまざまな方法で無効になり得る。例えば、リモートノードがフォーク上にある場合、ローカルノードのチェーンに適合しない新しいブロックを送信する可能性がある。深さ 1~2 ブロックの小さなフォークは頻繁に発生する。送信されたデータは無効だが、リモートノードの内部状態が異なっていたことは理解できるため、悪意があるとはみなされない。一方、無効な署名を持つデータの送信は、署名検証がノードの状態に依存しないため、リモートノードが悪意を持っていることを明確に示す。ノードの状態に依存しない他の検証失敗も同様である。これらすべての場合において、リモートノードはバンされる。

読み込みデータレートの超過

各ノードはピアからデータを受け入れるすべてのソケットの読み込みレートを監視する。これにより、ノードは障害のあるピアが予想しない量のデータを生成していることを検出できる。設定された時間間隔内に読み込まれたデータが最大値を超えた場合、ソケットは閉じられ、ノードはバンされる。最大読み込みレートは設定可能である。

予想しないデータ受信

ノード通信中に、ローカルノードがリモートからデータを受信することを期待していない状況がある。リモートがそのような状況でもデータを送信した場合、プロトコルに違反しており接続は切断される。この場合、接続は即座に閉じられるが、ノードの永続的なバンは発生しない。

14 コンセンサス

((

ねえ、時々思うんだけど、私たちは自分たちで作り上げた世界に生きているように見える。何が良くて何がそうでないかを決め、自分たちのために意味の地図を描く…そして人生の全てを自分たちが発明したものと格闘して過ごす。問題は、私たちそれぞれが独自のバージョンを持っているため、人々がお互いを理解するのが難しいことだ。

))

- Olga Tokarczuk

BYZANTINEコンセンサスはすべての分散システムが直面する重要な問題である。本質的に、問題の核心は独立した参加者を不正なく協力させる方法を見つけることである。ビットコインの主要なイノベーションは、Proof of Work (PoW) に基づくこの問題の解決策であった。ビットコインのメインチェーンに新しいブロックが受け入れられるたびに、すべてのマイナーは次のブロックを見つける競争を開始する。最も累積ハッシュパワーが高いチェーンが参照チェーンであるため、すべてのマイナーはフォークではなくメインチェーンを延長するインセンティブがある。マイナーは現在のネットワーク難易度目標を下回るハッシュを持つ候補ブロックを1つが生成するまで、できる限り速くハッシュを計算する。マイナーがブロックをマイニングする確率は、ネットワークの総ハッシュレートに対するそのマイナーのハッシュレートに比例する。これは必然的に計算軍拡競争につながり、多くの電力を消費する。

Proof of Stake (PoS)[KN12][BCN13] ブロックチェーンはビットコインの後に登場した。これらは大きな電力消費を必要としないビザンチンコンセンサス問題への代替ソリューションを提供した。基本的に、これらのチェーンは1つの重要な違いを除いてビットコインと同様に動作した。ブロックを作成する確率をノードの相対ハッシュレートに基づかせる代わりに、その確率はネットワークにおけるノードの相対ステークに基づく。より裕福なアカウントはより貧しいアカウントよりも多くのブロックを生成できるため、このスキームは富める者がより富む傾向がある。

Symbol は保有者よりもユーザーを優先して報酬する PoS の修正版を使用する。パフォーマンスとスケーラビリティを犠牲にせずにアカウントのインポートランスの総合的なスコアを計算しようとしている。

健全なエコシステムに貢献する複数の要因がある。他の条件が等しければ、より大きなステークを持ちより多くのトランザクションを行いノードを運営しているアカウントはゲームにより多くのスキンを持ち、それに応じて報酬されるべきである。第一に、より大きな残高を持つアカウントはネットワークにより大きなステークを持ち、エコシステム全体の成功を見届けるより大きなインセンティブを持つ。アカウントが所有する通貨量はそのステークの尺度である。第二に、アカウントはトランザクションを行うことでネットワークを使用するよう奨励されるべきである。ネットワー

ク使用量はアカウントが支払ったトランザクション手数料の総額で近似できる。第三に、アカウントはネットワークを強化するためにノードを運営するよう奨励されるべきである。これはアカウントがブロックの受益者に指定された回数で近似できる³⁹。ノード所有者は受益者の定義を完全に制御しているため、どの善意のノード所有者もサードパーティのためにこの尺度を高めることができる。

インポートانسは *network:importanceGrouping* ブロックごとに再計算される。インポートانسの計算は比較的高価であり、毎ブロック処理することは禁止的になるため、これはブロックチェーンへの圧力を軽減する。さらに、インポートانسを定期的に再計算することで、自動的な状態エイジングが可能になる。全体的に、インポートانسを毎ブロックではなく定期的に計算することが有益である。

良い行動を奨励するために、古い時期にアクティブだったアカウントは以前の徳ある行動により永続的な優位性を得るべきではない。代わりに、トランザクションとノードスコアによって付与されるインポートانسブーストは時間制限付きである。ブーストは 5 つの *network:importanceGrouping* インターバルにわたって続く。

14.1 重み付けアルゴリズム

少なくとも *network:minHarvesterBalance* の残高を持つすべてのアカウントがインポートانس計算に参加し、高価値アカウントと呼ばれる。このアカウントのセットはブロック生成の資格があるアカウントのセットのスーパーセットであることに注意されたい（8.3: [ブロック生成](#) を参照）。言い換えると、最近のインポートانس再計算でのゼロ以外のインポートانسはブロック生成の必要条件であるが、十分条件ではない。

アカウントのインポートانسスコアは、ステーク、トランザクション、ノードの 3 つのコンポーネントスコアを組み合わせる計算される。

アカウント A のステークスコア S_A は、すべての高価値アカウントが所有する総通貨量に対してそれが所有する通貨のパーセンテージである。このパーセンテージは、すべての流通通貨に対してアカウントが所有する通貨のパーセンテージ以上である。 B_A をアカウント A が所有する通貨量とする。アカウント A のステークスコアは各適格アカウントについて以下のように計算される：

$$S_A = \frac{B_A}{\sum_{a \in \text{高価値アカウント}} B_a} \quad (13)$$

³⁹ この尺度はすべてのアカウントがアクティブにノードを運営している場合、ステークと強く相関する。その意図はノードを運営しているアカウントとアイドル状態のアカウントを区別することである。

アカウント A のトランザクションスコア T_A は、時間帯 P 内に高価値アカウントが支払ったすべての手数料に対してそれが支払ったトランザクション手数料のパーセンテージである。FeesPaid A を時間帯 P において A が支払った手数料量とする。アカウント A のトランザクションスコアは各適格アカウントについて以下のように計算される：

$$T_A = \frac{\text{FeesPaid}(A)}{\sum_{a \in \text{高価値アカウント}} \text{FeesPaid}(a)} \quad (14)$$

アカウント A のノードスコア N_A は、時間帯 P 内に高価値アカウントの受益者の総数に対して受益者として指定された回数のパーセンテージである。BeneficiaryCount A を時間帯 P において A が受益者として指定された回数とする。アカウント A のノードスコアは各適格アカウントについて以下のように計算される：

$$N_A = \frac{\text{BeneficiaryCount}(A)}{\sum_{a \in \text{高価値アカウント}} \text{BeneficiaryCount}(a)} \quad (15)$$

トランザクションスコアとノードスコアは共にアクティビティスコアと呼ばれる。これは両方が動的であり、ステークとは対照的にアカウントのアクティビティから導出されるからである。トランザクションスコアは 80% の重み付けがなされ、ノードスコアは 20% である。さらに、ステークが増加するにつれてインポータンスに対するアクティビティの減衰効果があるように、組み合わせスコアはアカウントの残高に対して相対的にスケーリングされる⁴⁰。これにより実質的に、アクティブな小規模アカウントがアクティブな大規模アカウントに比べて不釣り合いに大きなブーストを得られる。これはインポータンスを一部、富裕アカウントから貧困アカウントへ再分配し、PoS に固有の富める者がより富む現象にある程度対抗する。ステークに対するアクティビティの重要性は *network:importanceActivityPercentage* で設定できる。この値がゼロの場合、**Symbol** は純粋な PoS ブロックチェーンのように動作する。この値を高く設定しすぎると、攻撃者が過半数のインポータンスを取得して 51

パフォーマンス最適化として、アクティビティ情報は最近のインポータンス計算時点で高価値なアカウントのみ収集される。インポータンス再計算間では、新しいデータはワーキングバケットに格納される。各インポータンス再計算時に、既存のバケットがシフトされ、ワーキングバケットが確定されて新しいワーキングバケットが作成される。各バケットは最大 5 回のインポータンス再計算に影響する。その結果、アクティビティ情報は素早く期限切れになる。

network:totalChainImportance 設定はネットワーク内のすべてのアカウントに分配

⁴⁰アクティビティスコアは減衰後に再スケーリングされ、インポータンス計算に望ましい *network:importanceActivityPercentage* を貢献する。

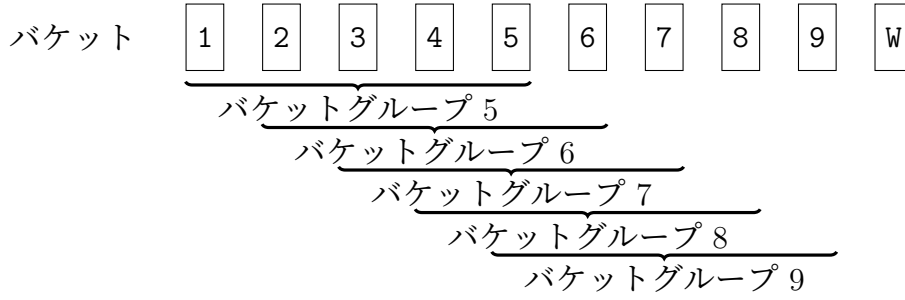


Figure 29: アクティビティバケット

される総インポートانسを指定する。それを踏まえて、アカウント A のスポットインポートانس I'_A は以下のように計算できる⁴¹：

$$\begin{aligned} \gamma &= \text{importanceActivityPercentage} \\ \text{ActivityScore}'_A &= \frac{\text{minHarvesterBalance}}{B_A} \cdot (0.8 \cdot T_A + 0.2 \cdot N_A) \\ \text{ActivityScore}_A &= \frac{\text{ActivityScore}'_A}{\sum_{a \in \text{高価値アカウント}} \text{ActivityScore}'_a} \\ I'_A &= \text{totalChainImportance} \cdot ((1 - \gamma) \cdot S_A + \gamma \cdot \text{ActivityScore}_A) \end{aligned}$$

アカウント A の最終インポートانسスコア I_A は、現在と前回のインポートانس計算での I'_A の最小値として計算される。これはステークグラインディング攻撃に対する予防措置であり、不必要なステーク移動を最小限に抑える一般的なインセンティブとして機能する。再スケーリングはないため、すべての高価値アカウントの I_A の合計は `network:totalChainImportance` より少ない可能性がある。

14.2 シビル攻撃

ピアツーピアネットワークへのシビル攻撃⁴²は、攻撃者がネットワークまたは他の優位性に対して不均衡に大きな影響力を得るために複数の ID を作成する場合に発生する。**Symbol**では、攻撃者がインポートانسを高めるためにこのような攻撃を試みる可能性がある。インポートانسスコアの各コンポーネントはこのような攻撃に対して堅牢である必要がある。

⁴¹ゼロコンポーネントスコアの処理に関する方程式には反映されていない追加のエッジケース処理がある。トランザクションまたはノードスコアのいずれかがゼロの場合、もう一方がスケールアップされ、完全に重み付けされたアクティビティスコアとして機能する。両方がゼロの場合、ステークスコアがスケールアップされ独占的に使用される。

⁴² 15: [ファイナライゼーション](#) に追加の議論がある。

14.1: 重み付けアルゴリズム で説明されているように、アカウントのアクティビティスコアはその残高に対して減衰される。したがって、アカウントの残高を複数のアカウントに分割すると、適用される平均減衰係数が下がる。再分配の前後で一定レベルのアクティビティが維持されると仮定すると、累積インポータンスは分割後に高くなる⁴³。この効果は設計によるものであり、インポータンスブーストはアクティビティが維持される場合にのみ実現されるため、徳ある行動を奨励する。トランザクションスコアの維持は複数のアカウントからのトランザクションと手数料の支払いを奨励する。ノードスコアの維持は追加のノードの運営とネットワークへの接続を奨励する。

$\mu := \text{minHarvesterBalance}$ と総通貨 $N \cdot \mu$ を所有する攻撃者を仮定する。2つの極端なケースを考える：

1. 攻撃者は $N \cdot \mu$ の通貨を持つ単一のアカウントを持つ。
2. 攻撃者は μ の通貨を持つ N 個のアカウントを持つ。

ステークスコアのブースト

両方の極端なケースにおいて、攻撃者が所有する総通貨は同じである。したがって、ステークスコアは同じであり、アカウントを分割することによる利益はない。強調すると：

$$B_A = \sum_{a \in \{1, \dots, N\}} B_a \quad (16)$$

ノードスコアのブースト

Symbol はノード所有者がノードでハーベストされた各ブロックの受益者を指定することを許可する。アカウントが受益者として指定されるたびに、それがすでに高価値アカウントであれば、ノードスコアがわずかに上昇する。

両方の極端なケースにおいて、攻撃者の総受益者数は同じである。したがって、ノードスコアは同じであり、アカウントを分割することによる不当な利益はない。強調すると：

$$\text{BeneficiaryCount}(A) = \sum_{a \in \{1, \dots, N\}} \text{BeneficiaryCount}(a) \quad (17)$$

⁴³これは1つのアカウントのみが分割すると仮定している。アクティビティスコアは相対的であるため、複数のアカウントが分割すると効果が薄れる。

より多くのノードを運営することで攻撃者のノードがより多くの委任ハッシュをホストできる場合、攻撃者はより高いノードスコアを得る可能性がある。これは悪い結果ではなく、設計による。これはネットワーク内のより多くのノードを奨励し、ネットワークを強化する良いことである。

攻撃者は 1 台の物理マシンを指す N 個の仮想ノードを設定することで不正を試みる可能性がある。これらの仮想ノードのそれぞれはネットワークの残りの部分から通常のノードとして扱われ、基礎となる物理ノードはネットワーク内の通常のノードより N 倍多く相互作用される。これは仮想ノードが強力な物理サーバーで動作していることを意味し、それでも弱い物理サーバーよりネットワークに有益である。

トランザクションスコアのブースト

トランザクションスコアは手数料のみに基づいている。手数料に X を費やす 1 つの巨大なアカウントと、それぞれ手数料に $\frac{X}{N}$ を費やす N 個の小さなアカウントとの間に違いはない。強調すると：

$$\text{FeesPaid}(A) = \sum_{a \in \{1, \dots, N\}} \text{FeesPaid}(a) \quad (18)$$

トランザクションスコアを高める唯一の可能性は手数料攻撃であり、これは [14.4: 手数料攻撃](#) で詳しく議論される。

14.3 ステークなし攻撃

PoS コンセンサスへの一般的な批判はステークなし攻撃⁴⁴である。この攻撃はブロックを作成する機会コストが無視できる場合に理論的に存在する。この攻撃には 2 つのバリエーションがある。

最初のバリエーションでは、攻撃者以外のすべてのハッシュターがすべてのフォークでハッシュトする。説明を簡略化してバイナリフォークを仮定すると、攻撃者は一方のブランチに支払いを送信し、すぐに他のブランチでハッシュティングを開始する。攻撃者がブロックをハッシュトするのに十分なインポートランスを持つと仮定すると、攻撃者の支払いを含まないブランチは最終的により高いスコアを持つため参照チェーンになる⁴⁵。攻撃者の支払いはこのブランチに含まれていないため、攻撃者の資金は実質的に返還される。

⁴⁴ [15: ファイナライゼーション](#) に追加の議論がある。

⁴⁵ これは攻撃者が 1 人だけであるか、すべての攻撃者が同じブランチからハッシュティングを控えるために共謀すると仮定している。

この攻撃に対する主な 3 つの防御がある。まず、攻撃者がより良いチェーンを生成できる時間は限られている。これは最大 `network:maxRollbackBlocks` ブロックしかロールバックできないからである。マーチャントがこのブロック数以上確認されるまでサービスを提供するのを待てば、攻撃は不可能である。次に、ステークなし攻撃を成功させるためには、攻撃者はネットワークにおいて重大なインポータンスを所有する必要がある⁴⁶。第三に、ネットワークに対するこの攻撃の成功した実行は通貨価値に悪影響を及ぼす可能性が高い。他のハーベスターはすべてのフォークでハーベスティングすることでこの攻撃を可能にするため、利益最大化ハーベスターはこれを防ぐために単一のチェーンのみでハーベストすべきである。

2 番目のバリエーションでは、単一の攻撃者がすべてのフォークでハーベストし、どのチェーンが参照チェーンになるかに関係なく、すべての手数料を獲得しようとする。攻撃者は 2 番目のブロックから始まるすべてのフォークでハーベストし、攻撃者が最も多くの手数料をハーベストしたチェーンを探すことができる。ブロック受け入れは確率論的であるため、理論的には攻撃者が攻撃者がすべてのブロックをハーベストした完璧なチェーンを構築するのに無限の時間を費やすことができる。

ほとんどの理論的なステークなし攻撃は理想化されたブロックチェーンを想定し、このような攻撃から保護するプロトコルレベルのセーフガードを無視している。実際には、攻撃者が少数の通貨を所有している場合、このタイプの攻撃は実現不可能である。前述の 2 つの防御もここに適用できる。さらに、ブロック難易度の変化（[8.1: ブロック難易度](#)を参照）は 5% に制限されている。攻撃者のチェーンの難易度が下方調整されるには時間がかかり、秘密チェーンの最初のブロック時間がメインチェーンのものよりも大幅に遅れる。これらの大きな時間差により、攻撃者がより良いスコアを持つチェーンを生成することが困難になる（[8.2: ブロックスコア](#)を参照）。

少量のステークエイジングもこの 2 番目のバリエーションの可能性を減少させる。ハーベスティングの前提条件として、2 回連続するインポータンス再計算においてゼロ以外のインポータンスを持つことをアカウントに要求すると、生成ハッシュグライディング⁴⁷攻撃が実行不可能になる。これを悪用するためには、攻撃者は攻撃が実行できる前に `network:importanceGrouping` ブロック以上前に特定のアカウントにすべての通貨を移動する必要がある。攻撃者は中間期間に確認されるすべてのブロックを知ることができないため、そのような移動はいかなる利益ももたらさない。

⁴⁶理論的には、攻撃者はこの攻撃を実行するために `network:minHarvesterBalance` だけ必要だろう。実際には、成功した実行を保証するためには、攻撃者はロールバック間隔内で常にブロックをハーベストするのに十分な大きさのインポータンスが必要だろう。

⁴⁷これはブロックヒット（[8.3: ブロック生成](#)を参照）をブルートフォースしようとする試みであり、これは生成ハッシュに依存している。

14.4 手数料攻撃

手数料攻撃は攻撃者がトランザクションスコアを悪用して自身のインポータンスを高めるために大きな手数料を支払おうとする試みである。この攻撃は正の期待値をもたらす場合に有効とみなされる。

このセクションの分析は推奨パブリックネットワーク設定を使用して実行される。これらは *network:totalChainImportance* が 90 億、*network:importanceGrouping* が 359 ブロック、*minHarvesterBalance:*が 10000 通貨を含む。さらに、*network:importanceActivityPercentage* は 5 であり、累積トランザクションスコア (Equation 14.1: 重み付けアルゴリズム を参照) はインポータンスの 4% を占める。

大規模アカウント

アクティビティブーストなしでインポータンス再計算インターバルあたり 1 ブロックをハーベストするのに十分大きなアカウントを考える。90 億のうち 2 だけがアクティビティにハーベスティングしていると仮定すると、アカウントはこれほど頻繁にハーベストするために少なくとも 557 万の通貨を必要とする。

アカウントは各再計算インターバルに自身のハーベストしたブロックの 1 つに高手数料のトランザクションを追加することで利益を得ようとするかもしれない。これによりアカウントのインポータンスが上昇し、将来的により多くのブロックをハーベストでき、その結果より多くの手数料を収集できる。ただし、このアクティビティはリスクフリーではない。より良いブロックがそのブロックを置き換えた場合、アカウントは高手数料を支払うリスクがある。元のブロックが取り消されると、高手数料のトランザクションは未確認トランザクションキャッシュに入り、別のハーベスターによって作成された新しいブロックへの包含の資格を得る。このシナリオはアカウントが高手数料を支払わなければならないため、純損失となる。

P を損失をもたらすフォークの確率、 F をブロック内の高手数料、 $\bar{F}$ をブロック内の平均手数料とする。期待値 EV は以下のように近似できる：

$$\beta = 0.04 \cdot \frac{1}{557} \cdot \frac{F}{359 \cdot \bar{F} + F} \quad (\text{インポータンスブースト})$$
$$EV = \beta \cdot \frac{359}{P} \cdot \bar{F} - F \quad (\text{期待値})$$

期待値は小さい P の値では正である。 P または F が増加すると、すぐに負になる。推奨パブリックネットワーク設定を使用すると、期待値が正になるためには P が 0.0001 未満である必要がある。これは損失をもたらすフォークが 10000 ブロックに 1 回未満発生することを意味する。分散コンセンサスのメカニズムを考えると、こ

れはほぼ不可能である。1 ブロックまたは 2 ブロックの小さなフォークはかなり頻繁に発生する。

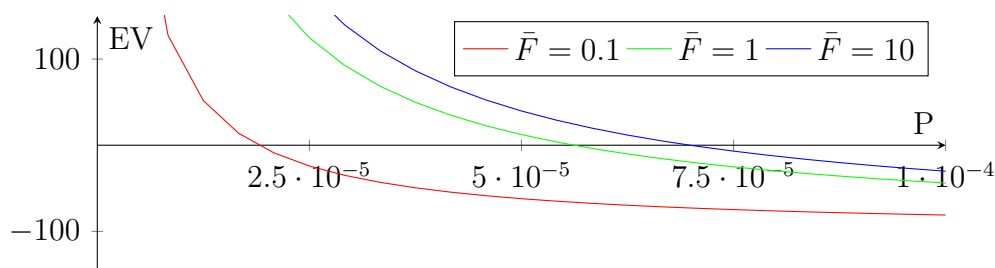


Figure 30: 手数料攻撃大規模アカウント分析 ($F = 100$)

小規模アカウント

`network:minHarvesterBalance` に等しい残高を持つアカウントを考える。アカウントが 2 回連続する再計算インターバルで高手数料のトランザクションを 1 つ行くと仮定する。アカウントがブロックをハーベストする確率はかなり小さいため、これらの手数料は他のハーベスターに失われる。支払われた高手数料はアカウントのインポートانسをインポートانس再計算インターバルあたり少なくとも 1 ブロックをハーベストできるほど上昇させる。この時点から、アカウントは前のセクションの大規模アカウントのように動作する。また、各再計算インターバルに自身のハーベストしたブロックの 1 つに高手数料のトランザクションを追加する。アカウント所有者は、ハーベスティングのブロック確率の増加により、追加収集された手数料がコストを超えることを望む。

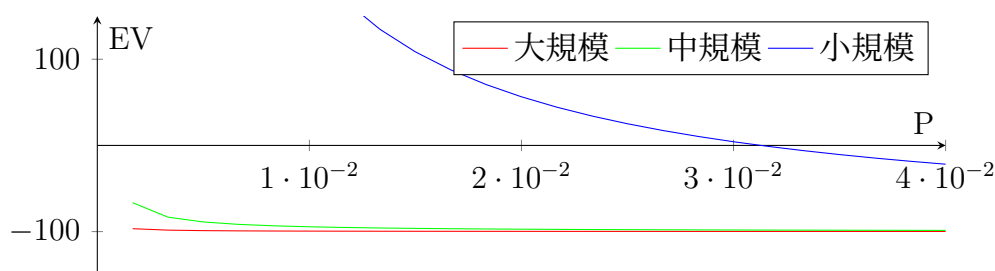


Figure 31: 手数料攻撃残高感度 ($F = 100, \bar{F} = 1$)

P を損失をもたらすフォークの確率、 F をブロック内の高手数料、 $\bar{F}$ をブロック内の平均手数料とする。最初のトランザクション手数料を除く期待値 EV は以下のよ

うに近似できる⁴⁸：

$$\beta = 0.04 \cdot \frac{F}{359 \cdot \bar{F} + F} \quad (\text{インポートンスブースト})$$
$$EV = \beta \cdot \frac{359}{P} \cdot \bar{F} - F \quad (\text{期待値})$$

期待値は大規模アカウントのシナリオよりも大きい P の値で正である。後者のアクティビティスコアは前者よりも積極的に減衰されるため、手数料攻撃は大規模アカウントよりも小規模アカウントに不釣り合いに大きな利益をもたらす。具体的には、大規模アカウントのアクティビティスコアには $\frac{1}{557}$ の減衰係数が適用されるが、小規模アカウントのアクティビティスコアには減衰係数が適用されない。

期待値は $\bar{F}$ が増加するにつれて増加する。 P または F が増加すると、すぐに負になる。推奨パブリックネットワーク設定を使用すると、期待値が正になるためには P が 0.05 未満である必要がある。これは損失をもたらすフォークが 20 ブロックに 1 回未満発生することを意味する。分散コンセンサスのメカニズムを考えると、これは可能である。

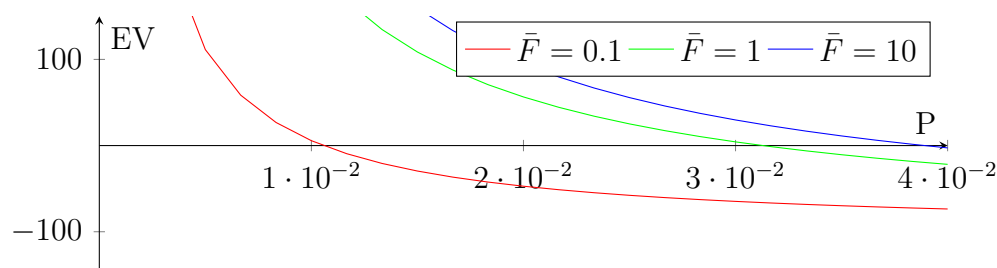


Figure 32: 手数料攻撃小規模アカウント分析 ($F = 100$)

さらなる議論

単一の小規模アカウントはこの攻撃を実行することで正の期待値を得られるが、複数のアカウントが同時に試みると見返りが減少する。正の期待値があるため、利益最大化アクターはすべてこの攻撃を試みるべきである。より多くのアカウントが試みるにつれて、各個別アカウントが得るインポートンスブーストが急速に減少し、その結果期待値も減少する⁴⁹。

⁴⁸大規模アカウントの例との違いは減衰係数が完全に取り除かれることである。

⁴⁹この攻撃を試みるより多くのアカウントが高手数料のトランザクションを生成するにつれて $\bar{F}$ が増加する。多数の攻撃者の場合、 F が比例して引き上げられなければ、各アカウントのインポートンスブーストが減少しても攻撃の期待値は増加する可能性がある。ブロックの価値も大幅に増加するため、これは予想された結果である。

さらに、この攻撃を同時に実行できる小規模アカウントの数には上限がある。この攻撃が成功するためには、アカウントはインポータンス再計算インターバルあたり少なくとも 1 ブロックをハーベストできる必要がある。これは小規模アカウントがトランザクションスコアコンポーネントを悪用することでインポータンススコアを上昇させられることを前提としている。トランザクションスコアに割り当てられたインポータンスと再計算インターバルの両方が有限であるため、十分に大きなブーストを達成できるアカウント数には理論的な限界がある。推奨パブリックネットワーク設定を考えると、この限界は約 $0.04 \div \frac{1}{359} \approx 14.36$ アカウントである。

N を攻撃を試みる小規模アカウントの数、 P を損失をもたらすフォークの確率、 F をブロック内の高手数料、 $\bar{F}$ をブロック内の平均手数料とする。期待値 EV は以下のように近似できる：

$$\beta = 0.04 \cdot \frac{F}{359 \cdot \bar{F} + N \cdot F} \quad (\text{インポータンスブースト})$$

$$EV = \beta \cdot \frac{359}{P} \cdot \left(\bar{F} + \frac{(N-1) \cdot F \cdot P}{359} \right) - F \quad (\text{期待値})$$

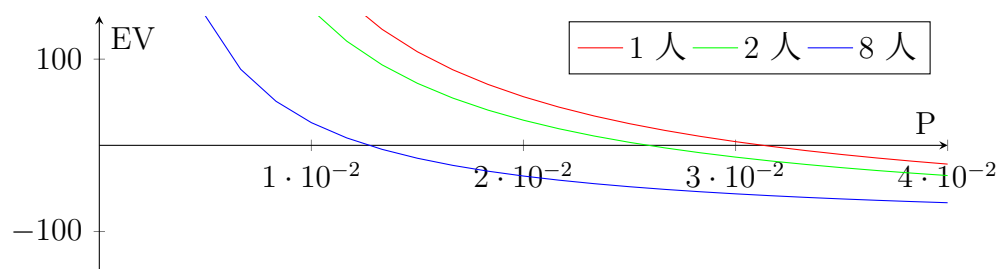


Figure 33: 攻撃者が増えるにつれて低下する手数料攻撃 ($F = 100$, $\bar{F} = 1$)

15 ファイナライゼーション

“

そして、やっという感じがする。

”

- Patrick Ness

CAP の定理は、ネットワーク障害またはパーティションが存在する場合、分散システムは一貫性または可用性のいずれかを選択しなければならないと主張する。

一貫性のあるシステムでは、2つのノードに送信されたリクエストは常に同じ値を返す。値がグローバルに合意されていない場合、リクエストは失敗する。これによりすべてのクライアントがシステムの統一されたビューを持つことが保証される。BFT システムは通常、一貫性を選択し、ネットワーク停止のリスクを冒す。単純な一貫性のあるシステムは、すべてのステークホルダーが各ブロックに投票し、ステークホルダーの $\frac{2}{3}$ が承認した場合にのみ次のブロックを許可することを要求する可能性がある。ステークホルダーが迅速に投票しない場合、ブロック生成が遅延する可能性がある。

可用性のあるシステムでは、2つのノードに送信されたリクエストは常に直ちに値を返す。返された値は異なる場合があるため、異なるクライアントがシステムの異なるビューを持つ可能性がある。PoW と NXT スタイルの PoS システムは通常、可用性と最終的なコンセンサスを選択する。これらはシステムの異なる（解決不可能な可能性のある）ビューを伝播するリスクがある。ビットコインのような可用性のある PoW システムは、最も多くの作業を持つチェーンを選択するシンプルなフォークルールを持つ。ネットワークが等しいハッシュパワーを持つパーティションに分割されている場合、すべてのパーティションはパーティション分割されていることを知らずに独立して進行する。最終的に再接続されると、高コストの（場合によっては深い）フォーク解決が発生する。

Symbol は常に一貫性より可用性を優先するが、ネイティブコンセンサス（[14: コンセンサス](#) を参照）の上にファイナリティガジェットのオプション使用をサポートする。このガジェットはブロック生成とブロックコンセンサスと直交する、BFT 着想の投票システムを導入する。オプション性の結果として、**Symbol** は確定的ファイナライゼーション（ガジェットが有効な場合）または確率的ファイナライゼーション（ガジェットが無効な場合）を使用するネットワークをサポートできる。ガジェットは `node: maxRollbackBlocks` がゼロの場合に自動的に有効化される⁵⁰。

ガジェットアプローチは Polkadot が使用する GRANDPA [SK20] をモデルにしている。GRANDPA はさらに、PBFT [CL99] に影響を受けた CASPER [BG17] に影響を受けている。伝統的に、PBFT は pre-prepare、prepare、commit の 3 種類のメッセージを使用する。ラウンドを開始するために使用される pre-prepare メッセージは

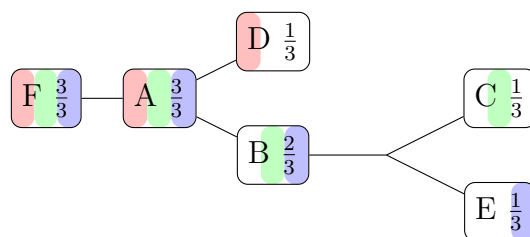
⁵⁰ファイナライゼーションエクステンションも有効化されている必要がある。

Symbol では使用されない。代わりに、ラウンドを開始するために経過ネットワーク時刻（16: 時刻同期 を参照）が使用される。PBFT の prepare と commit メッセージは **Symbol** の prevote と precommit メッセージにほぼ対応する。

15.1 高レベルの概要

ブロックファイナライゼーションは prevote と precommit の 2 種類のメッセージを含む複雑なプロセスである。ラウンドの開始時、各投票者はローカルチェーンに格納されたブロックのみを知っている。どの投票者も他の投票者のチェーンに格納されたブロックを知らない。したがって、投票者は自身のブロックのうちどれが超多数の票を受けて事前にファイナライズされるかを知ることが不可能である。

ファイナライゼーションの一般的な手順を説明するために、3 人の均等に重み付けされた投票者で構成されるネットワークを考える。超多数には 3 人の投票者のうち少なくとも 2 人が同じハッシュに投票することが必要である。 F は最後にファイナライズされたブロックを指す。



以降、投票者は図の色 赤、緑、青で参照される。

ファイナライゼーションは貪欲であり、各ラウンドでできるだけ多くのブロックをファイナライズしようとする。最初の *prevote* ステージでは、各投票者は最後にファイナライズされたブロックのハッシュ (F) から始まるローカルチェーンを表すハッシュチェーンを構築して公開する。この例では、prevote チェーンは次のようになる：

1. 赤: $H(F), H(A), H(D)$
2. 緑: $H(F), H(A), H(B), H(C)$
3. 青: $H(F), H(A), H(B), H(E)$

赤はネットワーク接続が悪いため、緑からの prevote のみを受信し 青からは受信しないと仮定する。緑と 青はすべての投票者から prevote を受信すると仮定する。この時点で、各投票者は以下の prevote チェーンを持つ：

1. 赤:

- (a) $H(F), H(A), H(D)$
- (b) $H(F), H(A), H(B), H(C)$

2. 緑:

- (a) $H(F), H(A), H(D)$
- (b) $H(F), H(A), H(B), H(C)$
- (c) $H(F), H(A), H(B), H(E)$

3. 青:

- (a) $H(F), H(A), H(D)$
- (b) $H(F), H(A), H(B), H(C)$
- (c) $H(F), H(A), H(B), H(E)$

各投票者はすべての prevote ハッシュチェーンを検査し、このラウンドでファイナライズされる**可能性がある**最良のブロックを計算する。赤は A の超多数しか見ないが、緑と 青は B の超多数を見る。次の *precommit* ステージでは、各投票者は計算した最良ブロックのハッシュを公開する：

1. 赤: $H(A)$

2. 緑: $H(B)$

3. 青: $H(B)$

緑はネットワーク接続が悪いため、赤からの *precommit* のみを受信し 青からは受信しないと仮定する。赤と 青はすべての投票者から *precommit* を受信すると仮定する。この時点で、各投票者は以下の *precommit* を持つ：

1. 赤: $H(A), H(B), H(B)$

2. 緑: $H(A), H(B)$

3. 青: $H(A), H(B), H(B)$

各投票者はすべての *precommit* ハッシュを検査し、ファイナライズできる最良のブロックを計算する。重要なことに、ブロックへの *precommit* はそのブロックのすべての祖先への *precommit* でもある。緑は A の超多数しか見ないが、赤と 青は B の超多数を見る。最終的な *commit* ステージでは、各投票者は以下のブロックをファイナライズする：

-
1. 赤: $H(A)$, $H(B)$
 2. 緑: $H(A)$
 3. 青: $H(A)$, $H(B)$

以下のセクションでファイナライゼーションアルゴリズムをより詳しく説明する。

15.2 ラウンド

ファイナライゼーションラウンドはファイナライゼーションプロセスのステップを表し、ファイナライゼーションエポックとファイナライゼーションポイントで構成される。

エポックはブロックのグループである。エポック内のすべてのブロックは単一の投票セットによってファイナライズされ、投票セットは *network:votingSetGrouping* ブロックごとに再計算される。最初のエポックはネメシスブロックのみを含むと定義され、本質的にファイナライズ済みとみなされる。後続のエポックは *network:votingSetGrouping* の倍数である高さのブロックで終わらなければならない。エポック内のすべてのブロックがファイナライズされた場合、エポックはファイナライズ済みとみなされる。エポック内の最後のブロックに対するファイナライゼーション証明は常に利用可能である。

ポイントはエポック内の細かいステップであり、そのエポックのファイナライゼーションへの進捗を表す。各ポイントはゼロ個以上のブロックをファイナライズできる。ポイントは親エポック内のブロックのみをファイナライズできる。これにより、安全違反につながる可能性のある複数の投票セットによるブロックのファイナライズの可能性を防ぐ。エポックに関連付けられたポイント数に制限はない。エポック内のすべてのブロックをファイナライズするために必要なだけポイントがある。エポック内の最後のポイントは常にそのエポックの最後のブロックをファイナライズする。ポイントがそのブロックまたはその子孫のいずれかをファイナライズした場合、ブロックはファイナライズ済みとみなされる。

ネットワークがパーティション分割されている場合、ポイントが追加のブロックをゼロ個ファイナライズする可能性がある。これは最後にファイナライズされたブロックのみが超多数の票を受けた場合に発生する。これは致命的なエラーではなく、ネットワークパーティションの存在下で自然に発生する可能性がある。その後、ファイナライゼーション手順は続行され、[15.5: アルゴリズム](#) で説明された条件に基づいて次のポイントに進む。

エポックとポイントの重要な違いは投票セットとの関係である。異なるエポックは異なる投票セットを持つ場合がある。同じエポック内の異なるポイントはすべてエポックに関連付けられた同じ投票セットを使用しなければならない。

	P 1	P 2	P 3	...	P J	P 1	...	P K	P 1	...	P L		
...	エポック X					エポック X + 1					エポック X + 2		...

Figure 34: エポックと (P) ポイントの関係

15.3 投票者

以下のすべてが真である場合、アカウントはエポックでの投票資格がある：

1. 前のエポックの最後のファイナライズ済みブロックでのハーベスティング残高がネットワーク定義の *network:minVoterBalance* 以上である。
2. 投票キーが $\text{StartEpoch} \leq \text{エポック} \leq \text{EndEpoch}$ となるように登録されている。

エポックの投票セットは前の条件を満たすすべてのアカウントのセットである。重要なことに、投票の重み付けにはインポータンスではなく残高のみが使用される。これはインポータンスを使用した場合に発生する可能性のある安全違反を防ぐ⁵¹。

投票資格があるすべてのアカウントが投票することが期待される。行儀の良い投票者は資格があるすべてのラウンドで投票し、ラウンドあたり複数の票を送信しないことが期待される。すべての投票者が高可用性ノードを運営し *finalization:EnableVoting* をオンにすることが暗黙的に仮定されている。これらの期待に違反する投票者はビザンチンとみなされ、将来的に罰せられる可能性がある。投票はより高い残高を持つ投票者の票がより大きな影響を持つように残高に比例して重み付けされる。

15.4 メッセージ

prevote と precommit メッセージは共通のレイアウトを共有する。

⁵¹複数のネットワークパーティションがある場合、各パーティションは独立してインポータンスを再計算し、潜在的に異なるアクティビティスコアになる。インポータンスはスケールされており絶対的ではないため、理論的には複数のパーティションがインポータンスの超多数を持ち、相反するブロックをファイナライズすることが可能である。

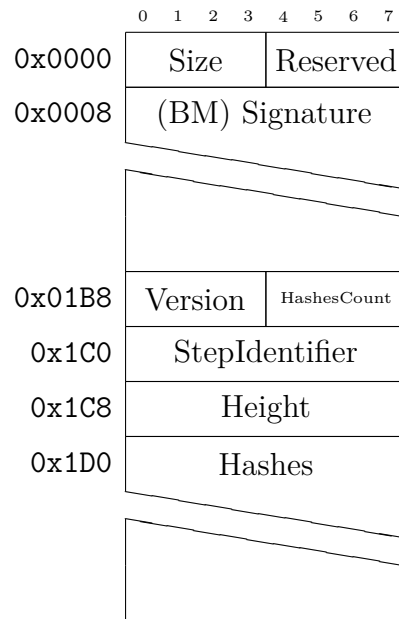


Figure 35: メッセージレイアウト

Signature はメッセージの BM 木署名（[3.4: 投票鍵リスト](#) を参照）である。ルート公開鍵はメッセージのエポックに登録された投票キーと一致しなければならない。ボトム公開鍵はエポックに対応する木内の公開鍵と一致しなければならない。

Version はメッセージバージョンである。

Hashes には HashesCount 個のハッシュが含まれる。prevote メッセージは少なくとも 1 つ、最大 *finalization:maxHashesPerPoint* 個のハッシュを含む。precommit メッセージは常にちょうど 1 つのハッシュを含む。

StepIdentifier はメッセージのファイナライゼーションラウンドを示す。ポイントの高ビットは、0 が prevote を 1 が precommit を示すメッセージタイプを示すために予約されている。

Height は Hashes に含まれる最初のハッシュのブロック高さである。

15.5 アルゴリズム

利用可能な投票ウェイトの少なくとも超多数の累積重み付き投票を持つ最後のブロックを選択する関数 $g(\dots)$ を定義する。 $V_{r,v}$ をラウンド r での投票者 v による prevote として定義する。 $C_{r,v}$ をラウンド r での投票者 v による precommit として定義する。 $E_{r,v}$ をラウンド r でファイナライズされた可能性があるものの投票者 v による推定

値として定義する。これはあくまで推定値であり貪欲なものである。 $E_{r,v}$ は $C_{r,v}$ の超多数を受けられる $g(V_{r,v})$ を含むチェーン上の最新のブロックでなければならない。ラウンドが完了可能であるのは次のどちらかの場合である：

1. $E_{r,v} < g(V_{r,v})$
2. $g(V_{r,v})$ の任意の子が $C_{r,v}$ の超多数を持つことが不可能である

投票者 v は前のラウンドが完了可能で、かつ資格があったすべての前のラウンドで投票した場合にラウンド r を開始できる。

15.5.1 Prevote

$1 * \text{stepDuration}$ が経過したか r が完了可能になったとき、 v は prevote を送信する。

投票者は潜在的にファイナライズできる最良のブロックを決定する。最後の（ローカル）ファイナライズ済みブロックのハッシュから始まるすべてのハッシュで構成される prevote メッセージを作成する。prevote メッセージハッシュチェーンには最大 *finalization:maxHashesPerPoint* 個のハッシュが含まれる。チェーン内の最後のハッシュは *finalization:prevoteBlocksMultiple* の倍数である高さのブロックに対応する⁵²。これにより、ノードがより積極的に集約できる同一チェーンを持つ prevote メッセージを送信する確率が高まる。

ファイナライズされていないブロックに対応するハッシュはエポックをまたぐことが禁止されている。これにより、任意の高さでブロックをファイナライズできる投票セットがちょうど 1 つであることが保証される。このプロパティにより動的な投票セットが可能になる。

各 **Symbol** ノードはブロックツリーではなく単一のブロックチェーンを格納するため、prevote メッセージには単一のハッシュではなくハッシュチェーンが含まれる。ハッシュチェーンの集約により、**Symbol** は仮想ブロックツリーを再構築し、見えるブランチと見えないブランチの両方に投票を適用できる。

概念的に、投票者は高さごとに 1 票を持つ。実質的に、prevote ハッシュチェーン内のすべてのハッシュにウェイトで投票する。

⁵²実際には、*finalization:maxHashesPerPoint* は *finalization:prevoteBlocksMultiple* よりもはるかに大きいことが期待される。さらに、*network:votingSetGrouping* は *finalization:prevoteBlocksMultiple* の倍数であるべきである。

15.5.2 Precommit

次に、投票者は $g(V_{r,v}) \geq E_{r-1,v}$ になるまで待つ。 $2 * \text{stepDuration}$ （ラウンド開始からの相対時間）が経過したか r が完了可能になったとき、 v は precommit を送信する。

投票者は潜在的にファイナライズできる最良のブロックを決定する。 $g(V_{r,v})$ に対応する単一のハッシュを持つ precommit メッセージを作成する。

概念的に、投票者は高さごとに 1 票を持つ。実質的に、最後にファイナライズされたブロックと precommit ハッシュの間のすべてのハッシュにウェイトで投票する。

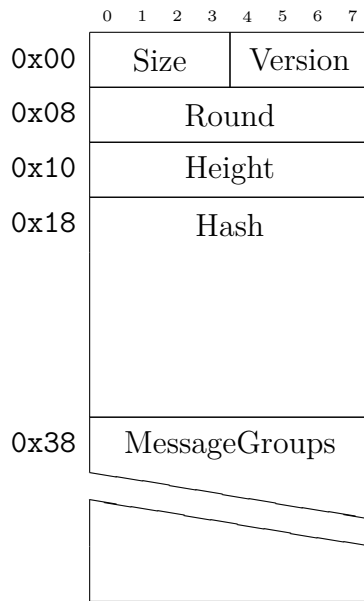
15.5.3 Commit

非同期に、投票者はラウンド r の prevote と precommit メッセージを収集する。実際には、これらのメッセージは現在のラウンドまたは前のラウンドのいずれかに関連付けられる。 $g(C_{r,v})$ が変化すると、そのブロックとそのブロックとローカルファイナライズ済みブロックの間のすべてのブロックがファイナライズされる。

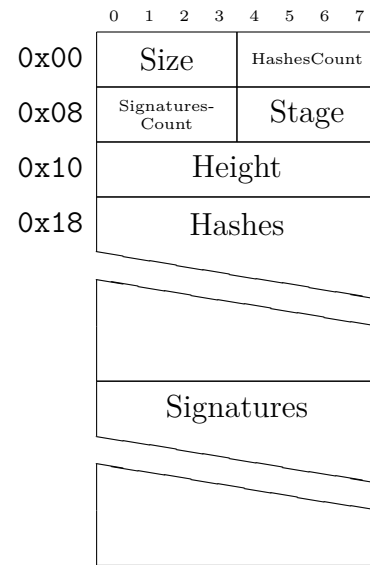
エポック e とポイント p を持つファイナライゼーションラウンドが与えられると、ほとんどの場合、次のラウンドは $(e, p+1)$ になる。例外はエポック e の最後のブロックがファイナライズされた場合である。その場合、次のラウンドは $(e+1, 0)$ になる。 $(e, p+1)$ と $(e+1, 0)$ の両方が開始される可能性があることに注意されたい。そのシナリオでは、 $(e+1, 0)$ が最終的に優勢になり、 $(e, p+1)$ は完了しない。

15.6 証明

ネットワークトラフィックを最小限に抑えるために、非投票者は個別の prevote または precommit メッセージを送受信しない。代わりに、これらのノードは *finalization: unfinalizedBlocksDuration* に基づいてネットワークから定期的にファイナライゼーション証明を取得して個別に検証する。この設定がゼロの場合、証明は各エポックの終わりにのみ取得される。この設定がゼロ以外の場合、最後のファイナライズ済みブロックが最後のファイナライズされていないブロックより少なくとも *finalization: unfinalizedBlocksDuration* 遅れている場合、証明は楽観的に取得される。検証後、これらの非投票ノードは証明エポックまでのすべてのエポックをファイナライズする。



(a) 証明ヘッダー



(b) メッセージグループ

Figure 36: ファイナライゼーション証明のレイアウト

ファイナライゼーション証明はヘッダーとメッセージグループのコレクションで構成される。ヘッダーは証明によってファイナライズされた最後のブロックを示す⁵³。このブロックはその高さで一意に識別される。ヘッダーはブロックがファイナライズされたファイナライゼーションラウンドも示す。

メッセージグループは証明を暗号的に検証するファイナライゼーションメッセージの集約である。Signature のみが異なるすべてのファイナライゼーションメッセージは、すべての署名が付加された単一のメッセージグループにグループ化される。証明には常に少なくとも 2 つのメッセージグループ（prevote 用と precommit 用）があるが、投票のカウント方法によってさらに多くなる可能性がある。

ファイナライゼーション証明の検証には、そのエポックに関連付けられた投票セットの知識が必要である。重要なことに、適格な投票者とそのウェイトが知られている必要がある。証明を検証するには、適格な投票者からの有効なサポートメッセージのみを含む必要がある。さらに、メッセージは証明ヘッダーで指定されたブロックが $g(C_{r,v})$ であり、 $g(V_{r,v}) \geq g(C_{r,v})$ であることを示す必要がある。

⁵³技術的には、証明は最後のブロックとそのすべての祖先をファイナライズし、それらは単一のチェーンを形成することが保証される。

15.7 シビル攻撃

確定的ファイナライゼーションが有効な場合、投票者間のシビル攻撃はすべての投票をアカウント残高で重み付けすることによって防止される。攻撃者がより多くの投票力を得る唯一の方法はより多くのステークを取得することである。アカウントの残高を複数のアカウントに分割しても全体の投票力は変わらない。

15.8 ステークなし攻撃

確定的ファイナライゼーションが有効な場合、マーチャントがブロックがファイナライズされるまでサービスを提供するのを待てば、ステークなし攻撃を防ぐことができる。マーチャントが待たない場合、この攻撃に対する追加の防御がある。

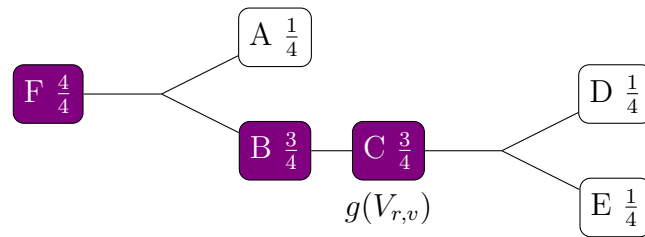
最も重要なことに、攻撃者がより良いチェーンを生成できる時間は限られている。攻撃者はネットワークがロールバックしたいブロックをファイナライズする前に、より良いチェーンを生成しなければならない。さらに、難易度の大幅な低下に必要な時間は、特定のチェーン部分をファイナライズするのに必要な時間よりも長い可能性が高い。

15.9 例

すべての例で、4 人の均等に重み付けされた投票者を持つネットワークを考える。超多数には 4 人の投票者のうち少なくとも 3 人が同じハッシュに投票することが必要である。 F は常に最後にファイナライズされたブロックを指す。ブロック B に対してキャストされた prevote メッセージは F から B までのハッシュ（両端含む）で構成される prevote ハッシュチェーンを意味する。

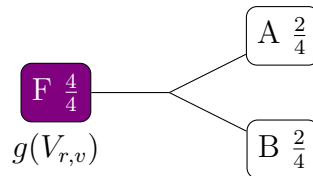
例 1

prevote メッセージが A 、 C 、 D 、 E に対してキャストされる。 C は D と E のブランチ上にあるため、1 人の投票者しか明示的に投票しなかったにもかかわらず超多数の票を持つ。



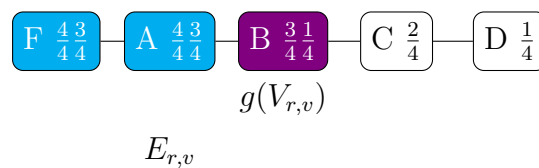
例 2

ネットワークが均等サイズの 2 つのパーティションに分割されていると考える。prevote メッセージが A、A、B、B に対してキャストされる。F は A と B のブランチ上にあるため、どの投票者も明示的に投票しなかったにもかかわらず超多数の票を持つ。重要なことに、F はすでにファイナライズ済みであるため、新しいブロックはファイナライズされないことに注意されたい。



例 3a

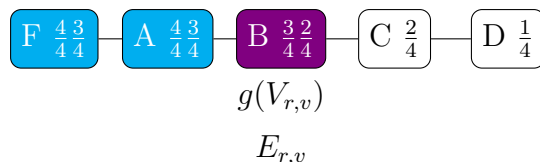
prevote メッセージが A、B、C、D に対してキャストされ、B が prevote の超多数を持つ。2 人の投票者が prevote A、B、C を見て A に precommit を送信すると仮定する。1 人の投票者が prevote B、C、D を見て B に precommit を送信すると仮定する。投票者は未知の precommit のウェイト (25%) が B の precommit の超多数を引き起こせないと判断できる。B は 25% の precommit しか持たない。これは [15.5: アルゴリズム](#) の条件 1 を満たす。



例 3b

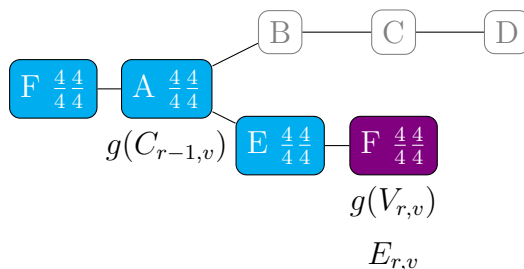
prevote が異なる順序で受信される前の例のわずかな変形を考える。上記と同様に、prevote メッセージが A、B、C、D に対してキャストされ、B が prevote の超多数を

持つ。2 人の投票者が prevote B 、 C 、 D を見て B に precommit を送信すると仮定する。1 人の投票者が prevote A 、 B 、 C を見て A に precommit を送信すると仮定する。投票者は未知の precommit のウェイト (25%) が C の precommit の超多数を引き起こせないと判断できる。 C は 0% の precommit しか持たない。これは 15.5: アルゴリズム の条件 2 を満たす。



例 4

A がポイント $r-1$ でコミットされたが B はコミットされていない前の例の拡張を考える。投票者は次のポイント r に進んだ。4 つの prevote メッセージと 4 つの precommit メッセージが F に対してキャストされる。 $r-1$ での B への precommit が A をコミットするために使用されたにもかかわらず、ブロック B 、 C 、 D はすべてメインチェーンからプルーニングされた。 $r-1$ での証明を検証するには、 B が A の子孫であるという記録が必要である。この情報は prevote メッセージハッシュチェーンには格納されるが、precommit メッセージには格納されない。これがファイナライゼーション証明の検証に prevote が必要な理由である。



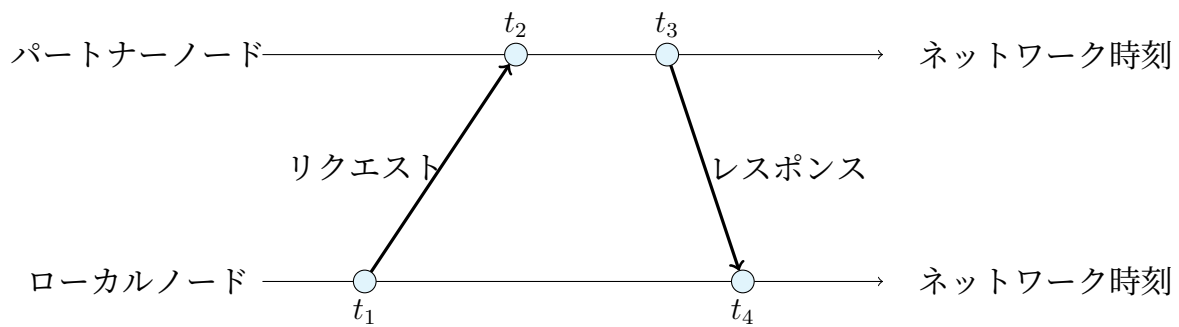


Figure 37: 時刻同期フロー

16 時刻同期

“

時は静かに、しかし絶えず流れる。

”

- Geoffrey Chaucer

ALLすべてのネットワーク参加者が整合性のある時計を持つことは重要である。なぜなら、トランザクションには有効期限があり、ブロックにはタイムスタンプがあるからである。ノードの時計は現実の時計から少し離れた時刻を指すことがある。そのため、すべての参加者間で時計のずれを最小化するためのメカニズムが必要となる。

このアルゴリズムは以下の手順で実施される：

1. サンプルの収集
2. 不良データを除去するためのフィルターの適用
3. 有効オフセットの計算
4. 結合と閾値

16.1 サンプル収集

時刻同期のサンプルを収集するため、ローカルノードは高インポートランスのアカウントを持つランダムなリモートノードのサブセットに通信を試みる。

このプロセスは次のように機能する：

1. ローカルノードはリモートノードに通信送信タイムスタンプ T_1 を送信する。

-
2. リモートノードはリクエストを受信し、リモートノードのローカル時刻でタイムスタンプ T_2 を取得する。
 3. リモートノードはリクエストを処理し、リモートノードのローカル時刻でタイムスタンプ T_3 を取得する。
 4. リモートノードは T_2 と T_3 をローカルノードに返す。
 5. ローカルノードはレスポンスを受信し、ローカルノードのローカル時刻でタイムスタンプ T_4 を取得する。

T_1 から T_4 を使用して、次のように計算できる：

$$rtt = (T_4 - T_1) - (T_3 - T_2) \quad (19)$$

$$offset = \frac{(T_2 - T_1) + (T_3 - T_4)}{2} \quad (20)$$

- rtt は往復時間（Round Trip Time）であり、ネットワーク遅延の尺度である。
- $offset$ はリモートノードとローカルノードの時計の差の推定値である。

オフセットが正の場合、リモートノードの時計はローカルノードの時計より進んでいる。逆に、オフセットが負の場合、リモートノードの時計はローカルノードの時計より遅れている。

16.2 不良データを除去するためのフィルタリング

時刻同期のサンプルには異常値が含まれる場合がある。これはネットワークの遅延、変動、またはノードが悪意ある挙動を示す場合に発生しうる。そのため、以下のフィルターが適用される：

1. 往復時間フィルター：最小往復時間より X 倍以上の往復時間を持つサンプルは除外される。`timesync:maxNodes` で設定された最大ノード数でも除外が発生することがある。
2. クランプフィルター： $|offset|$ が rtt の半分より大きいサンプルは除外される。そのようなサンプルは信頼性が低いとみなされる。
3. スケーリングフィルター：サンプルを昇順に並べ替え、上下 25% のサンプルを除外する。残りのサンプルだけが使用される。

16.3 有効オフセットの計算

フィルタリング後、残ったサンプルに基づいて加重平均を計算する。サンプルの収集元であるアカウントのインポートانس I は重みとして使用される。複数のサンプルが同じインポートانسのアカウントから収集された場合、それらは追加されて合計される。

n 個のサンプルが残ったとすると、加重平均は：

$$scale = \sum_{i=1}^n I_i \quad (21)$$

$$effectiveOffset = \frac{\sum_{i=1}^n I_i \cdot o_i}{scale} \quad (22)$$

ここで I_i はサンプル i の重み（インポートانس）、 o_i はサンプル i のオフセットである。

16.4 結合と閾値

時刻同期ラウンドから得られる有効オフセットは通常、有効なオフセット全体を反映していない。これは一部のノードが到達できない可能性があるためである。

正確な同期を実現するために、以下の数式が各ラウンドで適用される：

$$offset_{i+1} = offset_i + coupling \cdot effectiveOffset_i \quad (23)$$

$coupling$ はノードがオフセット変化に対してどれだけ迅速に反応するかを制御し、0 から 1 の間の値である。 $coupling$ の値が高いほど、ノードはより速くオフセットに反応するが、それは一部の不良データがシステムに影響を与えやすくなることも意味する。 $coupling$ の値が低いほど、ノードはより安定するが収束が遅くなる。

さらに、ノードの現在の時刻オフセットが *timesync:maxTimeBehindScheduleCompensation* と *timesync:maxTimeAheadScheduleCompensation* で定義された許容範囲を超えると、ノードはそれ以上更新されなくなる。

17 メッセージング

“

良いメッセージは必ずメッセージャーを見つける。

”

- Amelia Barr

BLOCKCHAINブロックチェーンクライアントアプリケーションはブロックチェーンのデータを取得してユーザーに表示する。これらのクライアントが最大限に役立つためには、常に最新のブロックチェーンデータを提示し、表示されているデータが変更されるたびにユーザーインターフェースを更新すべきである。単純なクライアントは、REST サーバーやローカルデータベースにブロックチェーンデータを定期的にポーリングすることができる。これは必要以上のネットワーク帯域幅やその他のリソースを使用するため、非効率的である。代わりに、**Symbol** はクライアントが単一のメッセージキューを通じてデータ変更をサブスクライブできるようにする。

17.1 メッセージチャンネルとトピック

クライアントに公開される **Symbol** メッセージキューは複数のチャンネルをサポートする。各チャンネルには一意のトピックがある。トピックは常に、受信されるメッセージの種類を示すトピックマーカで始まる。場合によっては、マーカの後に追加フィルタリングに使用される未解決アドレスが続く。クライアントは通常すべての種類のブロックチェーン状態変化に関心があるわけではないため、利用可能なトピックのサブセットをサブスクライブできる。Figure 38 にサポートされるすべてのトピックマーカを示す。

トピックマーカ名	トピックマーカ
ブロック	0x9FF2D8E480CA6A49
ブロック削除	0x5C20D68AEE25B0B0
ファイナライズ済みブロック	0x4D4832A031CE7954
トランザクション	0x61
未確認トランザクション追加	0x75
未確認トランザクション削除	0x72
パーシャルトランザクション追加	0x70
パーシャルトランザクション削除	0x71
トランザクションステータス	0x73
コサイネチャ	0x63

Figure 38: トピックマーカ

17.2 接続とサブスクリプション

メッセージングのサポートは `zeromq` エクステンションによって追加される。ノードがメッセージングをサポートしたい場合、このエクステンションをブローカープロセスで有効化する必要がある。エクステンションはブロックおよびトランザクション関連イベント（[2.2: Symbol エクステンション](#) を参照）のサブスクライバーを登録し、それらのイベントをメッセージキューのメッセージにマッピングする。有効化されると、ブローカーは新しいサブスクライバーのためにポート `messaging:subscriberPort` でリッスンする。クライアントは接続して 1 つ以上のトピックのメッセージキューをサブスクライブできる。

17.3 ブロックメッセージ

ブロックメッセージのトピックはトピックマーカのみで構成される。すべてのブロックメッセージのレイアウトは [Figure 39](#) に示す。サポートされるブロックメッセージは以下の通りである：

- ブロック: 新しいブロックがチェーンに追加された。
- ブロック削除: 指定された高さ以降のブロックが削除された。
- ファイナライズ済みブロック: ブロックがファイナライズされた。確定的ファイナライゼーションが有効な場合のみ公開される。

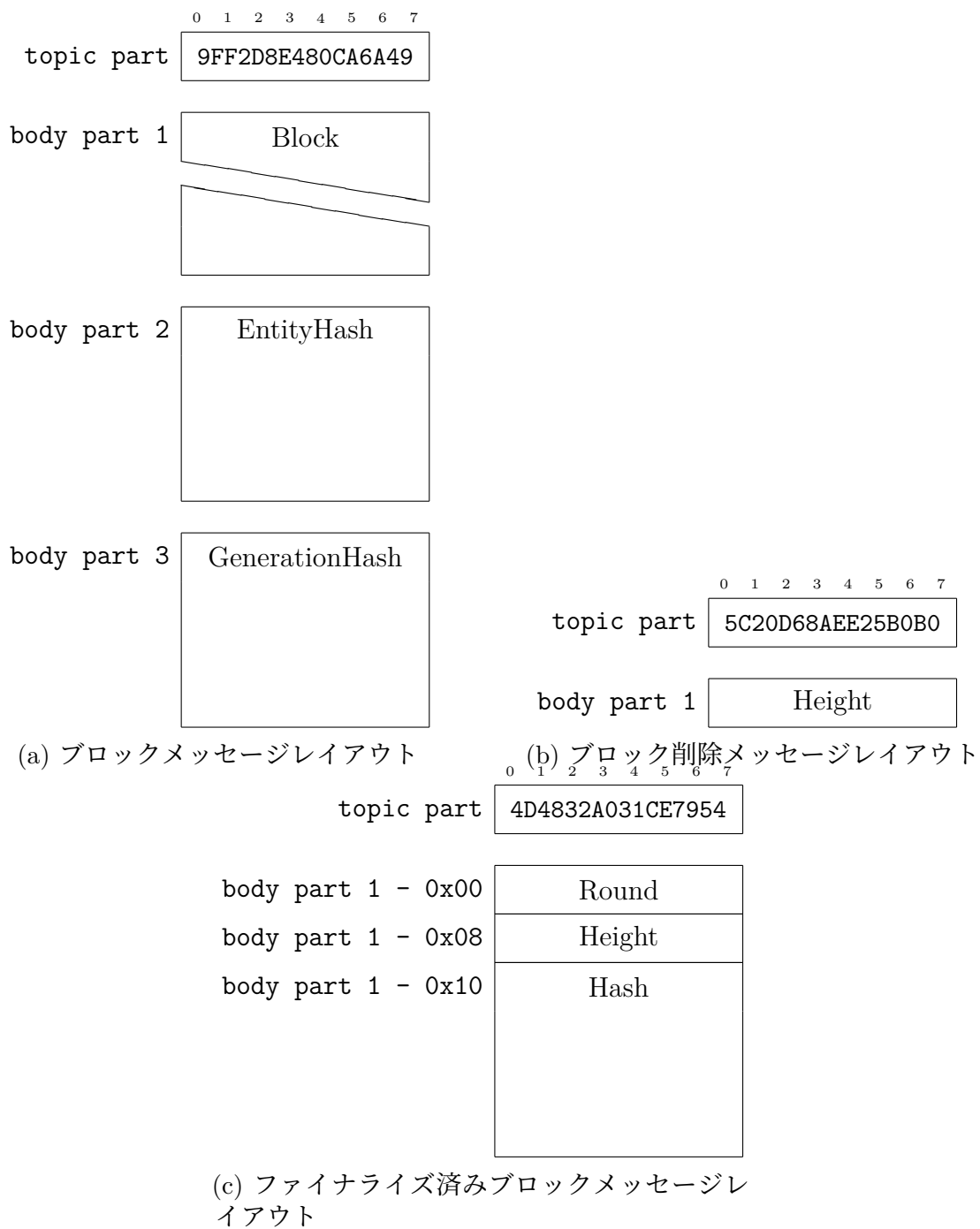
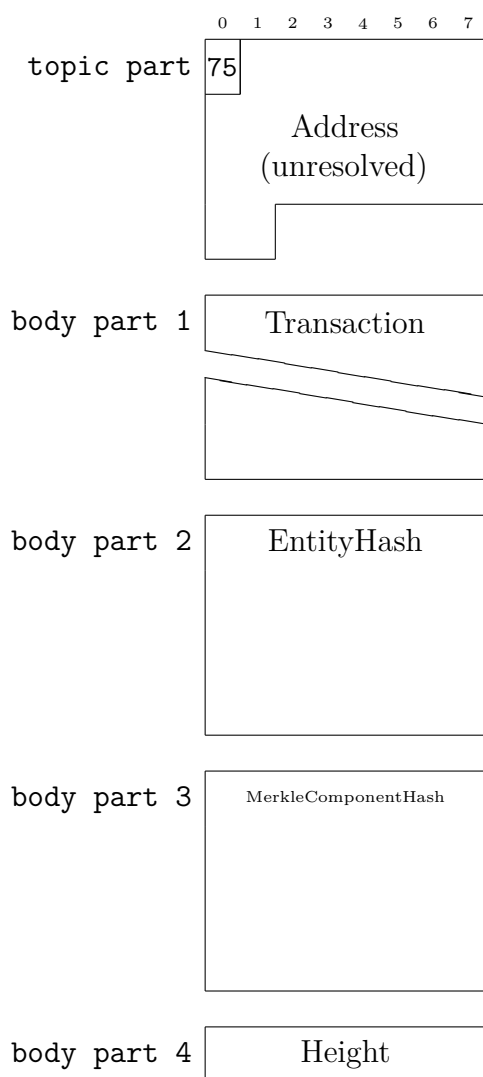


Figure 39: ブロック関連メッセージ

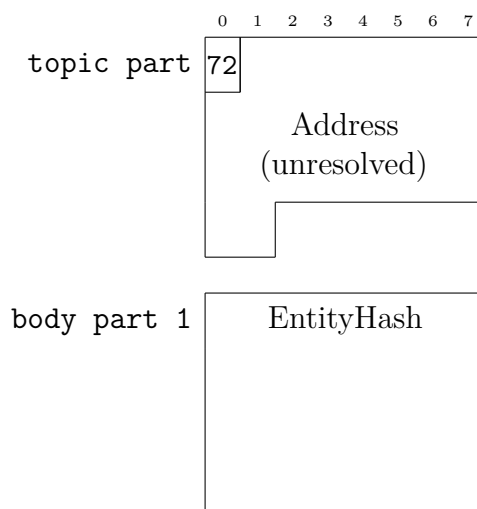
17.4 トランザクションメッセージ

トランザクションメッセージのトピックはトピックマーカとオプションの未解決アドレスフィルターの両方で構成される。未解決アドレスフィルターが指定された場合、指定された未解決アドレスが関係するメッセージのみが発行される。例えば、転送トランザクションのメッセージは、指定された未解決アドレスが転送の送信者または受信者である場合にのみ発行される。未解決アドレスフィルターが指定されない場合、すべてのトランザクションのメッセージが発行される。すべてのトランザクションメッセージのレイアウトは [Figure 40](#)、[Figure 41](#)、[Figure 42](#) に示す。サポートされるトランザクションメッセージは以下の通りである：

- トランザクション: トランザクションが確認された、つまりブロックの一部となった。
- 未確認トランザクション追加: 未確認トランザクションが未確認トランザクションキャッシュに追加された。
- 未確認トランザクション削除: 未確認トランザクションが未確認トランザクションキャッシュから削除された。
- パーシャルトランザクション追加: パーシャルトランザクションがパーシャルトランザクションキャッシュに追加された。
- パーシャルトランザクション削除: パーシャルトランザクションがパーシャルトランザクションキャッシュから削除された。
- トランザクションステータス: トランザクションのステータスが変化した。

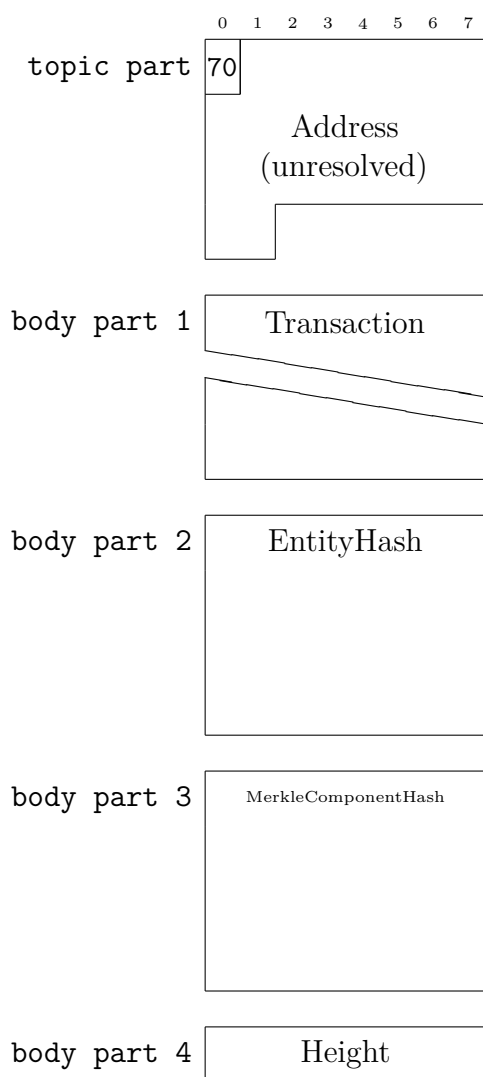


(a) 追加メッセージ

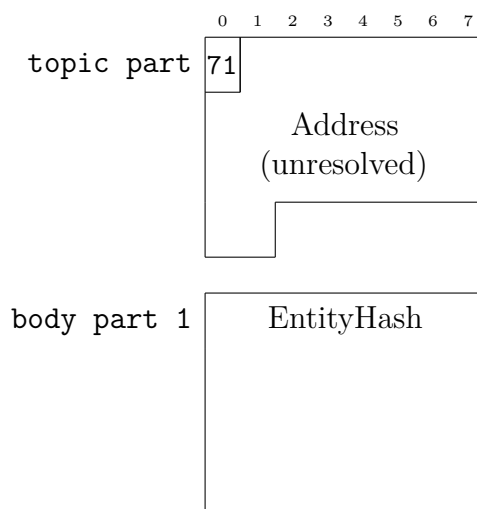


(b) 削除メッセージ

Figure 40: 未確認トランザクション関連メッセージ



(a) 追加メッセージ



(b) 削除メッセージ

Figure 41: パーシャルトランザクション関連メッセージ

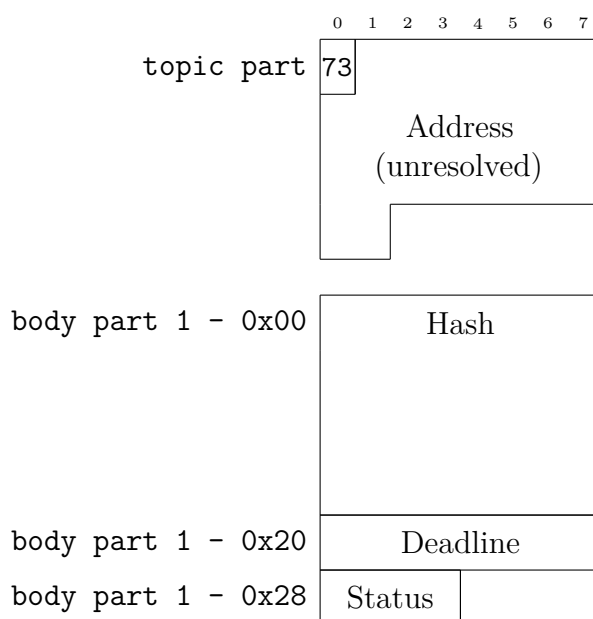


Figure 42: トランザクションステータスメッセージ

17.4.1 コサイネチャメッセージ

コサイネチャメッセージのトピックはトピックマーカとオプションの未解決アドレスフィルターの両方で構成される。メッセージは、アグリゲートトランザクションの新しいコサイネチャがパーシャルトランザクションキャッシュに追加されたときに、サブスクライブされたクライアントに発行される。未解決アドレスフィルターが指定された場合、指定されたアドレスが関係するアグリゲートトランザクションのメッセージのみが発行される。それ以外の場合、すべての変更のメッセージが発行される。コサイネチャメッセージのレイアウトは [Figure 43](#) に示す。

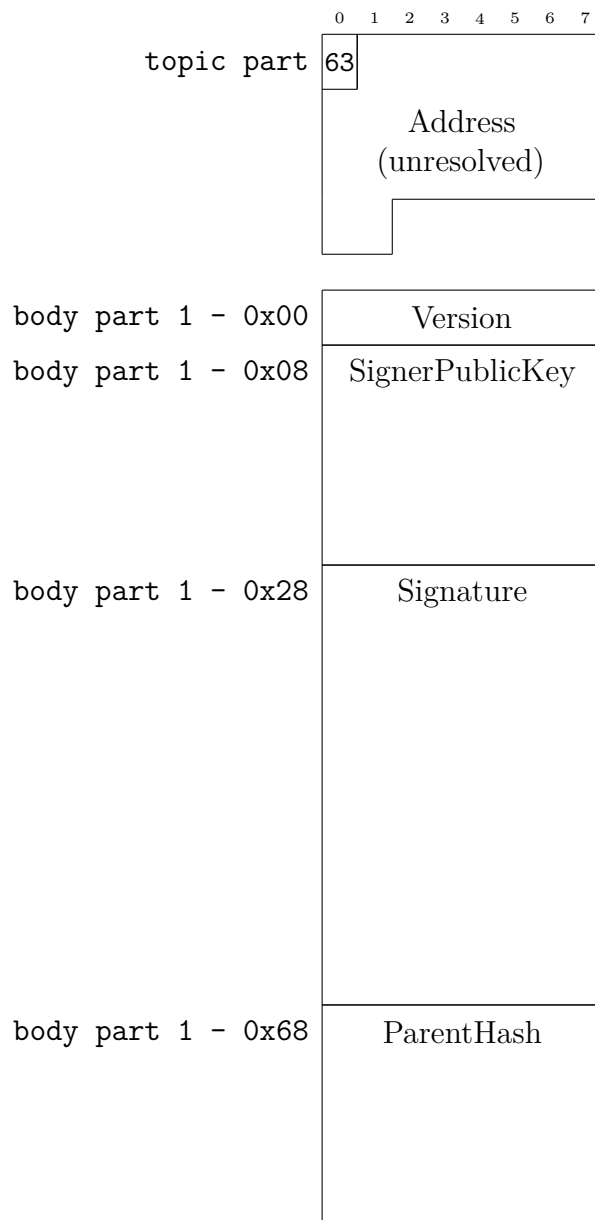


Figure 43: コサインチャメッセージ

References

- [BCN13] BCNext. *Whitepaper:Nxt*. Web document. 2013. URL: <https://nxtwiki.org/wiki/Whitepaper:Nxt#Blocks>.
- [Ber+11] Daniel J. Bernstein et al. “High-Speed High-Security Signatures”. In: *Cryptographic Hardware and Embedded Systems - CHES 2011 - 13th International Workshop, Nara, Japan, September 28 - October 1, 2011. Proceedings*. 2011, pp. 124–142. DOI: [10.1007/978-3-642-23951-9_9](https://doi.org/10.1007/978-3-642-23951-9_9). URL: http://dx.doi.org/10.1007/978-3-642-23951-9_9.
- [BG17] Vitalik Buterin and Virgil Griffith. *Casper the Friendly Finality Gadget*. 2017. eprint: [1710.09437](https://arxiv.org/abs/1710.09437). URL: <http://arxiv.org/abs/1710.09437>.
- [BM99] Mihir Bellare and Sara K. Miner. “A Forward-Secure Digital Signature Scheme”. In: *Advances in Cryptology - CRYPTO ’99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*. Springer, 1999, pp. 431–448. DOI: [10.1007/3-540-48405-1_28](https://doi.org/10.1007/3-540-48405-1_28).
- [CL99] Miguel Castro and Barbara Liskov. “Practical Byzantine Fault Tolerance”. In: *Proceedings of the Third Symposium on Operating Systems Design and Implementation*. USENIX Association, 1999, pp. 173–186. ISBN: 1880446391. DOI: [10.5555/296806.296824](https://doi.org/10.5555/296806.296824).
- [Gol+20] Sharon Goldberg et al. *Verifiable Random Functions (VRFs)*. Internet-Draft draft-irtf-cfrg-vrf-07. Work in Progress. Internet Engineering Task Force, June 2020. 39 pp. URL: <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-vrf-07>.
- [KN12] Sunny King and Scott Nadal. *PPCoin: Peer-to-Peer Crypto-Currency with Proof-of-Stake*. Web document. Aug. 2012. URL: <https://decred.org/research/king2012.pdf>.
- [Mer88] R. C. Merkle. “A Digital Signature Based on a Conventional Encryption Function”. In: *Advances in Cryptology — CRYPTO ’87*. 1988, pp. 369–378.
- [Mor68] Donald R. Morrison. “PATRICIA - Practical Algorithm to Retrieve Information Coded in Alphanumeric”. In: *Journal of the ACM*, *15*(4). 1968, pp. 514–534.
- [Nak09] Satoshi Nakamoto. *Bitcoin: A peer-to-peer electronic cash system*. 2009. URL: <http://www.bitcoin.org/bitcoin.pdf>.
- [SK20] Alistair Stewart and Eleftherios Kokoris-Kogia. *GRANDPA: a Byzantine Finality Gadget*. July 2020. arXiv: [2007.01560](https://arxiv.org/abs/2007.01560) [cs.DC].

Index

- address
 - decoded, 22
 - encoded, 22
- alias address, 25
- block
 - difficulty, 44
 - fee multiplier, 46
 - generation, 45
 - generation hash, 48
 - hit, 48
 - nemesis, 35
 - score, 45
 - synchronization, 53
 - target, 48
 - time smoothing, 49
- consumers, 57
 - block, 60
 - common, 58
 - transaction, 61
- Ed25519, 11
- generation hash, 48
- key
 - private, 11
 - public, 11
- messaging
 - channels, 107
 - messages, 108
- node, 70
 - banning, 78
 - dynamic, 74
 - selection, 77
 - static, 70
- plugins, 3
- reputation
 - connection management, 76
 - node banning, *see* node, banning
 - node selection, *see* node, selection
- score
 - importance, 82
 - node, 83
 - stake, 82
 - transaction, 82
- signature
 - BM tree, 97
 - malleability, 12
 - voting, 15
- sybil attack, 84
- synchronization, 53
- TLS, 71
- tree
 - branch, 19
 - leaf, 19
- unconfirmed transactions
 - cache, 63
 - spam throttle, 63
- VRF, 13
- エクステンション, 5
- ネメシスブロック, 35
- ハーベスティング, 45
- ファイナライゼーションエポック, 95
- ファイナライゼーションポイント, 95
- ファイナライゼーションラウンド, 95
- フィンガープリント, 2
- 初期難易度, 44
- 目標ブロック時間, 35
- 署名, 12